

单位代码	10476
学号	2108183009
分类号	TP393

河南师范大学

硕士学位论文

基于 Lyapunov 优化的感知数据协作卸载机制研究

学科、专业：计算机科学与技术
研究方向：分布式网络实时计算
申请学位类别：理学硕士
申请人：李茗
指导教师：袁培燕 教授

二〇二四年五月

RESEARCH ON COOPERATIVE
OFFLOADING MECHANISM OF SENSING
DATA BASED ON LYAPUNOV
OPTIMIZATION

A Dissertation Submitted to
the Graduate School of Henan Normal University
in Partial Fulfillment of the Requirements
for the Degree of
Master of Science

By

Ming Li

Supervisor: Prof. Peiyan Yuan

May, 2024

摘要

在 6G 及各种新兴高科技技术（车对万物通信、沉浸式扩展现实和人类数字孪生）的快速发展过程中，计算密集型与延迟敏感型应用程序服务的出现，实时在线计算的需求越来越多。移动边缘计算（MEC）作为一种物联网的重要计算范式，满足了移动设备日益增长的计算需求。在实时物联网系统中，无线信道的状态、用户的位置、数量以及数据生成过程是高度动态和随机的，导致了系统负载的不均衡、不可靠数据传输和流量激增等问题，因此保持系统稳定性是系统运行的前提。目前现有的动态卸载系统在稳定性方面仍存在许多挑战。主要包括：（1）实时设备优先级：由于设备计算能力有限，需要服务器与设备进行协同实时处理动态数据。然而，在部分卸载任务过程中，服务器忽略了实时优先选择关联时延敏感的应用程序服务相关的设备，引起了更长的响应时间。（2）多边缘服务器之间的流量不均衡：由于服务器的计算资源有限，设备通过上行链路上传大量数据导致网络拥塞。导致数据传输延迟增加，系统的吞吐量下降。（3）多变量决策通信场景：通信场景的复杂性增加了计算卸载的复杂度，使得传统的 Lyapunov 优化框架无法解决系统长期多变量卸载决策问题。此外，传统的 Lyapunov 优化框架存在固有的系统瞬时贪婪局限性。这意味着在长期和全局的情况下无法做出最优的卸载决策。本文基于边缘计算的数据卸载问题，将 Lyapunov 优化作为基准，对终端设备感知数据的卸载模式以及决策方式进行研究。本文主要研究工作如下：

（1）针对单小区 MEC 计算密集型任务的在线协作卸载问题，提出了一种在线联合部分卸载和设备优先级的算法（PODP）。首先，针对物联网设备，创建队列模型来表示其工作负载积压来反映系统稳定性。然后，基于 Lyapunov 优化，获得最优 CPU 计算速率以及计算出设备优先级，进而根据物联网设备计算能力与当前系统性能确定任务卸载量和卸载位置，完成资源分配。最后，通过理论分析以及大量的实验结果，表明算法在满足动态任务到达率下的同时，降低了系统能耗。

（2）针对边缘层多边缘服务器协作卸载过程中的缓冲区队列稳定性对系统性能影响的问题，提出了一种能够有效降低系统成本的在线动态服务卸载算法（LDSO）。首先，本文提出了一种边缘节点多跳协作服务数据传输模型，并从数据收发代价角度构

建融合能耗与时延的成本模型。另外通过 Lyapunov 优化，为每个边缘服务器维护一个缓冲区队列模型，以实现边缘网络节点队列的稳定性。然后，通过调整控制参数，使系统动态地做出卸载决策，在成本和缓冲区队列积压之间进行权衡，提高边缘服务卸载系统数据处理能力。最后，理论分析以及仿真实验结果证明算法在保证网络稳定的同时有效提高边缘卸载系统的长期效用。

(3) 针对在线多边缘服务器协作计算卸载固有的系统瞬时贪婪局限性以及自适应系统节能控制问题，提出一种基于深度强化学习 (DRL) 改进 Lyapunov 优化的云边协同服务在线卸载策略。首先，本文通过 Lyapunov 优化框架，构建应用程序服务数据队列模型反应系统稳定性，进而推导出系统目标函数能耗上界并制定优化问题。然后根据上界中系统稳定部分构造系统动作和状态空间，形成适用最小化系统能耗的马尔可夫 (MDP) 决策模块。并且通过分析带有奖励折扣因子的强化学习 (RL) 表达，创建基于 DRL 的奖励函数。最后通过 SAC (Soft Actor-Critic) 算法进行自适应云边服务卸载系统策略的训练，大量实验结果表明了系统不需要在每个时隙获得局部最优解，就可以保证队列的稳定性以及降低系统能耗，证明了所提策略的有效性。

关键词：边缘计算；Lyapunov；协作卸载；时变队列；DRL

ABSTRACT

In the rapid development of 6G and various emerging high-tech technologies (vehicle-to-everything communication, immersive extended reality and human digital twin), the demand for computing-intensive and delay-sensitive application services has increased. And the demand for real-time online computing is increasing. Mobile edge computing (MEC), as an important computing paradigm for the Internet of Things, meets the growing computing needs of mobile devices. In a real-time IoT system, the status of the wireless channel, the location and number of users, and the data generation process are highly dynamic and random, leading to problems such as imbalance of system load, unreliable data transmission, and traffic surge, thus maintaining system stability is a prerequisite for system operation. Currently, existing dynamic offloading systems still have many challenges in terms of stability. Mainly include: (1) Real-time device priority: Due to the limited computing power of the device, the server and the device need to collaborate to process dynamic data in real time. However, during partial offloading tasks, the server ignores real-time preference for devices associated with latency-sensitive application services, causing longer response times. (2) Traffic imbalance among multiple edge servers: Due to limited computing resources of servers, devices upload large amounts of data to edge servers through uplinks, causing network congestion. As a result, data transmission delay increases and system throughput decreases. (3) Multi-variable decision-making communication scenario: The complexity of communication scenarios increases the complexity of computational offloading, making the traditional Lyapunov optimization framework unable to solve the long-term multi-variable offloading decision-making problem of the system. In addition, the traditional Lyapunov optimization framework has inherent limitations of system instantaneous greed. This means that optimal offloading decisions cannot be made in the long term and global context. Based on the data offloading problem of edge computing, this paper uses Lyapunov optimization as a benchmark to study the offloading mode and decision-making method of terminal device sensing data. The main works of this article are as follows:

(1) Aiming at the problem of online collaborative offloading of single-cell MEC computing-intensive tasks, an online joint partial offloading and device priority algorithm (PODP) is proposed. First, for IoT devices, a queue model is created to represent its workload backlog and reflect system stability. Then, based on Lyapunov optimization, the optimal CPU computing rate is obtained and the device priority is calculated, and then the task offloading amount and offloading location are determined based on the computing power of the IoT device and the current system performance, and resource allocation is completed. Finally, through theoretical analysis and a large number of experimental results, it is shown that the algorithm reduces system energy consumption while meeting the dynamic task arrival rate.

(2) Aiming at the impact of buffer queue stability on system performance during collaborative

offloading of multiple edge servers at the edge layer, an online dynamic service offloading algorithm (LDSO) is proposed that can effectively reduce system costs. First, this paper proposes an edge node multi-hop collaborative service data transmission model, and constructs a cost model that integrates energy consumption and delay from the perspective of data transmission and reception costs. In addition, through Lyapunov optimization, a buffer queue model is maintained for each edge server to achieve the stability of the edge network node queue. Then, by adjusting the control parameters, the system can dynamically make offloading decisions, weighing costs and buffer queue backlog, and improving the data processing capabilities of the edge service offloading system. Finally, theoretical analysis and simulation experimental results prove that the algorithm can effectively improve the long-term effectiveness of the edge offloading system while ensuring network stability.

(3) In view of the problem of inherent system instantaneous greed limitations of online multi-edge server collaborative computing offloading and adaptive system energy-saving control, an online offloading strategy for cloud-edge collaborative services based on deep reinforcement learning (DRL) improved Lyapunov optimization is proposed. First, this paper uses the Lyapunov optimization framework to build an application service data queue model to reflect system stability, then derive the upper bound of the system objective function energy consumption and formulate optimization problems. Then the system action and state space are constructed according to the stable part of the system in the upper bound, and a Markov (MDP) decision module suitable for minimizing system energy consumption is formed. And by analyzing the reinforcement learning (RL) expression with reward discount factor, a DRL-based reward function is created. Finally, the SAC (Soft Actor-Critic) algorithm is used to train the adaptive cloud edge service offloading system strategy. A large number of experimental results show that the system does not need to obtain the local optimal solution in each time slot to ensure the stability of the queue and reduce the system energy consumption, proving the effectiveness of the proposed strategy.

KEY WORDS: Edge computing, Lyapunov, Collaborative offloading, Time-varying queue, DRL

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.2.1 计算卸载.....	2
1.2.2 Lyapunov 优化.....	3
1.3 边缘物联网典型应用.....	6
1.4 论文主要内容与结构安排	7
1.4.1 主要内容	7
1.4.2 结构安排.....	8
第二章 MEC 中一种基于 LYAPUNOV 优化的在线节能卸载算法	11
2.1 引言.....	11
2.2 系统架构.....	12
2.3 问题建模与制定.....	13
2.3.1 问题建模.....	13
2.3.2 问题形式化.....	15
2.4 LYAPUNOV 在线卸载优化算法	16
2.4.1 Lyapunov 优化框架.....	16
2.4.2 在线卸载计算.....	18
2.4.3 算法理论边界分析.....	20
2.5 仿真结果.....	21
2.5.1 系统稳定性.....	22
2.5.2 系统能耗.....	23
2.5.3 对比实验.....	24
2.6 本章小结.....	24
第三章 一种基于 LYAPUNOV 优化的动态服务卸载算法	27
3.1 引言.....	27
3.2 边缘服务卸载系统架构.....	27
3.3 问题制定与分析.....	28
3.3.1 问题表述.....	28
3.3.2 问题分析.....	30
3.4 基于 LYAPUNOV 优化的动态服务卸载算法 (LDSO).....	33
3.4.1 Lyapunov 优化解决框架.....	34
3.4.2 基于贪心策略的匹配决策.....	36
3.4.3 系统稳定性分析.....	41

3.5 实验.....	42
3.5.1 系统稳定性.....	42
3.5.2 系统控制参数 V 的影响.....	43
3.5.3 卸载系统的效用.....	44
3.6 本章小结.....	45
第四章 基于 DRL 自适应 LYAPUNOV 优化的云边协同服务在线 卸载策略.....	47
4.1 引言.....	47
4.2 边缘云系统模型架构.....	48
4.3 系统建模与问题制定.....	49
4.3.1 系统建模.....	49
4.3.2 基于 Lyapunov 的问题制定.....	53
4.4 基于 DRL 的解决框架.....	55
4.4.1 状态与动作空间设计.....	55
4.4.2 奖励设计.....	57
4.4.3 系统的稳定性.....	57
4.4.4 惩罚项最小化的奖励.....	59
4.4.5 实际预期总奖励.....	64
4.4.6 系统算法实现.....	64
4.5 仿真结果.....	66
4.5.1 系统稳定性.....	67
4.5.2 系统参数 V 影响.....	69
4.5.3 系统高动作维度操作.....	71
4.5.4 比较实验.....	72
4.6 本章小结.....	72
第五章 总结与展望.....	75
5.1 总结.....	75
5.2 展望.....	76
参考文献.....	77

第一章 绪论

1.1 研究背景及意义

随着第六代（6G）移动通信网络技术的成熟和社会通信基础设施的不断增强，网络连接设备数量感知社会能力大幅度提高^[1-6]。物联网设备数量（手机、可穿戴设备、平板电脑、智能车辆等）呈指数级增长^[7-13]，各种新型高科技应用程序，如人脸识别，虚拟现实交互式游戏和自动驾驶，不断涌现，产生了巨大的网络数据流量。数据流量激增导致链路拥塞，增加了网络延迟。传统的云计算模式显然不能满足海量数据实时处理、高安全性与低能耗要求，骨干核心网络面临前所未有的资源供给压力^[14-16]。边缘计算作为一种新型网络服务架构，通过在靠近数据源的边缘侧部署算力中心，提供内容存储、计算和分发等服务，有效缓解了核心网络和回程链路的流量压力，降低了用户请求服务响应时间^[17-22]。更好地满足了物联网终端设备对低延迟和高带宽的需求。

移动边缘计算（MEC）将计算能力从集中式云迁移到分布式边缘服务器^[23-28]。MEC 服务器连接在基站或者无线接入点的边缘，地理位置靠近用户移动设备。通过在边缘服务器进行密集的计算工作，确保了为用户提供低延迟和高质量服务^[29-34]。同时为了保证应用程序服务请求得到及时处理以及任务在线计算卸载的连续性，仅靠边缘服务器的实时决策以及处理是不够的。因此多边缘服务器在线协作计算卸载方式引起了研究人员的关注。其中 Lyapunov 优化与 RL 的结合是目前研究热点。RL 的目标是学习一种能够使累积的预期回报最大化的策略^[35-40]，因此从长期系统性能考虑，不仅提高了系统实时算力，也降低了服务响应延迟。然而，目前在实时物联网系统中，网络状态是高度动态随机的，例如无线信道的时变性、不断增长的服务请求等，这会直接影响数据的传输能耗，即在不良网络连接下进行传输会消耗更多的能量^[41-47]。因此，在与动态环境不断地实时交互过程中，有效的在线计算卸载策略对降低系统能耗显得尤为重要。

在保证边缘卸载系统稳定的前提下，进行在线性能评估以及研究其系统吞吐量，

是支撑实时动态物联网系统优化的理论前提。高效的在线数据协作卸载模型以及算法设计不仅提高系统计算能力、降低网络延迟，也缓解了骨干网压力。本文通过设计多设备-单服务器，多设备-多边缘服务器，多设备-多边缘服务器-云服务器三种通信场景下的在线数据卸载模型及算法或策略，在保证系统稳定性的前提下，使性能指标得到在线优化。

1.2 国内外研究现状

本文主要从卸载角度出发对边缘物联网实时动态系统数据协作处理机制进行研究。同时在保证系统长期稳定基础之上在线优化时延、能耗等性能指标。下面从协作计算卸载、Lyapunov 优化两个方面，就国内外研究现状情况进行介绍。

1.2.1 计算卸载

计算卸载是一种关键的物联网数据处理技术，对于提高用户体验和确保系统性能至关重要。它主要包括卸载模式、资源分配和决策方式三个部分。在该过程中，设备端动态生成数据并将其上传到连接在基站或无线接入点的 MEC 服务器，或者直接上传到云中心。这涉及到数据量的大小以及数据在本地、边缘服务器或云中心的执行位置选择。此外，协作场景的卸载模式选择和带宽资源分配的时间也是为用户提供低延迟和高质量服务的保证。

目前的边缘计算服务卸载根据其协作模式可以分为水平协作与垂直协作。水平协作卸载指的是边缘层多个节点，共同处理用户/设备的服务请求，又称为多边缘服务器协作卸载。垂直协作卸载指的是终端、边缘服务器和云服务器之间通过一定的协商机制联合决定卸载策略^[48-52]。由于可以借助云中心提供的强大算力支持，垂直协作卸载模式适合于延迟不敏感、计算复杂度高的应用场景^[53]。与之不同，水平协作卸载模式更加适合海量数据的分布式处理与低延迟响应，近年来引起了研究人员的广泛关注^[54-57]。

垂直协作单小区内的在线部分卸载属于边端协同数据处理方式，虽然考虑了设备与边缘服务器协作处理关联的应用程序服务，但并未对边端协同数据处理方式中设备生成数据上传顺序进行在线卸载模型的设计。与此同时也出现了云边端协同任务卸载方式，虽然提高了计算能力，然而并不利于延迟容忍的关联设备的应用层序服务请求处理。边缘计算系统虽然能够快速响应用户请求，但在边缘节点协作过程中，边缘系

统处于复杂多变的网络环境中，各种状态信息会随时间动态变化，如时变服务请求等，这使得智能设备端传输到其通信范围内的边缘服务器的工作负载到达率很难准确预测^[58]。同时，进行密集且繁杂的服务请求处理时，多个节点缓冲区服务数据的积压会引起上行链路拥塞，导致带宽资源不足以及网络系统不稳定等问题容易造成服务响应时间过长和大量能量消耗，进而加重系统成本^[59-61]。由于物联网设备数量指数级增长，导致通信场景越来越复杂，考虑到更多应用程序服务的出现和 MEC 有限的计算资源，需要设计新颖的智能卸载决策框架满足海量数据的处理。

综上所述，现有的协作卸载研究方案中忽略了三方面：（1）在线部分卸载框架的设计以及设备优先级问题。（2）边缘层多节点协作卸载过程中的缓冲区队列长期稳定性对系统性能优化的影响。（3）云边系统的瞬时贪婪局限性和多变量组合决策问题。因此为了解决这些问题，考虑资源可用性、工作负载动态到达率的时间波动以及系统长期稳定性等因素，在与动态环境不断地实时交互过程中，保证单小区高效卸载、多边缘服务器之间动态数据流量控制以及云边系统性能在线动态优化，是协作计算卸载技术中面临的难点。

1.2.2 Lyapunov 优化

俄国著名的数学家 Lyapunov 于 1892 年首次提出了动态系统的一般稳定性判据。在 2010 年 M.J.Neely 等人将 Lyapunov 理论进一步改写及其应用在通信随机网络优化领域。Lyapunov 属于自动控制化的数学理论，将其应用到通信和排队系统的随机网络优化场景，既可以保证网络系统长期约束条件下每一时刻的稳定性，也可以使系统目标函数实时优化。Lyapunov 优化框架通过 Lyapunov 函数，Lyapunov 漂移，Lyapunov 漂移+惩罚，对动态系统进行实时控制，以及实现在线性能优化。

近年来，研究人员使用 Lyapunov 优化进行无线网络中的资源分配以及数据卸载等问题的深度研究。下面从基于 Lyapunov 优化的单小区、多边缘服务器协作、云边协同三种通信场景下的数据卸载研究工作进行阐述。

（1）单小区数据卸载

随着设备数量的增加，终端感知的数据量也急剧增加，而设备本身的计算能力远远不足。如果将计算集中在同一节点（设备或者边缘服务器），虽然保证在时间槽内的计算完整性，但是终端工作负载的超量导致了系统不稳定。因此一部分研究人员提出部分卸载框架^[74-77]，部分卸载即终端设备与边缘服务器协同处理产生的数据。

目前在单小区用户端生成的数据部分卸载方案中,文献[68]研究了 MEC 网络中的在线服务卸载问题,解决单槽优化的 NP-hard 问题,并使用随机取整方法来实现服务迁移和请求路由。文献[78]建立了一个虚拟队列模型来描述两层 MEC 网络中物联网设备的工作负载卸载问题,基于 Lyapunov 优化来有效平衡系统的队列积压和能耗。文献[80]研究了多用户 MEC 系统中设备的功耗和计算任务的执行延迟之间权衡的问题,并基于 Lyapunov 优化获得用于设备计算卸载的最优发射功率和带宽分配。文献[81]研究以内容为中心的网络中车辆缓存方案,并将缓存决策问题表述为分数优化模型,利用非线性规划和 Lyapunov 优化理论,推导出理论解来实现最佳的车辆缓存。文献[85]研究了协作任务卸载和数据缓存模型。基于 Lyapunov 优化提出一种联合任务卸载和动态数据缓存策略。文献[86]采用边端协作框架,使计算资源更加靠近用户,考虑了设备端计算能力,并通过组合决策优化,获得最优卸载策略,有效解决了延迟容忍的关联设备的应用层序服务请求处理。文献[69]基于 Lyapunov 优化提出了一种计算卸载策略 (DCCO),研究有效地卸载应用程序,实现卸载成本和系统性能的权衡,以满足用户的优化目标。然而以上的研究工作忽略了上传过程中物联网设备的实时优先级问题。

(2) 多边缘服务器协作数据卸载

当设备产生的任务量足够大的情况下,无论是本地计算还是边缘计算,都将超过节点的工作负荷,造成无线网络系统数据流的不均衡。因此将物联网设备产生的任务在边缘层通过多边缘服务器进行卸载,分担了节点流量压力。然而,在数据通过上行链路传输到连接边缘服务器的小基站时,海量数据的计算使其边缘服务器的工作负载超载,进而导致边缘层网络拥塞。

目前基于 Lyapunov 优化的多边缘服务器协作卸载数据通信场景,文献[67]通过 Lyapunov 漂移和正则化,将非凸优化问题转化为凸优化问题,进而求解包括服务更换、存储和维护三部分的成本。文献[70]采用多服务器协作方式处理数据,并联合服务器存储能力和服务执行延迟的约束来制定优化问题。利用 Lyapunov 优化方法来近似未来期望的系统效用。文献[71]考虑了租用边缘服务器的数据资源进行数据缓存的成本以及数据传输到边缘服务器的迁移成本,并通过 Lyapunov 漂移得到了成本的上限优化。文献[72]为每个基站构建并维护了一个虚拟的服务迁移成本队列,其中成本包括两部分:服务的动态部署和额外的迁移。通过 Lyapunov 优化和 Chernoff 边界理论得到迁移成本和系统稳定性之间的权衡。文献[73]的作者考虑了多个时间片的数据上传和处理的

成本，并通过 Lyapunov 优化和分布式马尔可夫近似方法解决了该成本。文献[82]研究了用户移动场景中卸载服务问题，基于 Lyapunov 优化实现了多用户多服务器物联网系统的能源效率（EE）和服务延迟之间的权衡。

虽然以上文献考虑了延迟与能耗，但是时延是根据利特尔定律将队列长度作为其度量标准，且未考虑多节点协作之间因排队时延引起的流量不均衡问题，另外在具有动态工作负载的物联网系统中，也忽略了每比特数据的能耗。

（3）云边协同数据卸载

由于 MEC 可以有效地降低计算服务的网络延迟，而云端可以提供更强大的计算能力，因此云边协同卸载方式在 Lyapunov 优化的研究工作中引起研究人员极大关注。现有云边协同相关工作主要包括在预测未来信息下的离线优化方法^[62]、预测依赖的优化方法^[63-64]。文献[76]研究了云边协同系统的资源分配和功耗问题。提出云边动态队列模型，并形成漂移加惩罚目标函数来优化系统能耗问题。文献[83]考虑了一个云边端区块链场景，在线选择任务最佳计算位置来最小化能源消耗和任务响应时间。文献[84]联合优化服务缓存和计算卸载，以最大化移动边缘云计算（MECC）中的系统利润，使用正则化和 Lyapunov 优化将问题转化为两个子问题进行求解。以上是基于传统的 Lyapunov 优化进行问题转化，并求出系统渐进最优解。

最近也有很多研究将 Lyapunov 优化与人工智能相结合来解决卸载决策问题。文献[79]提出一种更加具有创造性地新型卸载框架 LyDROO，且结合了 DRL 和 Lyapunov 函数的优点，降低系统延迟。文献[87]研究了跨模式协作通信中的资源重用问题，将网络切片作为可视化资源分配，并将过程建模为 MDP 过程，通过概率约束和 Lyapunov 优化分解问题。文献[88]采用数字孪生网络对拓扑结构和随机任务到达进行建模，并提出了一种异步执行者批评算法来最小化长期能耗。文献[89]基于 DRL 解决了动态 Lyapunov 优化中的计算卸载和传输延迟问题。以上的高级智能决策都是建立在边缘网络，而未考虑到云节点的强大算力。虽然文献[90]利用 DRL 的自适应能力强的优势将其应用场景延伸到云计算，但是忽略了边缘层计算能力。因此，现有的卸载策略大多数只考虑 Lyapunov 优化与 RL 结合的边端场景方面，且很少有研究同时考虑基于 Lyapunov 优化的智能卸载决策和自适应云边节能控制主题。

1.3 边缘物联网典型应用

物联网作为一种互联性强的技术，被广泛应用于智能家居、智能工厂和智能交通等领域。而移动边缘计算是一种将计算和数据存储能力从云端移至网络边缘的技术。它通过在接近终端设备的位置部署计算资源，实现低延迟的数据处理和分析。首先，它能够满足新兴智能高科技应用对超低时延和强大算力资源的需求。其次，提供了更好的数据安全性，因为数据可以在边缘设备上进行处理，减少了数据在传输过程中的风险。此外，还具备实时性、动态性和可扩展性等特点，适用于不同场景和需求。下面对三个典型的物联网应用场景进行汇总。

（1）智能家居

智能家居是物联网技术在家庭环境中的应用，旨在提供更智能化、便捷和舒适的居住体验。在智能家居方面，我们介绍了以下几个关键应用场景和功能：①环境控制：通过智能家居系统，用户可以使用手机应用程序或语音助手控制家中的灯光、温度、空调、窗帘等设备。智能家居系统可以根据用户的偏好和日常行为，自动调整室内环境，提高舒适度和节能效果。②安防监控：智能家居系统可以与安防设备集成，如摄像头、门窗传感器、烟雾报警器等。用户可以通过手机远程监视家中的安全情况，接收警报通知，并采取相应的措施。③能源管理：通过智能插座和电器控制器，用户可以实时监测和控制家中电器的能耗。智能家居系统可以分析能耗数据，提供能源管理建议，帮助用户优化用电计划，节约能源成本。④智能家电：智能家居系统可以与各类家电设备集成，如智能电视、音响系统、冰箱等。用户可以通过智能手机或语音控制设备，实现远程操作和智能化控制，提供更便利的家居体验。然而随着物联网设备数量的增加，伴随着大量感知数据的产生，并且需要及时处理，而传统的云计算已经满足不了用户体验要求，因此将计算资源存储在离用户更近的地方，降低延迟，缓解核心网络的流量压力。

（2）智能工厂

智能工厂是物联网技术在工业生产领域的应用，旨在提高生产效率、降低成本和改善生产过程的可视化和自动化。在智能工厂方面，我们从以下关键应用场景和功能进行阐述。①设备监控和维护：物联网技术可以实时监测设备的状态和性能，收集数据并进行分析，以实现预测性维护和故障预警。通过及时的设备维护和管理，可以减少停机时间，提高生产效率。②生产优化：物联网技术可以实现生产过程的实时监控

和优化。通过传感器和数据分析，可以实时获取生产线上的数据，并根据需求进行调整和优化，提高生产效率和产品质量。③供应链管理：物联网技术可以实现供应链的可视化和实时追踪。通过与供应商、物流公司和客户的连接，可以实时监测物料的运输和库存情况，优化供应链管理，提高供应链的可靠性和效率。④自动化控制：物联网技术可以实现工厂设备和系统的自动化控制。通过连接各类设备和传感器，实现自动化的生产过程，减少人工干预，提高生产效率和安全性。

（3）智能交通

智能交通是物联网技术在交通运输领域的应用，旨在改善交通系统的效率、安全性和便利性。在智能交通方面，探讨以下应用场景和功能：①实时交通监控：物联网技术可以实现实时交通监控和数据收集。通过交通摄像头、传感器和智能监控系统，可以获取道路流量、拥堵情况和交通事故信息，为交通管理部门提供决策支持。②拥堵管理：通过物联网技术，可以实现智能交通信号灯和路况导航系统的协调和优化。根据实时交通数据，可以调整信号灯的时序和优先级，减少网络拥堵现象，提高交通流畅性。③导航和路径优化：物联网技术可以提供个性化的导航和路径优化服务。通过与车辆导航系统的连接，可以根据实时交通信息和用户的需求，提供最佳的导航路径和交通建议，减少行车时间和燃料消耗。④车辆管理和安全：边缘服务器可以实现车辆的远程监控和管理。通过与车辆终端设备的连接，可以实时监测车辆的位置、状态和行驶数据，提供车辆安全性评估和驾驶行为分析，提高道路安全性。

1.4 论文主要内容与结构安排

1.4.1 主要内容

针对当前工作中忽略单小区设备优先级，多边缘服务器之间协作长期稳定性数据卸载和云边智能卸载决策问题，本文首先针对单小区提出了一种在线联合部分卸载和设备优先级的算法（PODP），然后又针对多边缘服务器之间协作数据卸载提出了一种在线动态服务卸载算法（LDSO），实现卸载系统长期稳定与卸载性能优化之间权衡，最后设计了一种基于深度强化学习（DRL）改进 Lyapunov 优化的云边协同服务在线卸载策略。因此本文主要研究内容如下：

（1）根据现有单小区数据协作卸载方案未考虑设备优先级问题，我们研究了联合部分卸载和设备优先级的在线卸载问题。并且构造了物联网设备工作负载积压队列反

映 MEC 系统的稳定性。采用 Lyapunov 优化框架，将本文随机优化问题分解为每一时段确定性子问题。然后通过凸分解和贪心求解子问题，获得最优 CPU 计算速率和确定设备优先级。开发了一种在线能耗优化算法（PODP），保证了系统的稳定性，并且也有效降低系统能耗。同时算法性能边界严格用理论分析。另外大量实验结果证明本文提出方案的有效性。

（2）结合多边缘服务器之间协作数据卸载，我们提出了一种边缘节点多跳协作服务数据传输模型，从数据收发代价角度考虑构建了包含能耗与时延的系统成本模型。创建了边缘服务器缓冲区队列模型，采用 Lyapunov 漂移惩罚函数平衡缓冲区数据队列积压和系统成本，提出了一种基于贪心算法的参数确定匹配策略进行求解，并通过严格的数学分析给出了原问题的上界最优近似解。设计了一种在线动态服务数据卸载算法(LDSO)，该算法无需任何网络先验知识，将时间片上的原始问题解耦，并将其转化为确定性最优上界问题，分析了系统的稳定性。大量实验结果进一步验证了 LDSO 算法的渐进最优性。

（3）针对云边自适应动态环境节能问题，我们提出了一种新颖的云边自适应在线资源管理框架，有效的降低了系统能耗。通过构造状态和动作空间，设计出适合 Lyapunov 优化的 MDP 模块。进而定义了适应 Lyapunov 优化的 RL 目标函数：包含系统稳定和系统能耗（最小化惩罚项）两部分。并通过一阶差分方式设计出与动态环境强交互的奖励函数。实现了从系统长期角度出发，实现最大奖励预期内的全局最优卸载决策。采用 Soft-Actor Critic(SAC)算法，并从系统长期性能角度出发验证本文基于 DRL 设计的卸载策略的有效性以及系统的稳定性。

1.4.2 结构安排

论文总共五章，各章节之间的关系结构如图 1-1 所示。

第一章：绪论介绍了边缘物联网研究背景和意义，针对计算卸载和 Lyapunov 和进行国内外研究现状调研汇总，并介绍了一些通信主要应用场景。进而引出本文主要研究内容。

第二章：研究了单服务器与多设备协作卸载系统节能问题，针对单小区通信场景建模，经过理论计算，获得最优解，通过 MATLAB 实验仿真。

第三章：研究了多边缘服务器之间在长期约束下的动态协作数据卸载问题，通过严格的数学理论分析，分析了系统稳定性，性能指标与吞吐量的变化。

第四章：研究了云边协同自适应数据卸载节能问题，从理论和实验上分析了系统高维度操作下的系统稳定性以及性能指标的变化。

第五章：针对本文研究内容进行总结与展望。

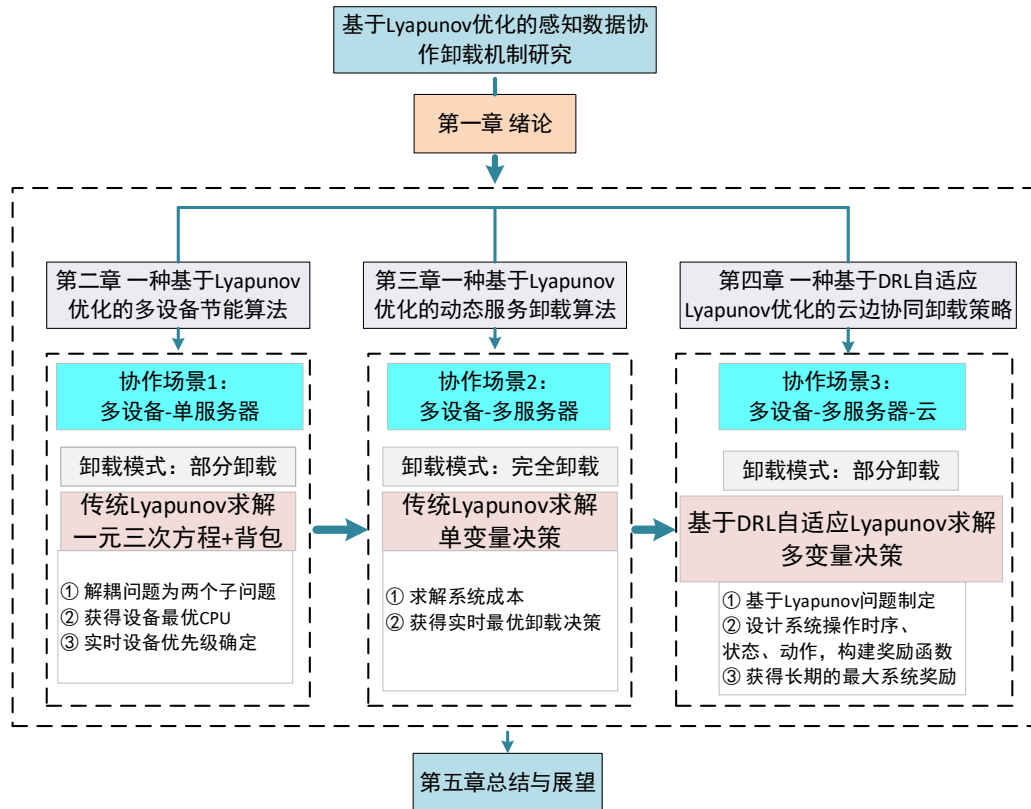


图 1-1 全文组织结构图

第二章 MEC 中一种基于 Lyapunov 优化的在线节能卸载算法

在单蜂窝通信场景中，多个用户设备利用自身的计算能力处理部分数据，并将剩余数据上传至附近的边缘服务器。然而，由于边缘服务器的计算能力有限，因此对设备数据处理顺序的选择变得至关重要。因此，本章考虑了基于设备优先级的在线部分协作卸载方法。首先介绍了单一小区系统的模型架构，然后根据数据卸载过程创建了系统能耗函数。通过 Lyapunov 优化将目标函数解耦为两个子问题，从而获得最优的卸载策略。最后，使用 Matlab 进行仿真，并分析不同系统参数对性能的影响。此外，通过与两个基准方案以及两种最相似算法的比较，验证了该方案在适应动态环境方面的优越性。

2.1 引言

移动边缘计算大大提高了计算能力，但无线网络中不可预测的物联网设备数量带来了新的挑战，另外随着更加智能的应用程序服务的出现^[91]，用户体验的提高远不只是简单地将云计算能力推向网络边缘就可以解决的。因此产生了协同网络数据处理。然而，为了保证物联网设备在不同时刻应用程序卸载的连续性，有效的在线卸载管理方案显得尤为重要。

现有基于 Lyapunov 优化的单小区 MEC 实时物联网系统中，计算卸载仍存在一些挑战。一方面，设备由于硬件原因导致计算能力受限，并且产生的能耗由本地计算和传输以及边缘计算共同决定。因此，如果每个设备的任务计算仅在本地图算，或者仅在边缘服务器计算，都会引起节点工作超载情况发生，继而加大系统能量消耗。另一方面，随着设备数量的增加，大量任务需要在边缘服务器卸载，那么 MEC 服务器首先需要确定并及时处理关联时延敏感应用程序服务的用户设备。因此，设备关联对于卸载非常重要。例如优先选择关联时延敏感的应用程序服务的设备，可以缓解宽带网络(wan)压力和降低系统能量消耗^[92-93]。鉴于此，本文提出了一个基于 Lyapunov 的在线部分卸载框架。该方案考虑了边端协作卸载，设备优先级的确定和系统性能指标评估。

2.2 系统架构

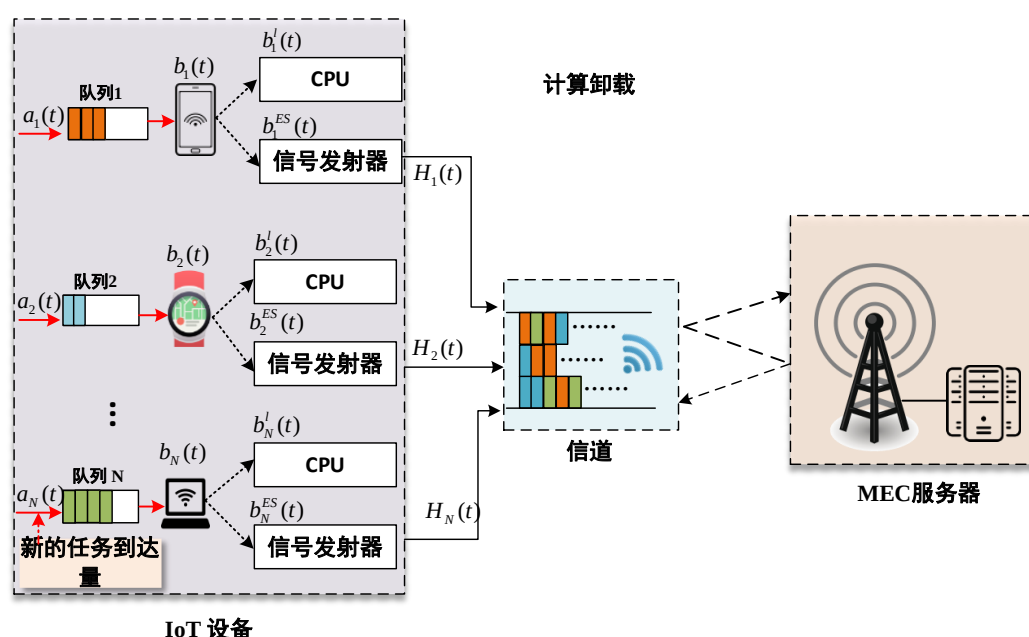


图 2-1 单小区计算卸载系统模架构图

本文应用场景包含 N 台物联网设备，和一个带有 MEC 服务器的 BS，BS 通过同一个局域网(LAN)连接物联网设备，形成一个网络系统。物联网设备通过无线通信发送服务请求，边缘服务器通过有线链路传输并连接到 BS，其中 BS 具有有限的计算资源，然后将其对应的计算任务卸载到相应的服务 BS 进行处理，如图 2-1 所示。定义 $\mathbb{N} = \{1, 2, \dots, N\}$ 为终端物联网设备集合，系统以离散时隙运行为单位，共有 T 个等长时隙部分，每一部分为一个时间槽，长度为 τ ，时间槽的集合记为 $t \in \{0, 1, \dots, T-1\}$ ，在每个时间段开始之前，物联网设备 $i \in \mathbb{N}$ 将产生计算任务量，即 $a_i(t)$ 表示（以比特为单位）。我们将 $a_i(t)$ 作为期望 λ_i 的泊松分布，满足独立且相同的分布。为了更好地为用户提供服务，我们对每个时隙到达的任务（比特位）进行卸载方式划分和处理。因此 IOT 设备的 CPU 可以计算任务，并且附近的 MEC 服务器处理任务请求。MEC 网络模型，并在此模型中提出了一个任务卸载的随机优化问题。

卸载问题中的主要符号如表 2-1 所示。

表 2-1 主要符号

符号	说明
$\mathcal{N}$	IOT 设备集合
V	系统控制参数
C	MEC 边缘服务器最大计算资源数量
L	CPU 计算一个任务（unit bit）所需的周期数
$P_i(t)$	设备 i 的传输功率
$q_i(t)$	设备 i 的工作负载队列积压
e_i^l	设备 i 本地计算能耗
e_i^{tra}	设备 i 传输能耗
e^{ES}	边缘服务器计算能耗
$a_i(t)$	设备 i 的计算任务到达率
$b_i(t)$	设备 i 总的计算任务量
$b_i^l(t)$	设备 i 本地计算任务量
$b_i^{ES}(t)$	设备 i 在边缘服务器计算的任务量
$f_i(t)$	设备 i CPU 的计算速率
$H_i(t)$	设备 i 的信道增益
$r_i(t)$	设备 i 的无线信道传输速率
W_s	MEC 服务器在待机状态下能耗
κ	设备计算能耗参数
σ^2	高斯白噪声功率
τ	时隙长度
w	信道带宽

2.3 问题建模与制定

2.3.1 问题建模

（1）工作负载队列模型：

为了保证系统稳定性，在时隙 t ，为每个物联网设备维护一个工作负载队列 $Q_i(t)$ ，存储已经到达并且等待处理的任务。假设到达任务数为 $a_i(t)$ ，处理量为 $b_i(t)$ 。那么，队列积压的变化过程如下所示：

$$Q_i(t+1) = [Q_i(t) - b_i(t) + a_i(t), 0]^+ \quad (2-1)$$

其中 $[x]^+ = \max\{x, 0\}$ 。另外，针对物联网设备，每个时隙内提交的任务量应该满足以下约束条件：

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} E[a_i(t) - b_i(t)] \leq 0, i \in N. \quad (2-2)$$

即到达的任务量平均值不应超过处理任务量的平均值，因此每个队列积压就不会无限增大，随着时间的推移，系统也会保持相对稳定。

(2) 能耗模型：

在时隙 t ，系统整体消耗为：

$$e(t) = e^l(t) + e^{tra}(t) + e^{ES}(t) \quad (2-3)$$

其中，系统能耗包含三部分：（1）本地计算能耗 $e^l(t)$ （2）传输能耗 $e^{tra}(t)$ （3）边缘层计算能耗 $e^{ES}(t)$ 。

关于能耗的计算，首先物联网设备 CPU 计算部分任务产生本地计算能耗 $e^l(t)$ ，然后部分将任务服务请求提交到附近的 MEC 服务器进行处理。因此出现传输能耗 $e^{tra}(t)$ 和边缘层计算能耗 $e^{ES}(t)$ 。

由上文可知，在时隙 t ，处理的计算任务量为 $b_i(t)$ 。它由两部分组成：一部分是通过本地计算生成的，大小为 $b_i^l(t)$ ；剩余部分通过无线信道向 MEC 服务器提交计算卸载任务请求，即 $b_i^{ES}(t)$ 表示。因此 $b_i(t) = b_i^l(t) + b_i^{ES}(t)$ 。在每个时隙开始之前，物联网设备通过获取当前时隙的环境信息，例如信道状况，然后更加智能地做出卸载决策。该模型中，信道条件在当前时隙中保持静态，而在时隙之间处于动态变化。物联网设备通过无线信道将数据发送到基站，再通过有线传输发送到边缘服务器，进行任务卸载。由于无线传输数据能耗占很大比例，所以只考虑无线部分的能耗。在无线通信过程中，数据通过上行链路上传并处理，然后通过下行链路返回结果。然而返回的结果小，所以下行链路的能量消耗忽略不计。

①本地计算能耗 $e^l(t)$

在时隙 t ，物联网设备 CPU 计算速率为 $f_i(t)$ ， L 为 CPU 计算一个任务（1bit）所需的周期数。那么，设备 i 在时隙 t 中的计算任务量 $b_i^l(t)$ 如下所示：

$$b_i^l(t) = \frac{f_i(t)\tau}{L} \quad (2-4)$$

每个物联网设备能耗计算如下：

$$e_i^l(t) = \kappa f_i^3(t) \quad (2-5)$$

其中 κ 表示设备计算能耗参数. 那么所有设备执行本地计算的能耗为:

$$e^l(t) = \sum_{i=1}^N e_i^l(t) \quad (2-6)$$

② 传输能耗 $e^{tra}(t)$

针对物联网设备 $i \in N$, 发射功率定义为 $P_i(t)$, 该时隙的信道增益用 $H_i(t)$ 表示。然后, 通过正交频分复用, 数据传输速率 $r_i(t)$ 由香农定理^{[68]–[70]}给出:

$$r_i(t) = \omega \log_2 \left(1 + \frac{H_i(t)P_i(t)}{\sigma^2} \right) \quad (2-7)$$

其中, ω 代表已分配的信道带宽, 而 σ^2 是高斯白噪声功率。由于卸载延迟不能超过一个时隙 τ 的大小。所以边缘服务器计算任务大小 $b_i^{ES}(t)$ 应遵循的约束条件为:

$$b_i^{ES}(t) \leq r_i(t)\tau \quad \forall i \in N \quad (2-8)$$

物联网设备 i 传输能耗如下:

$$e_i^{tra}(t) = P_i(t) \frac{b_i^{ES}(t)}{r_i(t)} \quad \forall i \in N \quad (2-9)$$

实际上, 物联网设备在边缘服务器卸载的任务数量是不能超过它当前的任务数量, 因此我们有

$$b_i^{ES}(t) \leq q_i(t) - b_i^l(t) \quad \forall i \in N. \quad (2-10)$$

在时隙 t , 所有物联网设备在边缘服务器卸载任务量的总能耗:

$$e^{tra}(t) = \sum_{i=1}^N e_i^{tra}(t) \quad \forall i \in N. \quad (2-11)$$

③ MEC 侧 $e^{ES}(t)$

本文 MEC 服务器串行处理不同设备的任务, 因此服务器在时隙 t 的能耗为:

$$e^{ES}(t) = W_s + \alpha \sum_{i=1}^N b_i^{ES}(t) \quad \forall i \in N \quad (2-12)$$

其中 MEC 服务器在待机状态下能耗为 W_s 。 α 是 MEC 服务器计算 1bit 任务的功率因子。 C 代表 MEC 服务器拥有的最大计算资源数量, 则

$$L \cdot \sum_{i=1}^N b_i^{ES}(t) \leq C. \quad (2-13)$$

2.3.2 问题形式化

本小节的目标是在长期系统稳定性的条件下进行系统能耗最小化。优化问题如下:

$$P_1: \min_{f_i(t), b_i^{ES}(t)} \left\{ \lim_{T \rightarrow +\infty} \frac{1}{T} \sum_{t=0}^{T-1} E\{e(t)\} \right\} \quad (2-14)$$

s.t.

$$\bar{Q} = \lim_{T \rightarrow +\infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i=1}^N E\{Q_i(t)\} < \infty \quad (2-15)$$

$$f_i(t) \leq f^{\max} \quad \forall i \in N \quad (2-16)$$

$$P_i(t) \leq P^{\max} \quad \forall i \in N \quad (2-17)$$

$$(8), (10), \text{和} (13). \quad (2-18)$$

约束 (2-15) 是一个队列稳定性约束, 表示物联网设备队列的总长度不能长时间无限增长, 并且平均时间内队列长度有界, 达到系统稳定的状态。约束 (2-16) 表示物联网设备每一时隙的 CPU 计算速率的最大值为 $f^{\max}$ 。约束 (2-17) 表示 $P^{\max}$ 是物联网设备的发射功率可以达到的上限值。

该优化问题是一个典型的随机优化问题。由于计算任务的产生量以及系统中无线信道的条件变化, 很难获得有关系统未来的信息, 因此在现实中, 离线解决该问题十分复杂。在下一节中, 我们将采用 Lyapunov 优化框架解决问题, 并制定在线任务卸载策略。

2.4 Lyapunov 在线卸载优化算法

为了解决本文系统能耗最小化问题, 本节采用 Lyapunov 优化框架设计了一种新颖的在线卸载算法, 称为 PODP。PODP 根据当前时隙系统状态信息做出任务最佳卸载决策, 而不依赖未来系统信息。该算法在系统能耗与队列积压之间进行权衡, 并且在系统稳定的条件下达到了系统最优能耗值。

2.4.1 Lyapunov 优化框架

基于 Lyapunov 优化, 我们获得以下两个引理, 分别为物联网设备工作负载的上界和系统能耗上界^[49], 同时等价转化问题 P_1 。

引理 1: 物联网设备工作负载的上界表示为:

$$\Delta L(\Theta(t)) \leq B - E \left\{ \sum_{i=1}^N \varepsilon \cdot Q_i(t) \right\} \quad (2-19)$$

其中 $\varepsilon = a_i(t) - b_i(t)$ 为靠近于 0 的正实数, 且 $B = 1/2 \sum_{i=1}^N (a_{\max}^2 + b_{\max}^2)$ 。

证明:

为了表示系统的当前状态和拥塞程度,我们为物联网设备定义了队列积压向量 $\Theta(t)$, 即 $\Theta(t) = (Q_1(t), Q_2(t), \dots, Q_N(t))$ 。对于向量 $\Theta(t)$, 定义二次 Lyapunov 函数如下:

$$L(\Theta(t)) = \frac{1}{2} \sum_{i=1}^N Q_i^2(t). \quad (2-20)$$

显然, Lyapunov 二次函数是非负的, 并且当所有队列 $Q_i(t)$ 的积压为 0 时, $L(\Theta(t))$ 才为 0。我们定义 $L(\Theta(0)) = 0$ 。接下来, 定义 Lyapunov 漂移函数如下:

$$\Delta L(\Theta(t)) = E \{ L(\Theta(t+1)) - L(\Theta(t)) | \Theta(t) \} \quad (2-21)$$

时隙之间的物联网工作负载队列长度的变化是衡量系统稳定的关键指标。

队列长度满足以下条件:

$$Q_i(t+1)^2 \leq (Q_i(t) - b_i(t) + a_i(t))^2, i \in N. \quad (2-22)$$

推导等式(2-22), 对 N 个物联网设备队列求和, 我们有

$$\begin{aligned} \frac{1}{2} Q_i(t+1)^2 &\leq \frac{1}{2} \sum_{i=1}^N Q_i(t)^2 + \frac{1}{2} \sum_{i=1}^N (a_i(t) - b_i(t))^2 \\ &\quad + \sum_{i=1}^N Q_i(t)(a_i(t) - b_i(t)). \end{aligned} \quad (2-23)$$

将不等式 (2-23) 根据等式带入 (2-21), 同时在两边取其期望, 我们有

$$\begin{aligned} \Delta L(\Theta(t)) &= \frac{1}{2} \sum_{i=1}^N E \{ Q_i(t+1)^2 - Q_i(t)^2 | \Theta(t) \} \\ &\leq \frac{1}{2} \sum_{i=1}^N E \{ (a_i(t) - b_i(t))^2 | \Theta(t) \} \\ &\quad + \sum_{i=1}^N E \{ Q_i(t)(a_i(t) - b_i(t)) | \Theta(t) \} \\ &\leq B + \sum_{i=1}^N E \{ Q_i(t)(a_i(t) - b_i(t)) | \Theta(t) \}. \end{aligned} \quad (2-24)$$

由于 $\forall i \in N$, $a_i(t) \leq a_{\max}$, $b_i(t) \leq b_{\max}$, 因此:

$$\frac{1}{2} \sum_{i=1}^N E \{ (a_i(t) - b_i(t))^2 | \Theta(t) \} \leq \frac{1}{2} \sum_{i=1}^N (a_{\max}^2 + b_{\max}^2). \quad (2-25)$$

设 B 为一个常数, 且 $B = (1/2) \sum_{i=1}^N (a_{\max}^2 + b_{\max}^2)$ 。由于设备计算能力大于任务到达率, 所以 $a_i(t) - b_i(t) \leq 0$, 因此引理 1 证明完毕。

接下来, 我们通过定义 Lyapunov 漂移加惩罚函数来最小化系统能耗, 同时我们也得到了引理 2, 即系统能耗上界值:

$$B + V \cdot E \{ e(t) | \Theta(t) \} + \sum_{i=1}^N Q_i(t)(a_i(t) - b_i(t)). \quad (2-26)$$

证明如下:

我们定义 Lyapunov 漂移加惩罚函数如下所示:

$$L_e(t) = \Delta L(\Theta(t)) + V \times E \{e(t) | \Theta(t)\}. \quad (2-27)$$

带入 $\Delta L(\Theta(t))$, 我们有

$$\begin{aligned} L_e(t) &= E \{L(\Theta(t+1)) - L(\Theta(t)) | \Theta(t)\} \\ &\quad + V \cdot E \{e(t) | \Theta(t)\} \\ &\leq B + V \cdot E \{e(t) | \Theta(t)\} \\ &\quad + \sum_{i=1}^N E \{Q_i(t)(a_i(t) - b_i(t)) | \Theta(t)\}. \end{aligned} \quad (2-28)$$

因此引理 2 被证明。

由于 B 是常数, 所以我们可以将问题 P_1 与问题 P_2 等价转换:

$$P_2: \min_{f_i(t), b_i^{ES}(t)} \left[V \cdot e(t) + \sum_{i=1}^N Q_i(t)(a_i(t) - b_i(t)) \right] \quad (2-29)$$

s.t.

$$(2-8), (2-10), (2-13) \text{ 和 } (2-15) - (2-17)$$

因此, 我们只需要知道系统当前信息, 然后求解每个时隙中(2-29)的最小值, 获得任务所需的卸载决策。

2.4.2 在线卸载计算

为了更好的解决问题 P_2 , 我们将系统能耗上界展开, 因此我们有:

$$\begin{aligned} &V \cdot e(t) + \sum_{i=1}^N Q_i(t)(a_i(t) - b_i(t)) \\ &= V \cdot (e^l(t) + e^{tra}(t) + e^{ES}(t)) \\ &\quad + \sum_{i=1}^N Q_i(t)(a_i(t) - b_i^l(t) - b_i^{ES}(t)) \\ &= V \left(\kappa \sum_{i=1}^N f_i^3(t) + \sum_{i=1}^N P_i(t) \frac{b_i^{ES}(t)}{r_i(t)} + W_s + \alpha \sum_{i=1}^N b_i^{ES}(t) \right) \\ &\quad + \sum_{i=1}^N Q_i(t)(a_i(t) - b_i^l(t) - b_i^{ES}(t)) \\ &= \sum_{i=1}^N \left[Q_i(t)a_i(t) + \kappa f_i^3(t)V - Q_i(t) \frac{f_i(t)\tau}{L} + W_s \right. \\ &\quad \left. + \left(\frac{VP_i(t)}{r_i(t)} + \alpha V - Q_i(t) \right) b_i^{ES} \right]. \end{aligned} \quad (2-30)$$

通过式 (2-30), 我们将优化问题分为两个子问题进行求解。

(1) 最优 CPU 计算速率

通过分离变量，本地计算表示如下：

$$\min_{f_i(t)} \left(\kappa f_i^3(t)V - Q_i(t) \frac{f_i(t)\tau}{L} \right). \quad (2-31)$$

显然目标函数是一个凸函数，因此在极值点或边界处得到最优值，即

$$f_i^*(t) = \min \left\{ \sqrt{\frac{Q_i(t)\tau}{3\kappa VL}}, f^{\max} \right\} \quad (2-32)$$

求解最优本地物联网设备 CPU 计算速率 $f_i^*(t)$ 后，代入公式 (2-4)，计算出每个物联网设备的本地计算任务量。

(2) 设备优先级

将式 (2-30) 中的 $b_i'(t)$ 计算后，可以得到 $b_i^{ES}(t)$ 的最优值通过解决

$$\min_{b_i^{ES}(t)} \left(\frac{VP_i(t)}{r_i(t)} + \alpha V - Q_i(t) \right) b_i^{ES}(t) \quad (2-33)$$

为了解决以上子问题，将问题最小化转化为最大化相反数，如下所示

$$\max_{b_i^{ES}(t)} \theta_i(t) b_i^{ES}(t) \quad (2-34)$$

方程 (2-34) 可以认为是背包问题，代表其每个物联网设备在边缘侧卸载的任务量，其中权重系数 $\theta_i(t)$ 可以看成每个物联网设备任务的单位价值，其中

$\theta_i(t) = Q_i(t) - \frac{VP_i(t)}{r_i(t)} - \alpha V$ ，因而我们可以确定设备优先级。则最优解是根据优先级

高低，依次将非负数价值高的设备请求任务放入背包，直到 MEC 服务器计算资源处理完任务。我们设定 MEC 服务器所有可用计算资源 $S(t)$ 的大小为 C 。

具体步骤如下。

① 遍历每个物联网设备的工作负载队列积压 $Q_i(t)$ 、发射功率 $P_i(t)$ 、速率 $r_i(t)$ 等信息。

② 按照权重系数 $\theta_i(t)$ 从高到低的原则对所有物联网设备进行优先级 $\mathbb{Q} = \{\theta_1(t), \theta_2(t), \dots, \theta_N(t)\}$ 确定。

③ 根据设备优先级，依次上传给 MEC 服务器计算卸载任务的大小。

其中每个设备上传给服务器的任务量 $b_i^{ES*}(t)$ 表示如下：

$$b_i^{ES*}(t) = \min \begin{cases} o_i(t), S(t)/L, & \text{if } \theta_i(t) \geq 0 \\ 0, & \text{if } \theta_i(t) < 0 \end{cases} \quad (2-35)$$

根据约束 (2-8) 和 (2-10), 其中 $o_i(t) = \min \{r_i(t)\tau, Q_i(t) - b_i^l(t)\}$ 。

④据 $S(t+1) = S(t) - b_i^{ES}(t)$, 更新边缘服务器剩余的计算资源数量。

至此, 为了解决问题 P_2 , 我们设计了一种在线系统能耗 PODP 优化算法。该算法最小化每个时隙系统能耗上界, 然后确定最优的任务卸载策略。算法伪代码如表 2-1。

2-1 算法 PODP

```

1:输入: 设备参数  $a_i(t), P_i(t), H_i(t)$  等
2:输出: 最优解  $f_i(t), b_i^{ES}(t)$ ;
3:初始化:  $\forall i \in N, b_i^{ES} = 0, S(t) = C$ 
4: for  $t = \{1, 2, \dots, T\}$ 
5:   for  $i \in N$ 
6:     获得  $r_i(t), P_i(t)$ ;
7:     计算  $H_i(t)$ ;
8:     根据公式 (2-31) 计算  $\sqrt{\frac{Q_i(t)\tau}{3\kappa VL}}$ ;
9:     根据公式 (2-32) 重新设置  $f_i(t)$ ;
10:    计算  $b_i^l(t)$ ;
11:   end for
12:   根据  $\theta_i(t)$ , 确定所有设备队列的优先级;
13:   for  $i \in N$ 
14:     根据(2-35)中的队列优先级确定设备要卸载的任务量
15:     根据  $S(t+1) = S(t) - b_i^{ES}(t)$  更新  $S(t)$ ;
16:     根据  $Q_i(t+1) = \max \{Q_i(t) - b_i(t) + a_i(t), 0\}$  更新  $Q_i(t)$ ;
17:   end for
18: end for

```

2.4.3 算法理论边界分析

在本小节中, 我们从系统目标函数的解和工作负载队列的稳定性进行分析我们的算法。根据 Lyapunov 定理^[49], 当系统达到一个稳定的状态时, 我们得出以下两个定理:

定理 1: 队列长度满足强稳定性, 且满足以下不等式:

$$\lim_{T \rightarrow +\infty} \frac{1}{T} \cdot \sum_{t=0}^{T-1} \sum_{i=1}^N E\{Q_i(t)\} \leq \frac{B + Ve^*}{\varepsilon} \quad (2-36)$$

定理 2:，当卸载性能保持稳定时，系统能耗满足以下不等式：

$$\lim_{T \rightarrow +\infty} \frac{1}{T} \cdot \sum_{t=0}^{T-1} E\{e(t)\} \leq e^* + \frac{B}{V} \quad (2-37)$$

以上两个定理的证明如下：

由于 Lyapunov 漂移和惩罚函数落在一个阈值范围内，表示如下：

$$E\{\Delta L(\Theta(t)) + V \cdot e(t)\} \leq B + V \cdot e^* - \varepsilon \cdot E\left\{\sum_{i=1}^N Q_i(t)\right\} \quad (2-38)$$

其中 V 是控制参数， e^* 是平均系统能耗，因此，基于 Lyapunov 稳定性定理，根据 (2-38)，对于 $t \in \{0, 1, 2, \dots, T-1\}$ ，采用伸缩求和定律，我们有

$$\begin{aligned} & E\{L\Theta(T)\} - E\{L\Theta(0)\} + V \sum_{t=0}^{T-1} E\{e(t)\} \\ & \leq (B + Ve^*) \cdot T - \varepsilon \sum_{t=0}^{T-1} \sum_{i=1}^N E\{Q_i(t)\} \end{aligned} \quad (2-39)$$

将不等式 (39) 除以 VT 得到 (40)，除以 εT 得到 (41)，如下所示

$$\lim_{T \rightarrow +\infty} \frac{1}{T} \cdot \sum_{t=0}^{T-1} E\{e(t)\} \leq e^* + \frac{B}{V} + \frac{E\{L\Theta(0)\}}{VT} \quad (2-40)$$

$$\lim_{T \rightarrow +\infty} \frac{1}{T} \cdot \sum_{t=0}^{T-1} \sum_{i=1}^N E\{Q_i(t)\} \leq \frac{B + Ve^*}{\varepsilon} + \frac{E\{L\Theta(0)\}}{\varepsilon T} \quad (2-41)$$

将以上不等式右边进行重新排列，在合适的时间忽略非负项 $E\{L\Theta(0)\}$ ，然后分别对 $t \rightarrow \infty$ 取 (2-40) 和 (2-41) 极限。显然，定理 1，定理 2 被证明。

PODP 算法中的系统平均能耗和物联网设备任务队列的平均大小都是稳定的。并且系统平均时间内能耗和物联网设备任务队列长度的时间复杂度分别为 $O(1/V)$ 和 $O(V)$ 。因此控制参数 V 能够有效地调整系统能耗和物联网设备队列的长度，从而在我们的场景中实现系统能耗和任务队列长度的平衡。

2.5 仿真结果

本文从系统稳定性、系统能耗、对比实验三个方面对系统性能进行了评估，并利用 MATLAB 对所得理论结果进行了验证。

该系统包含多个物联网设备和一个 MEC 服务器。模拟持续时间为 100 个时隙。假设到达的数据大小遵循泊松分布，平均值为 200 位，即 $a_i(t) \sim P(200)$ 。文中无线信道

为瑞利衰落模型， $H_i(t)$ 为数学期望为 1 的随机变量，服从指数分布，即 $H_i(t) \sim E(1)$ 。设置每个物联网设备的传输功率为 $P_i(t) \sim U[0.1, 1]W$ 。此外，我们设置 $\kappa = 0.5$ ， $\sigma^2 = 10^{-12}W$ ， $P^{\max} = 1W$ ， $f^{\max} = 1.2GHz$ ， $L = 500 \text{ cycles / byte}$ 。

实验中部分参数是按照固定的统计分布设置的，然而 LDSO 实际上并不需要统计前的信息。

2.5.1 系统稳定性

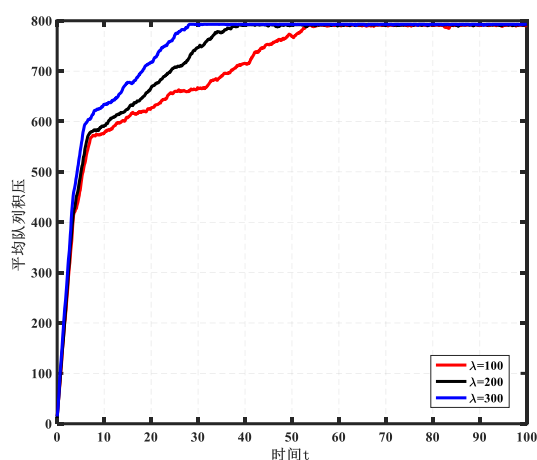


图 2-2 不同到达率 λ 下的队列积压

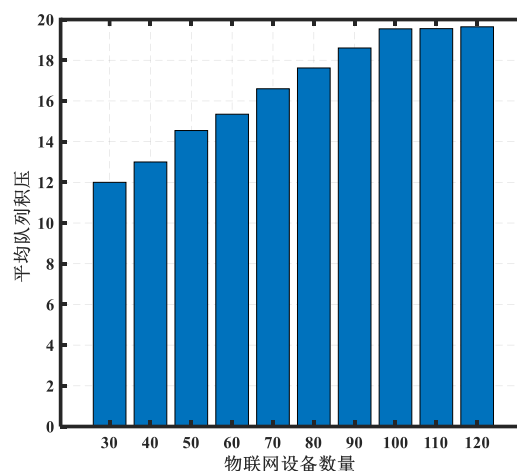


图 2-3 不同设备数量 N 的队列积压

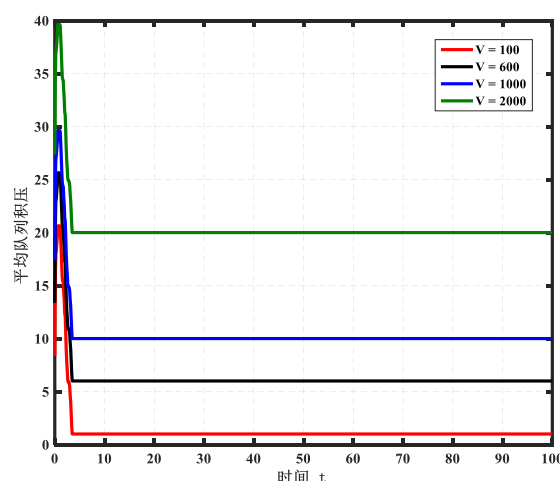


图 2-4 不同 V 值下队列积压

我们从不同的系统参数来分析系统稳定性，包括任务到达率 λ 、控制参数 V 和设备数量 N 。实验结果如图 2-2、2-3、2-4 所示。图 2-2 说明了在不同的数据到达率下，

队列积压首先增加，然后达到稳定。同时，数据到达率越大，达到稳定所需的时隙数就越少。我们可以看到，在 $\lambda=100$ 时候，系统在 58 个时隙后达到稳定，而在 $\lambda=300$ 时，只用了 22 个时隙，这一现象验证了所提出的 PODP 算法的可扩展性，并且在图 3 中可以得到类似的结论，其中系统在不同节点数量下达到稳定性。图 4 绘制了具有不同控制参数 V 的队列积压结果。 V 值越大，队列积压阈值越大。这主要是因为较大的 V 允许缓冲更多的数据量，如定理 1 所示。

2.5.2 系统能耗

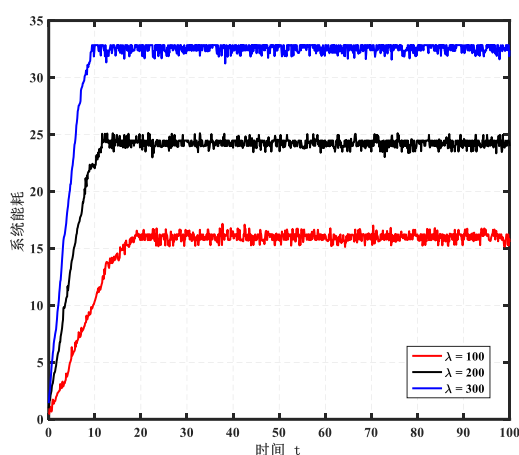


图 2-5 不同到达率 λ 下的系统能耗

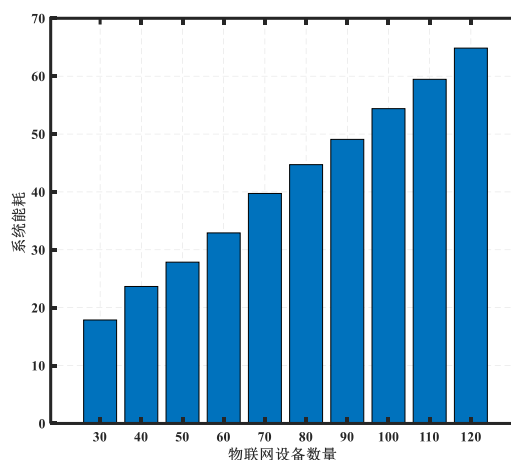


图 2-6 不同设备数量 N 的系统能耗

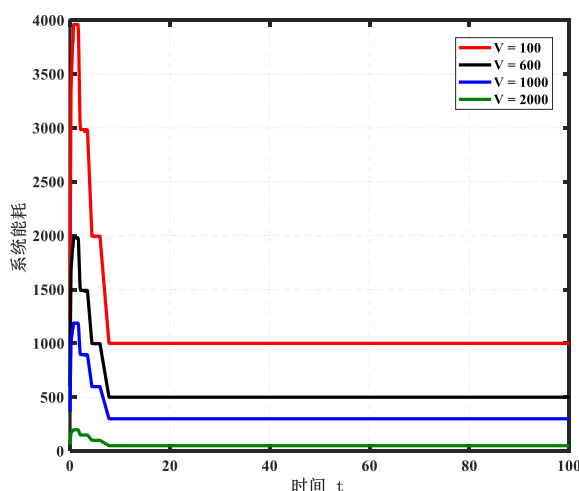


图 2-7 不同 V 值下系统能耗

与系统稳定性的结论类似，数据到达率和节点数量越大，消耗的系统能量越多，如图 2-5 和图 2-6 所示。例如，当 $\lambda=100$ 时，当系统处于稳定，用了 19 个时隙。而当 $\lambda=300$ ，而仅用了 9 个时隙。这证明了本文算法的有效性。当图 2-7 中观察到一个有

趣的现象，即 V 值越大，则能量消耗是越小，它与到达率和节点数量不同。这主要是因为能量消耗与 V 值成反比，这验证了定理 2。

2.5.3 对比实验

为了进一步测试系统性能，我们将 PODP 与以下四种算法进行比较。其中两个是基线算法，分别为 Local 和 MEC，另外两个是 EEDTO 和 DIP，与本文 PODP 最相似。

(1) Local: 物联网设备本身以最大的计算能力处理它们产生的所有工作负载。

(2) MEC: 边缘服务器以最大计算能力处理物联网设备产生的所有工作负载。

(3) EEDTO^[65]: 通过 Lyapunov 框架，将系统能耗解耦包含变量的子问题，然后根据是否超过延迟期限贪婪地选择最优卸载决策。

(4) DIP^[94]: 利用 Lyapunov 求得能耗上限，并在目标方程中添加多项式系数。然后将其转化为一个新的线性极值问题，包括卸载决策以获得端点处的最优解。

图 2-8 和图 2-9 说明了平均队列积压和系统能耗。可以清楚地看到，两种基线算法的可扩展性很差，因为缓冲数据随着时间窗口的增加而线性增长。而其他三种算法则先下降，然后达到稳定状态。此外，所提出的 PODP 实现了最低的队列积压，与 EEDTO 和 DIP 相比分别减少了 69.2%和 44.4%。具体来说，当系统达到稳定时，占用 12、20 和 27 个缓冲单元。系统能耗与队列积压情况类似，与其他四种算法相比，所提出的算法 PODP 消耗的能量最少。两个数值结果都表明 PODP 具有更好的可扩展性

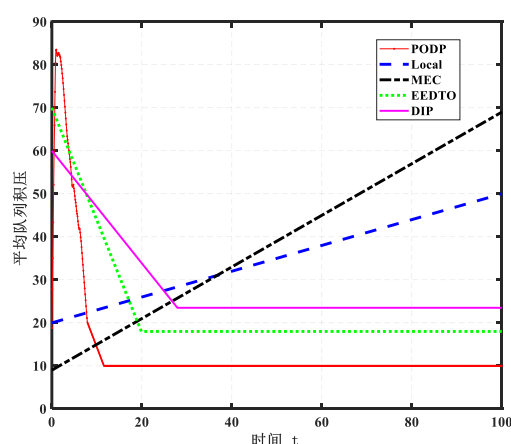


图 2-8 PODP 与四种算法的平均队列积压

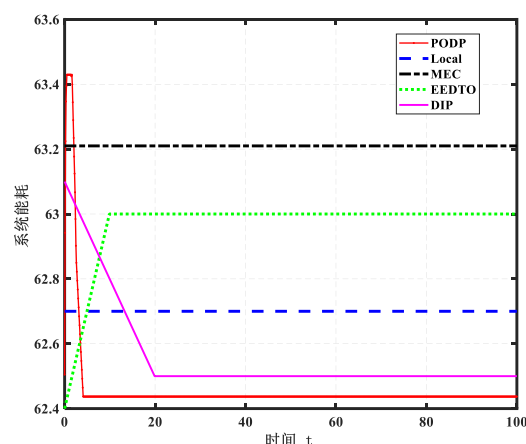


图 2-9 PODP 与四种算法的系统能耗

2.6 本章小结

本文研究了协作场景中 MEC 系统的能耗最小化问题。基于 Lyapunov 优化，我们

提出了一种在线动态计算卸载算法，该算法根据物联网设备优先级制定高效卸载策略，确定了本地和 MEC 服务器的计算任务数量。同时，该算法不需要环境的先验统计信息，就可以达到平衡双性能指标的目的。实验结果表明，PODP 在保持物联网设备稳定队列积压的同时，也获得了渐进最优的系统能耗。最后，该算法在面对动态变化时也表现出良好的鲁棒性。

第三章 一种基于 Lyapunov 优化的动态服务卸载算法

第二章研究单服务器通信场景下的数据协作卸载方案，然而未考虑多边缘服务器之间的数据流量控制问题。因此本章研究多边缘服务器缓冲区队列对系统性能的影响。首先为每个服务器缓冲区创建队列模型来保证系统稳定性，而终端服务卸载的决策控制了队列长度。然后通过 Lyapunov 优化框架获得系统成本近似解，同时也确定了服务卸载方式，即单跳卸载或者多跳卸载。最后通过与其他算法比较实验，证明所提算法以最低的系统成本获得了最大的吞吐量。

3.1 引言

边缘计算系统能够快速响应用户请求，边缘计算系统在多边缘服务器协作过程中面临着复杂多变的网络环境。一方面，各种状态信息可能随时间动态变化，例如时变的服务请求和任务到达率，这使得边缘服务器难以及时反映流量的变化^[73]。另一方面，在处理密集、复杂的业务请求时，多个边缘服务器缓存的服务数据存储量可能会因带宽资源不足而导致链路拥塞。因此，它将导致系统稳定性降低、用户体验服务响应时间更长以及系统能耗增加^[77,87,88,94]。因此，考虑到多个边缘服务器之间的资源可用性、任务到达率的波动以及系统的长期稳定性等因素，制定有效的数据分流策略并实现动态数据流量控制和系统成本之间的权衡是一个具有挑战性的问题。因此本章研究边缘服务器协同服务卸载的最小卸载成本问题。

3.2 边缘服务卸载系统架构

本文提出的系统模型如图 1 所示，边缘服务卸载系统架构中包括一个宏基站 BS， M 个小基站 SBS 和 n 个用户， M 个 SBS 被 BS 覆盖，其中 $P_k(t)$ 表示 t 时刻物联网终端应用程序服务请求概率，那么 $n \times P_k(t)$ 为 t 时刻第 k 类服务请求人数。与大多现有的单跳服务数据卸载不同，服务可以通过节点水平协作进行多跳卸载，最终由协作的边缘层服务器完成服务数据处理，即多个 SBS 通过协作共同为用户提供服务。当用户向附近小基站发出服务请求时，如果基站缓存用户请求的服务，则该基站处理，否则边缘

侧通过节点协作共同处理用户服务请求。

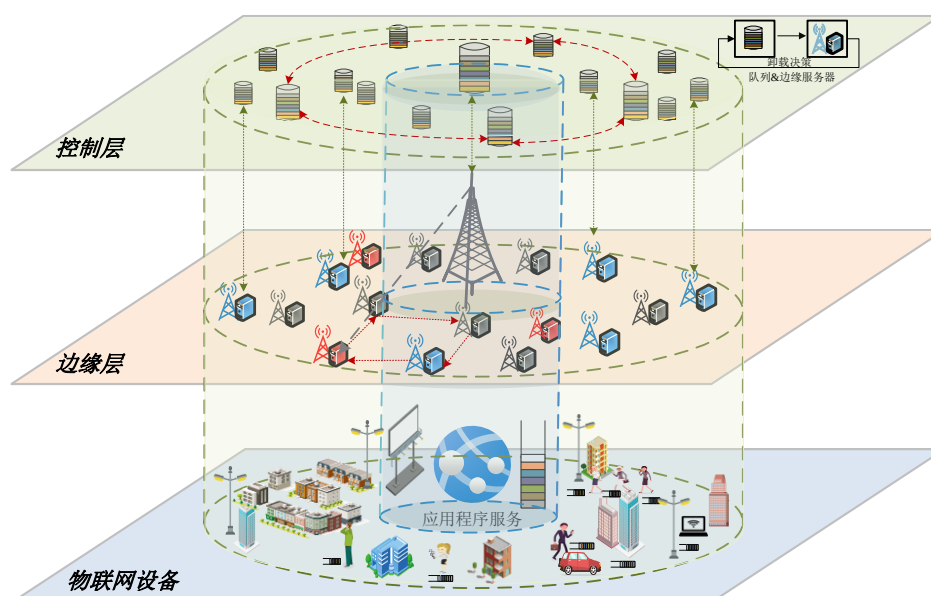


图 3-1 边缘服务卸载系统架构

用户依据强度 λ_u 出现。假设所有 SBS 都遵循 PPP 分布，其强度为 λ_e 。假设 BS 可以访问具有 F 个独立服务的数据库 $F = \{F_1, F_2, \dots, F_k\}$, 每个服务数据的大小为 s bit, 由于小基站存储量有限, 因此每个 SBS 缓存 S 种服务。系统中设有提供第 k 类服务的 SBS 共有 a_k 个, $f_i^k = \{f_1^k, f_2^k, \dots, f_m^k\}$ 是小基站服务放置向量, 指示第 k 个服务数据是否缓存在第 i 个 SBS 中, 因此 $\sum_{i=1}^m C_i^k(t) = a_k$ 是本系统第 k 类服务基站提供者数量, 当系统中用户向附近的边缘服务器发送请求服务时, $x_i^k(t)$ 表示为请求第 k 类服务用户是否在第 i 个边缘服务器卸载, 即卸载决策。

本文使用的主要参数如表 3-1 所示。

3.3 问题制定与分析

3.3.1 问题表述

本文的目标是降低长期边缘卸载系统单位成本 $Cost(t)$ 。物联网设备终端发送服务请求, 经过边缘节点处理后, 可以及时地向用户提供各类服务。考虑到服务请求的随机到达, 因此, 问题表述如下:

表 3-1 主要符号

符号	说明
n	用户数量
K	服务数量
M	小基站数量
$P_k(t)$	t 时刻第 k 类服务用户请求概率
b_k	第 k 类服务大小
$h(t)$	信道增益
ϕ^2	信噪比
B_w	带宽
$r(t)$	信道传输速率
P_u	设备发射功率
e_u	设备发射的单位数据能耗
e_s	服务器接收与转发单位数据的能耗
$E^c(t)$	通信能耗
$E^p(t)$	数据处理能耗
$T^c(t)$	通信延迟
$T^p(t)$	数据处理延迟
I	边缘服务器集合 $\{1, 2, \dots, M\}$
$Q_i(t)$	第 i 个边缘服务器缓冲区队列大小
$\mu_i^{\max}$	第 i 个边缘服务器最大计算资源数量
$A_i^{\max}$	第 i 个边缘服务器接收的最大数据量
$x_i^k(t)$	第 k 类服务在第 i 个边缘服务器的卸载决策

$$\min \left\{ \lim_{T \rightarrow +\infty} \frac{1}{T} \sum_{t=0}^{T-1} E\{Cost(t)\} \right\} \quad (3-1)$$

s.t.

$$\bar{Q} = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^T \sum_{i=1}^M E\{Q_i(t)\} < +\infty \quad (3-2)$$

$$\forall i \in [1, M], Q_i(t) \leq Q_i^{\max}(t) \quad (3-3)$$

$$\forall i \in [1, M], \sum_{k=1}^K n_i^k(t) b_k \leq (Q_i^{\max}(t) - Q_i(t)) \quad (3-4)$$

$$\forall i \in [1, M], \mu_i(t) \leq \mu^{\max} = \mu \quad (3-5)$$

$$\sum_{t=0}^{T-1} E^c(t) + E^p(t) \leq E^{\max}, \sum_{t=0}^{T-1} T^c(t) + T^p(t) \leq T^{\max} \quad (3-6)$$

$$\forall k \in [1, K], \sum_{i=1}^M x_i^k(t) = 1 \quad (3-7)$$

约束 (3-2) 表示边缘服务器缓冲区中数据队列的稳定性, (3-3) 和 (3-4) 表示边缘服务器中队列的库存约束。当缓冲区中待处理的业务数据存量达到最大时, 时延增大, 并且可能会丢失大量数据。(3-5) 表示每个边缘服务器的处理能力不得超过最大值。(3-6) 保证在每个时隙内, 系统的能耗和时延不超过其最大值。(3-7) 表示服务的 0-1 卸载约束, 其中每种类型的服务只能卸载到一台边缘服务器。

系统研究数据传输和数据处理的能量消耗, 服务卸载不可避免地也会出现计算和通信延迟。因此, 本文考虑包含能耗和延迟的系统单位成本模型, 同时, 假设物联网感知终端不具有计算能力, 因此不考虑本地计算。本文系统单位成本表示如下:

$$\text{Cost}(t) = \theta \cdot E_{\text{total}}(t) + (1 - \theta) T_{\text{total}}(t) \quad (3-8)$$

θ 是调整成本中能耗与延迟比例的权重因子, $E_{\text{total}}(t)$ 和 $T_{\text{total}}(t)$ 为总能耗与总时延。

本文考虑离散时间模型, t 表示时隙。在每个时隙 t , 需要将终端服务卸载到边缘服务器进行处理, 以保证能耗与延迟约束。另外, 在模型中考虑了 0-1 卸载模式。对于终端服务请求, 通信范围内的含有该服务的缓存基站可以直接完成卸载, 或者也可以将其转发到边缘协作节点区域内进行卸载。设备/用户端向边缘服务器发送服务请求时, 卸载决策 $x_i^k(t)$ 是一个布尔函数 (即 0 或 1), 1 代表在时隙 t 中第 k 类服务在邻接边缘服务器 i 被卸载, 相反 0 代表通过多跳协作进行卸载。

与传统的 Zipf 分布不同, 该部分利用二分图表示服务的请求概率^[50], 来计算第 K 类服务的被请求概率, t 时刻第 k 类服务请求的设备/用户数量 $n_i^k(t)$ 表示如下:

$$n_i^k(t) = n_i P_k(t) \quad (3-9)$$

其中 $P_k(t)$ 表示在 t 时刻用户请求第 k 个服务的概率, n_i 是第 i 个边缘服务器覆盖的用户数量。因此, 第 i 个边缘服务器在该时隙内接收到的服务数据量表示如下。

$$A_i(t) = \sum_{k=1}^K n_i^k(t) b_k \quad (3-10)$$

另外定义 $A_i^{\max}$ 作为第 i 个边缘服务器接收到的数据量最大值, 因此 $A_i(t) \leq A_i^{\max}$ 。

3.3.2 问题分析

(1) 边缘服务器缓冲区队列模型:

考虑了单时隙服务请求模型, 其中, 数据请求量由终端数量及其服务类型生成,

继而卸载到边缘服务器。然而，由于每个小基站缓存部分服务，因此边缘服务器通常无法在一个时隙内完成所有服务数据的卸载。为了低延迟和低能耗完成卸载数据，需要在边缘服务器中考虑缓冲区数据积压队列模型，以保证边缘卸载系统网络的稳定性。在提出的模型中（如图 3-2 所示），为每个边缘服务器维护一个缓冲区数据队列，来保证边缘层节点队列的稳定性。

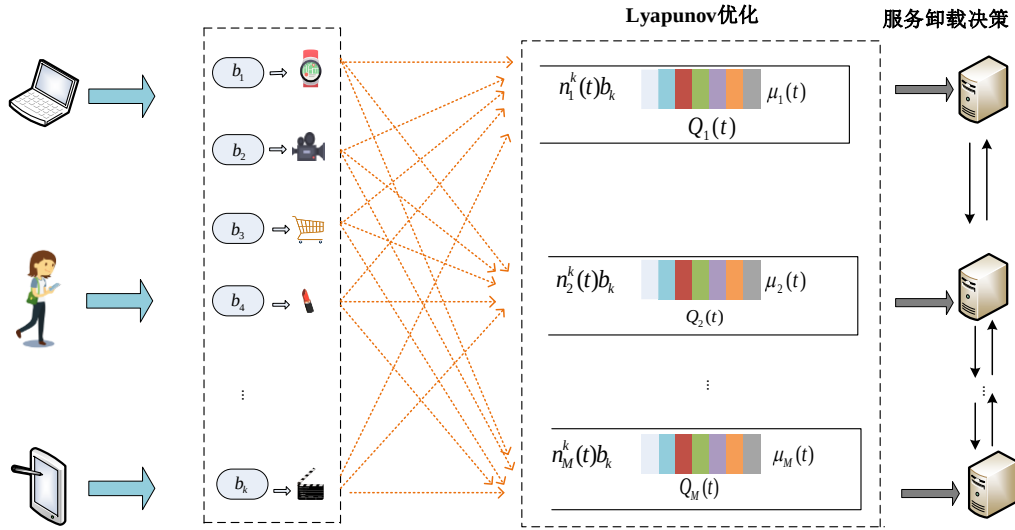


图 3-2 边缘服务器缓冲区队列模型

令 $Q_i^k(t)$ 表示第 i 个边缘服务器的缓冲区中来自第 k 个服务的数据量。各种服务对缓冲区的占用情况可以表示如下。

$$Q_i(t+1) = \sum_{k=1}^K Q_i^k(t) \quad (3-11)$$

因此，相邻两个时隙的缓冲数据量为 $Q_i(t+1)$ ，表示如下：

$$Q_i(t+1) = \max\{Q_i(t) - \mu_i(t), 0\} + A_i(t) \quad (3-12)$$

$\mu_i(t)$ 表示边缘服务器处理第 k 类服务计算资源数量。(3-12)表示在时隙 t 卸载的服务数据量将存储在队列中，并在下一个时隙进行处理。如果在某个时隙完成的数据处理量小于接收的数据量（即出现数据队列积压加剧），缓冲区队列的长度就会增长。反之，队列的长度会缩短（即队列积压减少）。另外，边缘服务器缓存部分服务，令 $\mu_i^{\max}(t)$ 表示存在缓存该服务边缘服务器在一个时隙内最大工作负载。因此，记为。在每个时间段，已处理和已完成的数据量应小于最大工作负载，即 $\mu_i(t) \leq \mu_i^{\max}(t)$ 。

此外，还应考虑队列稳定性，以确保网络在数据卸载过程中不会出现拥塞。也就是说，缓冲队列的长度不应该无限增长，否则会出现长时间等待的情况。因此，应该

限制缓冲队列的平均长度。也就是说，需要保证队列长度小于其上界，即 $Q_i(t) \leq Q_i^{\max}$ 。

边缘卸载成本模型：

(2) 能耗模型

能耗模型：系统能耗是由通信中的数据传输和边缘服务器的数据处理产生的。在本研究中，数据包转发和接收过程中的能耗被视为传输能耗的一部分。定义 $E^c(t)$ 为通信能耗， $E^p(t)$ 为数据处理能耗，总能耗 $E_{total}(t)$ 可表示为：

$$E_{total}(t) = E^c(t) + E^p(t) \quad (3-13)$$

事实上，数据传输包括两个阶段。一个是从用户设备到边缘服务器的传输，另一个是指边缘服务器之间的数据通信。两个阶段产生的能耗分别称为设备-服务器能耗和服务器-服务器能耗。定义设备-服务器能耗为 $E^{c1}(t)$ ，服务器-服务器能耗为 $E^{c2}(t)$ ，数据传输总能耗为：

$$E^c(t) = E^{c1}(t) + E^{c2}(t) \quad (3-14)$$

通常，数据传输的能耗取决于传输功率和传输数据量。设 e_u 代表用户产生的单位能耗，则设备-服务器能耗 $E^{c1}(t)$ 可表示如下：

$$E^{c1}(t) = \sum_{i=1}^M \sum_{k=1}^K n_i^k(t) b_k e_u \quad (3-15)$$

公式 (3-15) 表明，能耗与接收数据量呈线性关系，即随着工作负载的增加，能耗也会相应增长。

另一方面，假设协作区域中的边缘服务器的数量被表示为 $O_k(t)$ 。当业务数据经过多跳传输时，最大跳数为 $H_k(t) = O_k(t) - 1$ 。那么，服务器之间协作接收和转发数据的能耗可计算为

$$E^{c2}(t) = \sum_{i=1}^M \sum_{k=1}^K n_i^k(t) b_k e_s H_k(t) (1 - x_i^k(t)) \quad (3-16)$$

假设有 K 种服务，每个 SBS 只能缓存部分服务。当小基站收到请求时，该从属基站处理的概率为 $P_c = \bar{A}_i / K$ ，其中 $\bar{A}_i$ 表示第 i 个小基站缓存的服务类型数量。当服务请求转发给其他合作的小基站时，该服务得到其他小基站响应的概率为

$$P_{nc} = (K - \bar{A}_i) / K。$$

令 e_p 表示边缘服务器每比特数据的处理能力。数据处理的能耗 $E^p(t)$ 为

$$E^p(t) = \sum_{i=1}^M \sum_{k=1}^K e_p \cdot A_i(t) (P_c x_i^k(t) + P_{nc} \cdot (1 - x_i^k(t)) H_k(t)) \quad (3-17)$$

(3) 延迟模型

延迟模型：服务卸载过程中的延迟包括通信延迟 $T^c(t)$ 和计算延迟 $T^p(t)$ 。因此，总延迟可表示为

$$T_{total}(t) = T^c(t) + T^p(t) \quad (3-18)$$

根据香农定理，在时隙 t 内，用户设备到第 i 个边缘服务器的通信延迟与分流数据量 $n_i^k(t)b_k$ 相关，即 t 时隙的请求用户数与第 k 类服务大小的乘积。假设通信带宽为 B_w ，设备发射功率为 P_u ，信道增益表示为 $h(t)$ 。设 $r(t)$ 为信道传输速率，通信时延可表示为：

$$T^c(t) = \sum_{i=1}^M \sum_{k=1}^K \frac{n_i^k(t)b_k}{r(t)} \quad (3-19)$$

其中 $r(t) = B_w \cdot \log_2(1 + \frac{P_u(t)h(t)}{\sigma^2})$ 。

为了描述服务的计算延迟，将每个对第 k 类服务的数据卸载做出贡献的 SBS 视为一个 M/M/1 队列。根据排队论，到达流可以建模为泊松分布。令 λ_u^k 表示针对 u 个用户对于第 k 个服务的到达强度。平均等待时间可以表示为

$$t_k = \frac{1}{\mu_k(t) - \sum_{u=0}^{n_i^k(t)} \lambda_u^k b_k} \quad (3-20)$$

如以上服务卸载方法所述，有单跳和多跳两种情况。因此，计算延迟可以表示如下。

$$T^p(t) = \sum_{i=1}^M \sum_{k=1}^K P_c \cdot t_k x_i^k(t) + H_k(t) \cdot P_{nc} \cdot t_k (1 - x_i^k(t)) \quad (3-21)$$

3.4 基于 Lyapunov 优化的动态服务卸载算法 (LDSO)

为了解决式 (3-1) 表示的成本最小化问题，采用 Lyapunov 函数来控制边缘卸载系统。然后，将依赖于时间的长期优化问题转化为仅需要当前时隙信息的优化问题。此外，提出了一种基于贪婪策略的服务匹配算法来解决该优化问题。图 3 展示了 LDSO（基于 Lyapunov 优化的动态服务卸载）的框架。

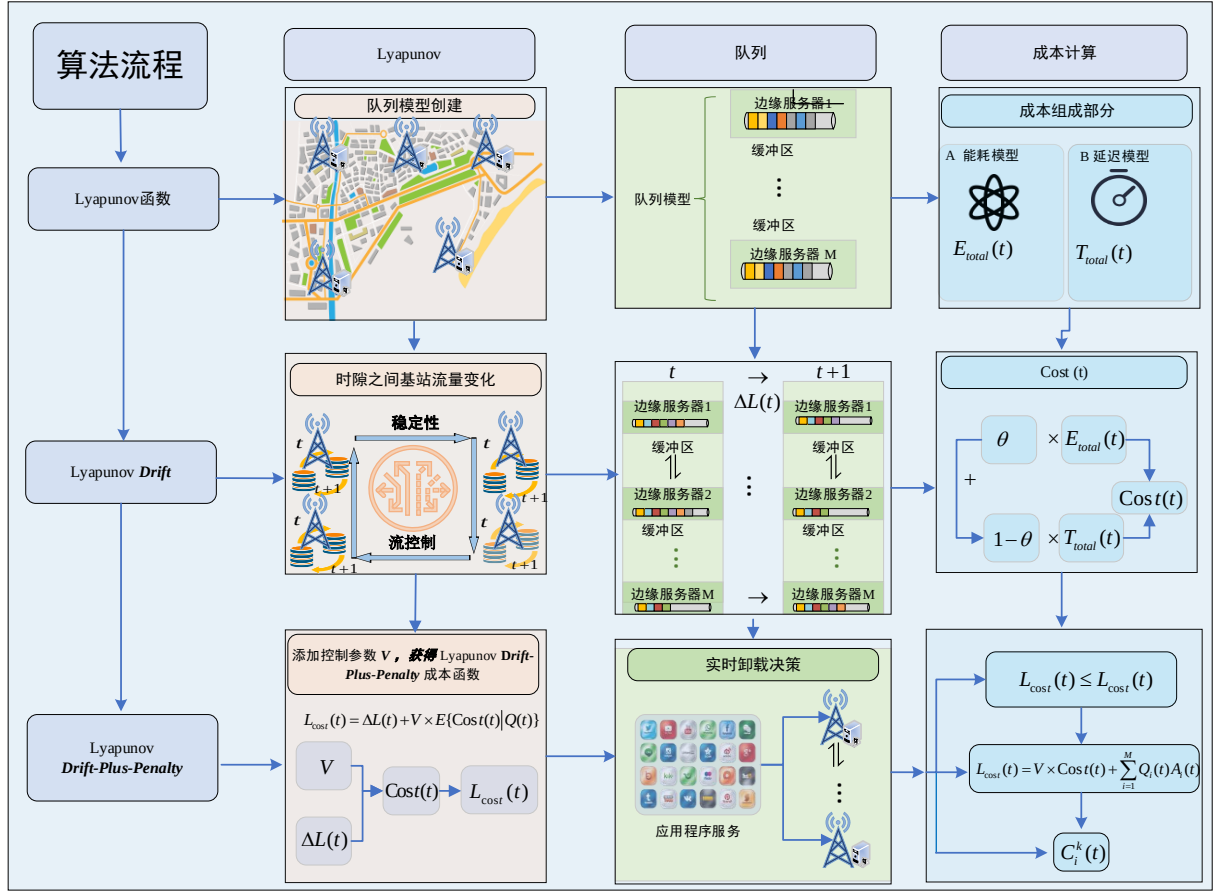


图 3-3 LDSO 算法框架

3.4.1 Lyapunov 优化解决框架

本研究将实时服务卸载的决策问题转化为系统成本最小化的决策问题。首先引入以下定义和引理来讨论系统成本的表示。

关于二次 Lyapunov 函数和 Lyapunov 漂移函数的定义如 2.4.1 节公式 (2-20) 和 (2-21) 所示。

注：本章节的 i 为边缘服务器的序号。

本小节 $\Delta L(t)$ 表示边缘服务器队列中缓冲数据的变化。基于以上两个函数定义，我们有以下引理 1。

引理 1： 边缘服务器中缓冲数据变化的上限可以表示为

$$\Delta L(t) \leq B + E \left\{ \sum_{i=1}^M Q_i(t) (A_i(t) - \mu_i(t)) \right\} \quad (3-22)$$

$$\text{其中 } B = \sum_{i=1}^M \frac{(\mu_i^{\max}(t))^2 + (A_i^{\max}(t))^2}{2}$$

证明：在每个时隙中，边缘服务器卸载的数据量始终大于等于 0，因此得到如下结果。

$$\max\{Q_i(t) - \mu_i(t), 0\} \leq Q_i(t) \quad (3-23)$$

$$\left(\max\{Q_i(t) - \mu_i(t), 0\}\right)^2 \leq (Q_i(t) - \mu_i(t))^2 \quad (3-24)$$

基于 Lyapunov 函数，公式 (3-12) 可更新如下

$$\begin{aligned} (Q_i(t+1))^2 &= \left(\max\{Q_i(t) - \mu_i(t), 0\}\right)^2 + (A_i(t))^2 \\ &\quad + 2 \max\{Q_i(t) - \mu_i(t), 0\} \cdot A_i(t) \\ &\leq (Q_i(t) - \mu_i(t))^2 + (A_i(t))^2 \\ &\quad + 2Q_i(t) \cdot A_i(t) \\ &= (Q_i(t))^2 + (\mu_i(t))^2 + (A_i(t))^2 \\ &\quad - 2Q_i(t)(\mu_i(t) - A_i(t)) \end{aligned} \quad (3-25)$$

然后，时隙 t 处的 Lyapunov 漂移函数改变如下。

$$\begin{aligned} \Delta L(t) &= E \left\{ \sum_{i=1}^M \frac{1}{2} \left((Q_i(t+1))^2 - (Q_i(t))^2 \right) \right\} \\ &\leq \sum_{i=1}^M \frac{(\mu_i^{\max}(t))^2 + (A_i^{\max}(t))^2}{2} \\ &\quad - \sum_{i=1}^M E \{ Q_i(t)(\mu_i(t) - A_i(t)) \} \end{aligned} \quad (3-26)$$

引理 1 被证明。

定义：Lyapunov 漂移加罚函数。

$$L_{\text{Cost}}(t) = \Delta L(t) + V \times E \{ \text{Cost}(t) | Q(t) \} \quad (3-27)$$

其中 V 是非负控制参数， $\text{Cost}(t)$ 是式 (3-1) 中定义的系统成本。通过动态调整控制参数 V ，可以实现系统成本和缓冲数据之间的平衡。

目标函数 $\text{Cost}(t)$ 在稳定性的基础上最小化。因此， $L_{\text{Cost}}(t)$ 表示优化时隙系统的卸载成本。由引理 1 可知， $\Delta L(t)$ 衡量系统的稳定性。我们有以下引理 2。

引理 2：系统成本 $L_{\text{Cost}}(t)$ 的上限表示为：

$$L_{\text{Cost}}(t) \leq L_{\text{Cost}}(t) \quad (3-28)$$

其中 $L_{\text{Cost}}(t) = B + V \cdot \text{Cost}(t) + \sum_{i=1}^M Q_i(t)A_i(t)$ 并表示带有李雅谱诺夫惩罚的 $L_{\text{Cost}}(t)$ 的上限。

证明：将引理 1 的结果代入式 (3-27)，得漂移加罚函数 $L_{\text{Cost}}(t)$ 的缩放如下。

$$\begin{aligned}
 L_{\text{Cost}}(t) &= \Delta L(t) + V \times E \{ \text{Cost}(t) | Q(t) \} \\
 &\leq B + E \left\{ \sum_{i=1}^M Q_i(t) (A_i(t) - \mu_i(t)) \right\} \\
 &\quad + V \times E \{ \text{Cost}(t) | Q(t) \} \\
 &= B + V \times \text{Cost}(t) + E \left\{ \sum_{i=1}^M (Q_i(t) A_i(t)) \right\} \\
 &\quad - E \left\{ \sum_{i=1}^M Q_i(t) \mu_i(t) \right\} \\
 &\leq B + V \times \text{Cost}(t) + E \left\{ \sum_{i=1}^M (Q_i(t) A_i(t)) \right\}
 \end{aligned} \tag{3-29}$$

然后, 证明引理 2。

根据条件期望机会最小化的框架^[49], 通过最小化系统成本的解来解决实时服务卸载优化问题。令 $C_i^k(t)$ 表示第 i 个边缘服务器使用 Lyapunov 漂移加惩罚函数处理第 k 个服务的成本。我们给出以下定理 1。

定理 1: 第 i 个边缘服务器的实时卸载成本可以表达如下。

$$C_i^k(t) = V\theta E_i^k(t) + V(1-\theta)T_i^k(t) + Q_i^k(t)A_i^k(t) \tag{3-30}$$

其中 $E_i^k(t) = A_i^k(t)(e_u + e_s H_k(t)(1-x_i^k(t))) + e_p A_i^k(t)(P_c x_i^k(t) + P_{nc}(1-x_i^k(t))H_k(t))$ 。并且 $T_i^k(t) = A_i^k(t)/r(t) + t_k(P_c x_i^k(t) + P_{nc}(1-x_i^k(t))H_k(t))$ 。这两项分别表示第 k 个服务的能量消耗和延迟。

证明:

因为 $V \times \text{Cost}(t) = V\theta E_{\text{total}}(t) + (V - V\theta)T_{\text{total}}(t)$, 所以我们有

$$\begin{aligned}
 L_{\text{Cost}}(t) &= V \times \text{Cost}(t) + \sum_{i=1}^M \sum_{k=1}^K Q_i^k(t) A_i^k(t) \\
 &= \sum_{i=1}^M \sum_{k=1}^K V\theta E_i^k(t) + V(1-\theta)T_i^k(t) + Q_i^k(t)A_i^k(t)
 \end{aligned} \tag{3-31}$$

因此定理 1 被证明。

能量消耗和延迟模型都包含二元变量 $x_i^k(t)$ 。也就是说, 关键步骤是确定卸载位置, 即 $x_i^k(t)$ 的值等于 1。通过将 Lyapunov 优化引入 LDSO 方案, 动态服务卸载优化问题可以转化为匹配最优参数问题, 降低了 LDSO 方案的复杂度。令 $X(t)$ 表示 $x_i^k(t)$ 的匹配矩阵, 我们提出并讨论算法 1 中 LDSO 的实现。

3.4.2 基于贪心策略的匹配决策

为了解决上式中的优化问题, 边缘层以较低的系统成本确定服务的最优卸载位

置。由于数据分流与缓存服务基站的位置密切相关，因此快速识别多跳协同分流的最佳匹配至关重要。为了在保证系统性能和低时间复杂度的情况下实现这一目标，我们提出了基于贪心方法的 LDSO 算法。该算法能够以较低的成本实现服务卸载的实时决策，并降低 BS 和 SBS 之间的通信成本。

因本研究采用贪心思想来近似求得 $x_i^k(t)$ 的解。首先，第 4-10 行计算用户请求数、服务比例和传输速率。 $C_{i*}^k(t)$ 是集合 $\{C_i^k(t)\}_{K \times M}$ 中最小的一个。如果队列长度能够满足约束，并且节点缓存了第 k 类服务，则将匹配参数 $x_i^k(t)$ 设置为 1，并将 $\{C_{i*}^k(t)\}$ 将从集合 $\{C_i^k(t)\}_{K \times M}$ 中删除（第 12-16 行）。然后针对第 k 类服务，除第 i 个边缘服务器之外的其他服务器匹配该类服务的系统成本将被重置（第 18-19 行）。最后，更新系统成本和缓冲区队列状态（第 25-26 行）。

接下来，我们分析 LDSO 算法的收敛性，根据 Lyapunov 优化算法 LDSO 的收敛理论^[49,104]以及支持超平面定理^[105]，可以得到以下收敛性。定义 $\varepsilon = 1/V(\varepsilon > 0)$ ，并且对于 $T \geq 1/\varepsilon^2$ ，我们有定理 2。

定理 2： 如果满足以下两个条件，LDSO 提供 $O(\varepsilon)$ 近似，收敛时间为 $O(1/\varepsilon^2)$ 。

$$\bar{C}(t) = C^* + O(\varepsilon) \quad (3-32)$$

$$\bar{A}_i(t) = \mu + O(\varepsilon) \quad (3-33)$$

其中 C^* 表示近似最优解， $\bar{C}(t) = (1/T) \sum_{t \in [0, T-1]} E\{\text{Cost}(t)\}$ 且 $\bar{A}_i(t) = (1/T) \sum_{t \in [0, T-1]} E\{A_i(t)\}$ 。

证明： 根据 Lyapunov 优化^[49]和收敛理论^[104]，本文问题是凸的，因此构造一个等效的最小化问题。

$$\begin{aligned} & \min C \\ & \text{s.t. } A_i \leq \mu_i \leq \mu, \forall i \in I \\ & (C, A_1, A_2, \dots, A_M) \in \bar{R} \end{aligned} \quad (3-34)$$

其中 $C = E\{\text{Cost}(t)\}$ ， $A_i = E\{A_i(t)\}$ 。并且 $\bar{R}$ 是期望向量集 $(C, A_1, A_2, \dots, A_M)$ 的闭包。由支持超平面定理^[105]，得出存在通过 $\bar{R}$ 集合闭点的超平面 $(C^*, \mu, \mu, \dots, \mu)$ ，同时 $C^* = \text{Cost}^*$ 。此外，还存在一个支持超平面的非负法向量 $\Upsilon = (\gamma_0, \gamma_1, \dots, \gamma_M)$ ，满足以下不等式：

$$\gamma_0 C + \sum_{i \in I} \gamma_i A_i \geq \gamma_0 \times C^* + \sum_{i \in I} \gamma_i \mu \quad (3-35)$$

当 $\gamma_0 \neq 0$ 时, 支持超平面不垂直。将上述不等式除以 γ_0 并定义 $\delta_i = \gamma_i / \gamma_0$, 可得不等式如下:

$$C + \sum_{i \in I} \delta_i A_i \geq C^* + \sum_{i \in I} \delta_i \mu \quad (3-36)$$

其中 $(\delta_1, \delta_2, \dots, \delta_M)$ 是拉格朗日乘子向量^[105], $\bar{R}$ 是闭凸集。

算法 3-1 LDSO

输入: 控制参数 V , 权重系数 θ , 信道增益 $h(t)$, 边缘服务器计算能力 μ , 队列最大长度 $Q_i^{\max}(t)$

输出: $x_i^k(t)$

```

1.  $Q_i(t) = 0$ ;
2. for  $t = 0, t++, t < T$ 
3.    $X(t) \rightarrow [-1]_{K \times M}$ ; // 卸载决策矩阵
4.   for  $k \in [1, K]$  // 服务大小和该服务请求的用户数量
5.     获得  $n_i^k(t)$  和  $b_k$ ;
6.   end for
7.   for  $i \in [1, m]$  // 服务器遍历
8.     获得  $u_i(t)$ 
9.   end for
10.  根据  $h(t)$  获得  $q(t)$  // 获得信道增益和信道传输速率
11.  While  $\{C_i^k(t)\}_{K \times M} \neq NULL$  // 系统成本函数
12.     $C_{i^*}^{k*}(t) \rightarrow \min\{C_i^k(t)\}_{K \times M}$ 
13.    if  $(Q_i(t) + X_{i^*}^{k*}(t)n_i^k(t)b_k \leq Q_i(t)^{\max}) \&\& (X_{i^*}^{k*}(t) \neq -1)$ 
14.       $n_i^k(t) = n_i^k(t+1)$ 
15.       $C_i^k(t)_{K \times M} \leftarrow \{C_i^k(t)\}_{K \times M} - \{C_{i^*}^{k*}(t)\}$ 
16.       $X_{i^*}^{k*}(t) \leftarrow 1$ 
17.      for  $i \in \{1, 2, 3, \dots, M\} - \{i^*\}$ 
18.         $X_{i^*}^{k*}(t) \leftarrow 0$ 
19.         $C_i^k(t)_{K \times M} \leftarrow \{C_i^k(t)\}_{K \times M} - \{C_{i^*}^{k*}(t)\}$ 
20.      end for
21.    else
22.       $X_{i^*}^{k*}(t) \leftarrow 0$ 
23.    end if
24.  end while
25.   $Cost(t) = \theta \cdot E_{total}(t) + (1 - \theta)T_{total}(t)$ 
26.   $Q_i(t+1) \leftarrow \max\{Q_i(t) - u_i(t)\} + A_i(t)$ 
27. end for
    
```

$\bar{C}(t) = (1/T) \sum_{t \in T} E\{\text{Cost}(t)\}$ 是 C 的时间平均值, $\bar{A}_i(t) = (1/T) \sum_{t \in T} E\{A_i(t)\}$ 是 A_i 的时间平均值。因此 $(\bar{C}(t), \bar{A}_1(t), \bar{A}_2(t), \dots, \bar{A}_m(t)) \in \bar{R}$ 满足上面的不等式(3-36), 我们有

$$\begin{aligned} \bar{C}(t) + \sum_{i \in I} \delta_i \bar{A}_i(t) &\geq C^* + \sum_{i \in I} \delta_i \mu \\ C^* - \bar{C}(t) &\leq \sum_{i \in I} \delta_i (\bar{A}_i(t) - \mu) \end{aligned} \quad (3-37)$$

由上文可知, 边缘服务器的缓冲区队列大小满足以下不等式:

$$Q_i(t+1) \geq Q_i(t) + A_i(t) - \mu_i(t) \quad (3-38)$$

对所有时隙 $t \in \{0, 1, 2, \dots, T-1\}$ 伸缩级数求和, 则

$$\begin{aligned} (Q_i(T) - Q_i(T-1) + Q_i(T-1) - Q_i(T-2) + \dots + Q_i(1) - Q_i(0)) \\ + \sum_{t=0}^{T-1} \mu_i(t) \geq \sum_{t=0}^{T-1} A_i(t) \end{aligned} \quad (3-39)$$

然后, 两边除以 T , 我们有

$$\frac{1}{T} (Q_i(T) - Q_i(0)) + \frac{1}{T} \sum_{t=0}^{T-1} \mu_i(t) \geq \frac{1}{T} \sum_{t=0}^{T-1} A_i(t) \quad (3-40)$$

不等式两边取其期望, 对于 $T \rightarrow \infty$, 队列限制满足以下约束。

$$\lim_{T \rightarrow \infty} \frac{1}{T} E\{Q_i(T)\} + \bar{\mu}_i(t) \geq \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} A_i(t) \quad (3-41)$$

将式 (3-41) 代入式 (3-37), 可得

$$C^* \leq \bar{C}(t) + \sum_{i \in I} \delta_i \frac{E\{Q_i(T)\}}{T} \quad (3-42)$$

另一方面, 对 $t \in \{0, 1, 2, \dots, T-1\}$ 伸缩级数求和, 且 $\text{Cost}(t) > C_l$, C_l 为惩罚函数的下界, 因此有

$$\begin{aligned} E\{L(T)\} - E\{L(0)\} \\ \leq TB + TV \times C^* - V \sum_{t=0}^{T-1} E\{\text{Cost}(t)\} \\ \leq TB + TV \times C^* - VC_l \end{aligned} \quad (3-43)$$

两边除以 T , 根据不等式 (3-42), 我们有

$$\begin{aligned} \frac{1}{T} E\{L(T)\} &\leq B + V(C^* - C_l) \\ &\leq B + V(C^* - \bar{C}(t)) \\ &\leq B + V \sum_{i \in I} \delta_i \frac{E\{Q_i(T)\}}{T} \\ &\leq B + \frac{V}{T} \|\delta\| \|E\{Q(T)\}\| \end{aligned} \quad (3-44)$$

其中 $\|\cdot\|$ 是向量的 2-范数, 因为两个向量的点积小于或等于它们的范数积, 因此替换 $L(T)$ 和 $\|Q(T)\|^2 = \sum_{i=1}^M Q_i(T)^2$, 我们有

$$\frac{E\{\|Q(T)\|^2\}}{2T} \leq B + \frac{V}{T} \|\delta\| \|E\{Q(T)\}\| \quad (3-45)$$

由于 $E\{\|Q(T)\|^2\} \geq \|E\{Q(T)\}\|^2$, 所以得

$$\frac{1}{2T} \|E\{Q(T)\}\|^2 \leq B + \frac{V}{T} \|\delta\| \|E\{Q(T)\}\| \quad (3-46)$$

因此我们有

$$2\|E\{Q(T)\}\|^2 - 2V\|\delta\|\|E\{Q(T)\}\| - 2BT \leq 0 \quad (3-47)$$

根据一元二次方程, 我们有

$$\|E\{Q(T)\}\| \leq V\|\delta\| + \sqrt{V^2\|\delta\|^2 + 2BT} \quad (3-48)$$

根据 (3-41) 不等式, 我们有

$$\begin{aligned} \overline{A_i}(t) &\leq \mu + \frac{E\{Q_i(T)\}}{T} \\ &\leq \mu + \frac{V\|\delta\| + \sqrt{V^2\|\delta\|^2 + 2BT}}{T} \end{aligned} \quad (3-49)$$

接下来, ε 取值为 $1/V$, 并且当 $T \geq 1/\varepsilon^2$, 进一步被推导为

$$\begin{aligned} &\frac{V\|\delta\| + \sqrt{V^2\|\delta\|^2 + 2BT}}{T} \\ &= \frac{\|\delta\|}{\varepsilon T} + \sqrt{\frac{1}{\varepsilon^2 T^2} \|\delta\|^2 + \frac{2B}{T}} \\ &\leq \|\delta\| \varepsilon + \sqrt{\varepsilon^2 \|\delta\|^2 + 2B\varepsilon^2} \\ &= \|\delta\| \varepsilon + \varepsilon \sqrt{\|\delta\|^2 + 2B} \\ &= O(\varepsilon) \end{aligned} \quad (3-50)$$

(3-49) 被推导如下

$$\overline{A_{m_i}}(t) \leq \mu + \|\delta\| \varepsilon + \varepsilon \sqrt{\|\delta_i\|^2 + 2B} \leq \mu + O(\varepsilon) \quad (3-51)$$

另一方面, 根据不等式 (3-42), 我们有

$$\overline{C}(t) \leq C^* + B \cdot \varepsilon \leq C^* + O(\varepsilon) \quad (3-52)$$

因此，我们的算法能够满足（3-34）式的收敛约束，算法的收敛性也得到证明。

3.4.3 系统稳定性分析

在本节中，我们分析边缘卸载系统的 LDSO 算法的稳定性。根据排队论中的 Lyapunov 稳定性定理^[49]，当系统达到稳定状态时，可以得出以下定理。

定理 3： 如果系统成本满足以下不等式，则卸载性能保持稳定

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \text{Cost}(t) \leq C^* + \frac{B}{V} \quad (3-53)$$

证明：

基于（3-43）不等式，有

$$E\{L(T)\} - E\{L(0)\} + V \sum_{t=0}^{T-1} E\{\text{Cost}(t)\} \leq TB + TV \times C^* \quad (3-54)$$

除以 VT，并重新排列各项，有：

$$\frac{E\{L(T)\} - E\{L(0)\}}{VT} + \frac{\sum_{t=0}^{T-1} E\{\text{Cost}(t)\}}{T} \leq \frac{B}{V} + C^* \quad (3-55)$$

$$\frac{1}{T} \sum_{t=0}^{T-1} E\{\text{Cost}(t)\} \leq \frac{B}{V} + C^* - \frac{E\{L(T)\} - E\{L(0)\}}{VT} \quad (3-56)$$

其中， $L(0) < \infty$ 是一个有限数，并且 $E\{L(0)\} \geq 0$ 。取 $T \rightarrow \infty$ 得系统成本上限

因此定理 2 被证明。

定理 4： 边缘服务器缓冲队列强稳定性，满足以下不等式：

$$\limsup_{T \rightarrow \infty} \frac{1}{T^2} \sum_{t=0}^{T-1} \sum_{i=1}^M E\{Q_i(t)\}^2 \leq B + V(C^* - C_l) \quad (3-57)$$

强稳定性表明平均缓冲区的数据量存在上界，这严格限制了队列增长的速度。

证明： 由公式（3-43）和（3-20）可知，我们有

$$\begin{aligned} & \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i=1}^M \frac{1}{2} \{Q_i(t)\}^2 - \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i=1}^M \frac{1}{2} \{Q_i(0)\}^2 \\ & \leq TB + TV(C^* - C_l) \\ & = T(B + V(C^* - C_l)) \end{aligned} \quad (3-58)$$

我们令 $Q_i(0) = 0$ ，因此队列的强稳定性被证明。

如上所述，Lyapunov **Drift-Plus-Penalty** 函数值具有稳定性保证。并且保证了队列稳定性。系统时间平均成本的时间复杂度为 $O(V)$ ，而缓冲队列时间平均长度的复杂度

为 $O(\frac{1}{V})$ 。因此，控制参数 V 可以有效调节平均卸载成本和缓冲队列长度之间的权衡。

3.5 实验

采用上海电信数据集模拟边缘服务卸载系统，如图 3-4 所示，其中蓝色的 SBS 代表存储第 k 种服务的 SBS，红色代表没有缓存该类型服务的 SBS。初始设置时，服务类型数量等于 10。每个 SBS 覆盖 50 个用户，用户与 SBS 之间的通信范围为 15m，SBS 之间的通信范围为 30m。模拟了 1000 个时隙，每个时隙的持续时间为 1ms。链路带宽为 40MHz，通道增益设置为 20dB。每个物联网设备的发射功率设置为 $P_u(t) \sim U[0.01, 1]W$ 。系统稳定性评估中，评估控制参数 V 对缓冲队列的影响，其中时变服务请求服从平均值为 20 的泊松分布，权重因子设置为 $\theta = 0.5$ 。

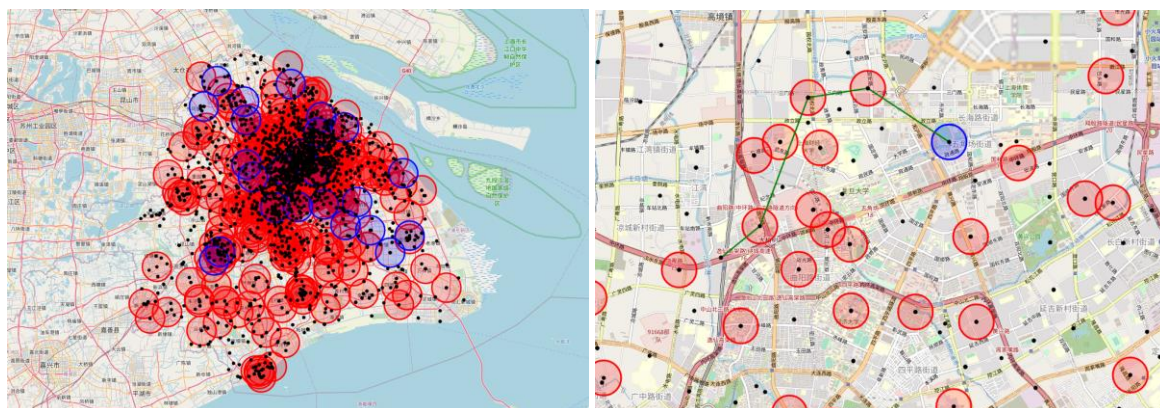


图 3-4 实验场景

3.5.1 系统稳定性

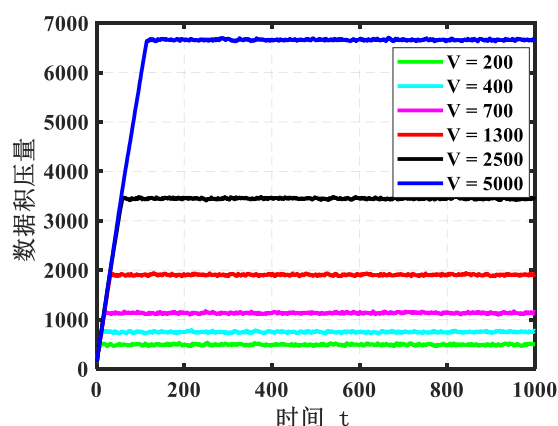


图 3-5 不同 V 值下的系统稳定性

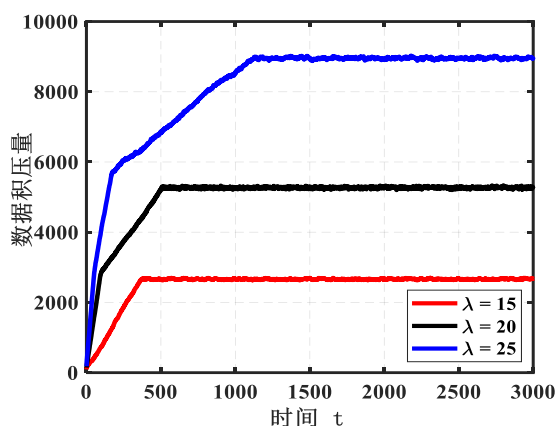


图 3-6 不同 λ 值下的系统稳定性

(1) 算法收敛性分析：本节分析了 LDSO 的收敛性，在控制参数 V 和服务到达率

λ 下的实验结果分别如图 3-5 和图 3-6 所示。这些数字中存在三个现象。第一个是两个关键参数在算法收敛性中扮演着不同的角色。一般情况下，控制参数 V 具有细粒度的调节作用，而到达率 λ 对算法收敛性的影响往往是粗粒度的。例如，LDSO 仅使用 50 个时隙和 100 个时隙即可达到图 3-5 中 $V=1300$ 和 $V=5000$ 值下的稳定性。也就是说，系统稳定性的比较明显的变化需要 V 的显著波动。相反，当图 3-6 中的 λ 值分别为 15 和 25 时，需要大约 350 和 1200 个时隙才能达到稳定。

第二个是 LDSO 在小时间窗口内很快达到稳定，验证了定理 2 的正确性和有效性。第三个是随着两个参数的增加，收敛速度变慢。原因是这两个参数越大，缓冲的数据就越多，需要花费更多的时间来处理数据。

我们现在将 LDSO 与最新的研究工作进行比较。比较算法是 DSARA^[71] 和 MECNC^[77]，它们都使用 Lyapunov 理论考虑成本模型和系统稳定性。

3.5.2 系统控制参数 V 的影响

(2) 控制参数 V 对系统成本的影响：我们首先分析三种算法的系统成本。图 3-7 和图 3-8 分别绘制了有/没有队列长度约束的实验结果（图 3-7 中队列长度以 4000 为界）。可以看出，当 V 值增大时，系统成本呈现下降趋势，验证了定理 3 的正确性，即

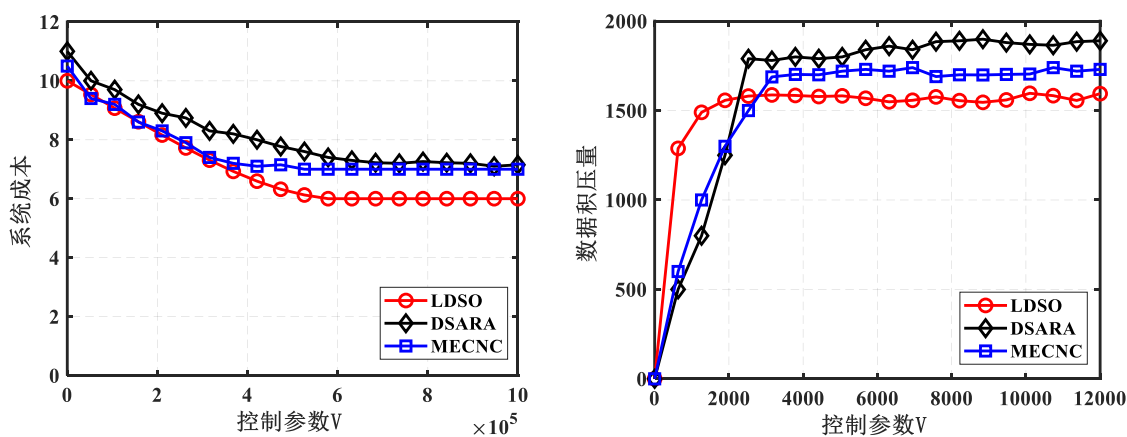


图 3-7 不同 V 值下的系统成本 ($Q_{m_i}^{\max}(t) = 4000$). 图 3-8. 不同 V 值下的数据积压 ($Q_{m_i}^{\max}(t) = 4000$). V 越大意味着较小的成本，如等式 (3-55) 所示。具体来说，与 DSARA 和 MECNC 相比，LDSO 实现了最小的系统成本。图 3-7 中平均降低了 10% 左右的成本，如果队列长度不受限制，图 3-8 中可以实现更多改进。

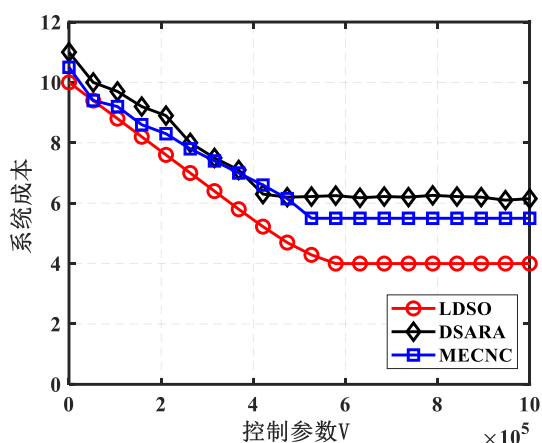


图 3-9 不同 V 值下的系统成本

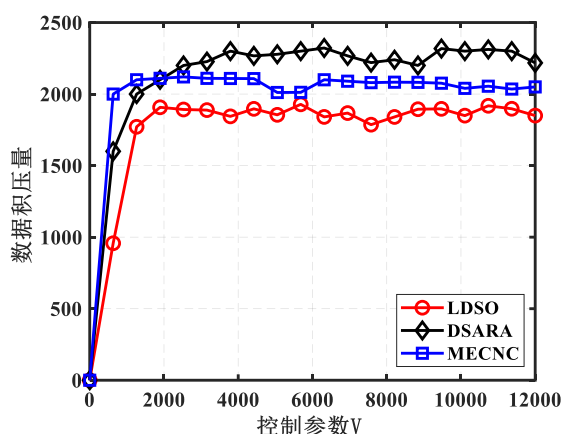


图 3-10 不同 V 值下的数据积压

(3) 控制参数 V 对缓冲数据量的影响：这部分分析了三种算法的缓冲数据量，实验结果如图 3-9 和图 3-10 所示。一般来说，缓冲数据量缓冲数据随着 V 值的增加而增加，这验证了等式 (3-59) 中的定理 4。即更大的 V 导致更多的缓冲数据可以缓存在边缘服务器中。与 DSARA 和 MECNC 相比，LDSO 缓冲的数据最少。图 3-9 中的缓冲区大小约为 1600，图 3-10 中的缓冲区大小约为 1900，这可以归因于队列约束所施加的相对严格的控制，从而获得较低的缓冲区占用。图 3-7，图 3-8，图 3-9，图 3-10 的结果验证了控制参数 V 对系统成本和缓冲数据有有效的调节作用。

3.5.3 卸载系统的效用

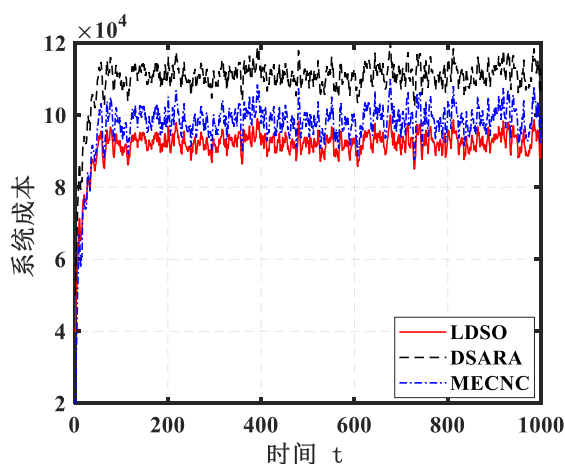


图 3-11 长时间内的系统成本

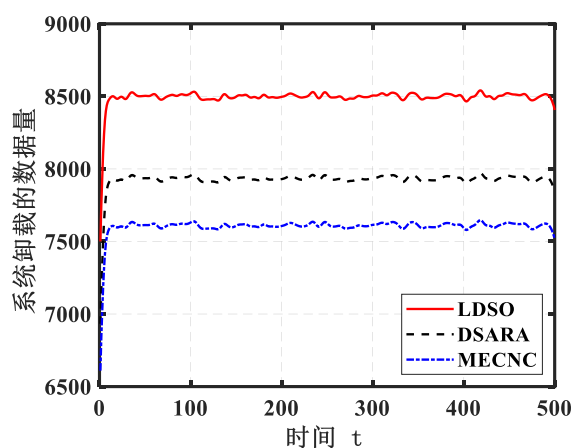


图 3-12 长时间数据卸载

(4) 三种算法的卸载效用：我们现在评估三种算法在长期演进中的系统利用率。请注意，服务数据处理的启动伴随着系统处理的服务请求数量的显着增加。反过来，这会导致计算和通信成本迅速上升，同时卸载数据量也会成比例增加，如图 3-11 和图 3-12 所示。具体来说，与 MECNC 和 DSARA 相比，LDSO 实现了系统成本最低，与图 3-8 和图 3-9 的结果类似。不同的是，系统成本先趋于上升，然后达到稳定，如图 3-11

所示。卸载量数据绘制在图 3-12 中。可以看出，三种算法中 LDSO 卸载的数据量最大。8500、7900 和 7600 数据块分别由 LDSO、DSARA 和 MECNC 卸载。综合这两个数字，可以得出一个简短的结论：LSDO 降低了系统成本 10%，同时卸载数据量平均增加了 18.75%，这表明与之前的算法相比，该算法可以以更低的系统成本处理更大的数据量。另外两个方案。因此，我们的方法表现出更好的实用性和可持续性。

3.6 本章小结

本章研究了边缘服务器之间协作的实时边缘卸载系统中的成本最小化问题。利用 Lyapunov 优化框架，提出了一种在线动态服务卸载算法 LDSO，以实现合理的服务决策。LSDO 不需要先验知识，实现了系统稳定性和卸载成本之间的权衡。深入的理论分析和广泛的实验结果证明了 LDSO 的有效性。

第四章 基于 DRL 自适应 Lyapunov 优化的云边协同服务在线卸载策略

在第三章多边缘服务器协作数据处理基础之上，本章又考虑了云节点。考虑到为多变量决策，以及通信场景的目标函数计算的复杂度，因此基于 Lyapunov 进行问题转化，采用 DRL 方法解决问题。通过设计系统状态以及动作，并将转化后的目标函数上界嵌入到带有折扣因子的奖励函数，并采用 SAC 算法进行云边系统策略的训练。实验结果表明本文策略在与环境进行实时交互的过程中，不仅保证了系统的稳定性，也降低了系统能耗。

4.1 引言

Lyapunov 优化是在线计算卸载和资源分配的强大工具^[95]。而目前通过 Lyapunov 函数优化控制的实时多边缘服务器在线计算卸载动态系统，虽然稳定了网络节点队列和最小化了系统操作的代价^[49,96,97]，且也不需要知道随机事件过程的概率分布就具有较低的计算复杂度^[98]。然而针对大型应用程序的卸载，边缘层的算力也是有限的，因此基于 Lyapunov 优化框架下的云边协同服务智能卸载方式应用而生，然而其中 Lyapunov 优化与 RL 的结合是目前大家研究热点主题。RL 的目标是学习一种能够使累积的预期回报最大化的策略^[75]，因此从长期系统性能考虑，不仅提高了系统实时算力，也降低了服务响应延迟。然而，目前在实时物联网系统中，网络状态是高度动态与随机的，例如无线信道的时变性、不断增长的服务请求等，都会直接影响数据的传输能耗，即在不良网络连接下传输会消耗更多的能量^[71]。因此，在与动态环境不断地实时交互过程中，有效的在线计算卸载策略对系统能耗的优化显得尤为重要，同时也引起了相关研究人员的广泛关注。

目前基于 Lyapunov 优化的在线协作计算卸载仍存在挑战。一方面为优化过程中的实时卸载需要每一时隙求局部最优解，即瞬时贪婪问题。然而每一时隙下的全局最优系统连续控制才是反应卸载策略长期有效性的关键所在，因此，解决每一时隙全局最优问题并不容易。另一方面，随着 DRL 的发展，以及 RL 在许多网络控制领域的应

用，再次成为研究人员的研究热点。然而对于网络系统时隙之间的连续控制问题，经典的强化学习解决方案并不那么有效^[99,102]。虽然 RL 模块能够有效地学习长期 Lyapunov 漂移函数的上界与近似解，但是由于基于 Lyapunov 的在线优化方法具有时差性质，以及 RL 的时隙之间的状态变换也是动态的，因此针对动态环境下的系统操作时序设计，以及将 Lyapunov “漂移函数+惩罚项”嵌入到 DRL 的奖励函数的再设计都是困难的。尤其是在处于动态的网络系统中，只选择网络状态最佳时刻进行数据分流处理，虽然可以降低系统一定程度上的能耗，但不符合实际通信场景。并且该数据分流方式会导致边缘服务器负载不均衡问题，增加了网络系统能耗，因此在设计有效的节能策略方面增加了新的挑战。为了解决上述挑战，本文研究了云边协同服务智能卸载问题。

4.2 边缘云系统模型架构

本文考虑一个边缘云系统，即包含一个远程云中心（云节点）和边缘层的 M 个边缘服务器（边缘节点），以及 K 个用户。其中，终端用户服务的请求属于这 K 种应用程序。当用户发送服务请求，即对应该类应用程序的数据通过上行链路传输，并卸载到离用户最近的边缘节点，如果该边缘节点未缓存服务，则通过边缘层多个节点协作进行处理，当边缘层计算资源不足时，进一步通过边缘层节点与云节点之间的通信链路协作进行数据分流处理。系统框架图如图 4-1 所示。

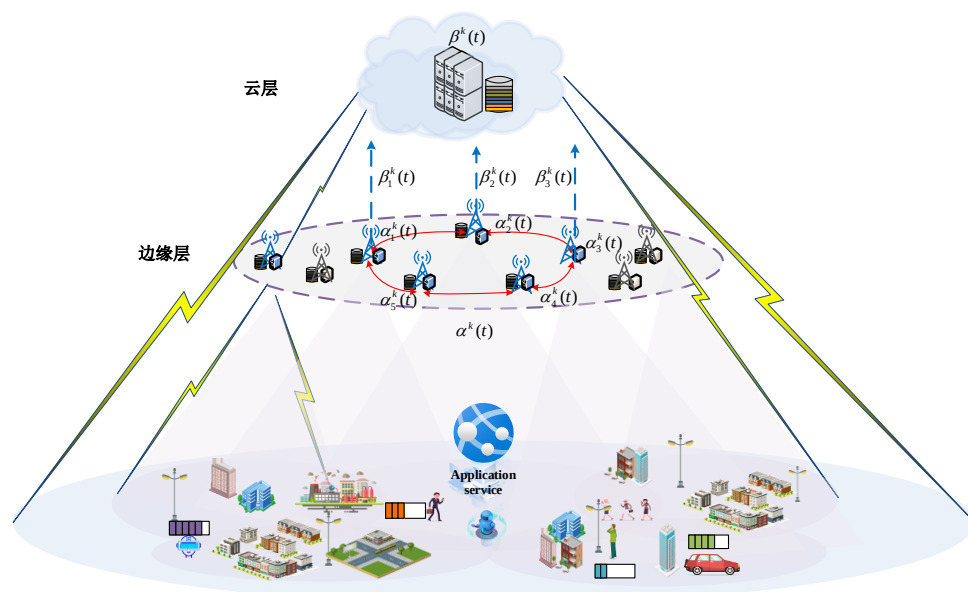


图 4-1 边缘云系统模型框架图

本文假设用户请求的第 k 类服务遵循泊松分布到达率 λ_k 。其中系统处理服务请求

的运行时间为离散时隙。它被分成 T 个等长的部分，每个部分的槽长度为 τ ，即时间索引为 $t \in T = \{0, 1, 2, 3, \dots, T-1\}$ 。系统提供第 k 类服务的边缘服务器共有 a_k 个， $C_i^k = \{C_1^k, C_2^k, \dots, C_M^k\}$ 是边缘节点服务放置向量，指示第 k 类服务是否缓存在第 i 个边缘节点中，因此 $\sum_{i=1}^M C_i^k(t) = a_k$ 是本系统第 k 类服务提供者数量。针对服务卸载，存在三种卸载情况：（1）仅边缘层多边缘服务器协作处理，（2）云边协同处理，（3）仅云节点处理。因此系统资源分配第 k 类应用程序的两个控制动作变量 $\alpha^k(t)$ 和 $\beta^k(t)$ 由下式给出，即

$$\alpha^k(t) = [\alpha_1^k(t), \dots, \alpha_i^k(t), \dots, \alpha_M^k(t)] \quad (4-1)$$

$$\beta^k(t) = [\beta_1^k(t), \dots, \beta_i^k(t), \dots, \beta_M^k(t)] \quad (4-2)$$

其中 $\alpha_i^k(t)$ 为第 i 个边缘服务器卸载第 k 类应用程序卸载的 CPU 计算资源比例， $\beta_i^k(t)$ 为该边缘服务器上传到云节点数据的带宽比例，即 $\alpha^k(t) = \sum_{i=1}^M \alpha_i^k(t) = 1$ 代表边缘层节点协作处理应用程序服务 k 数据的卸载，而云节点不参与。当 $0 \leq \alpha^k(t) < 1$ ，代表边缘层多节点协作处理数据的比例为 $\alpha^k(t)$ ，云节点处理剩余数据的比例为 $\beta^k(t)$ 。由于考虑到每个边缘节点缓存部分应用程序服务，因此在边缘层拥有该类服务的计算资源的多节点协作处理服务请求过程中，针对第 i 个边缘服务器进一步表示 $\alpha_i^k(t) = \alpha_{ii}^k(t) + \sum_{j=1}^M \alpha_{ij}^k(t)$ ，其中， $\alpha_{ii}^k(t)$ 表示对于第 k 类服务边缘服务器 i 自身的资源分配比例，而 $\alpha_{ij}^k(t)$ 代表 t 时刻第 j 个边缘服务器协作第 i 个边缘服务器处理第 k 类服务的计算资源比例，因此代表边缘层多节点协作共同卸载应用程序。如果 $\alpha^k(t) = 0$ ，则代表云中心处理，此时 $\beta^k(t) = 1$ 。其中，云中心包含 K 个独立的应用程序服务数据库 $k \in K = \{S_1, S_2, \dots, S_K\}$ 。

本文使用的主要参数如表 4-1 所示。

4.3 系统建模与问题制定

4.3.1 系统建模

（1）队列模型

为了保证系统稳定，我们为每个应用程序服务维护了一个数据队列。那么 $t+1$ 时

表 4-1 主要符号

符号	说明	单位
K	应用程序数量	-
f^e	边缘节点 CPU 最大时钟周期速率	GHz/s
a_k	边缘层协作节点数量	-
B	边缘节点与云节点间的通信带宽	bits/s
λ_k	的泊松到达率	arrivals/s
$f_{j,k}^e$	对于第 k 个 App 被分配的第 j 个边缘节点的时钟周期	GHz/bit
$A_k(t)$	在时刻 t 的 $W(t)$ 任务比特到达量	bits
$\mu_k(t)$	第 k 个 App 在时刻 t 被处理的数据量	bits
$d_e^k(t)$	在时刻 t 下边缘层节点对于第 k 个 App 处理的数据量	bits
w_k	第 k 个单位数据处理所需的 CPU 周期数	GHz/bit
$Q_k(t)$	在时刻 t 第 k 个 App 的队列长度	bits
$\alpha^k(t)$	在时刻 t , 边缘层给予第 k 个 App 的 CPU 资源分配比例	-
$\beta^k(t)$	在时刻 t 对于第 k 个 App 再上传云节点过程中, 被分配的通信带宽比例	-
$W^e(t)$	在时刻 t 下的边缘层所有节点计算能耗	W
$W_{j,k}^e(t)$	在时刻 t 下的第 j 个边缘层节点计算能耗	W
$W^c(t)$	在时刻 t 下的云节点计算能耗	W

刻第 k 个应用程序动态数据队列长度 $Q_k(t+1)$ 表示为:

$$Q_k(t+1) = \max \left\{ Q_k(t) - \underbrace{(d_e^k(t) + d_c^k(t))}_{=\mu_k(t)}, 0 \right\} + A_k(t) \quad (4-3)$$

其中 $d_e^k(t)$ 和 d_c^k 分别为边缘层多节点处理的数据量和云节点处理的数据量。其中 (3) 右侧中的第二项、第三项和第四项分别表示边缘层多节点协作处理的、卸载到云节点的以及时间 t 新到达的数据量。同时定义为 $\mu_k(t) = d_e^k(t) + d_c^k(t)$ 。在时隙 t 节点队列的长度, 即数据的积压量。在时刻 t 终端第 k 个应用程序服务 (或简称第 k 个队列) 的到来的任务数据位记为 $A_k(t)$, 即 $A_k(t)$ 是在 t 时刻到达的任务数据位 (bit) 的总和。

由于不同的应用程序服务需要不同数量的 CPU 周期来处理一位任务数据, 所以我们假设第 k 个应用程序服务的一位任务处理需要 w_k 个 CPU 时钟周期。另外我们假设边缘节点的最大 CPU 处理时钟速率为每秒 f^e 个周期, 边缘节点与云节点之间的通信带宽为每秒 Bbits。因此边缘层第 i 个边缘节点分配给第 k 个应用程序服务的时钟周期为 $\alpha_i^k(t)f^e$, 且在 t 时隙卸载到云节点的数据量为 $\beta_i^k(t)B$, 同时系统中第 k 类应用程序资源分配的两个控制变量 $\alpha^k(t)$ 与 $\beta^k(t)$ 满足以下约束:

$$\begin{aligned}\forall t \in T, \alpha^k(t) &= \sum_{i=1}^M \alpha_i^k(t) \leq 1 \\ \forall t \in T, \beta^k(t) &= \sum_{i=1}^M \beta_i^k(t) \leq 1\end{aligned}\quad (4-4)$$

本文边缘云系统中，由于每个队列中每一时隙数据量的差值是由两个单独的系统操作 $\alpha^k(t)$ 与 $\beta^k(t)$ （计算和卸载）产生的。因此边缘层的多节点和云节点分别处理的数据量 $d_e^k(t)$ ， $d_c^k(t)$ 分别如下所示：

$$d_e^k(t) = \left(a_k f^e \cdot \sum_{i=1}^M \alpha_i^k(t) \right) / w_k \quad (4-5)$$

$$d_c^k = B \sum_{i=1}^M \beta_i^k \quad (4-6)$$

在时刻 t ，第 i 个边缘节点卸载到云节点的该服务实际数据量如下式所示：

$$D^k(t) = \min(d_c^k(t), Q_k(t) + A_k(t) - d_e^k(t)) \quad (4-7)$$

这是因为边缘层节点在约束条件下处理部分数据后，则剩余的数据量可能小于卸载到云节点的数据量 $d_c^k(t)$ 。

为了保证系统稳定运行，队列需要在每一时隙达到一个稳定的状态。因此关于排队网络的队列稳定性^[49]，需要满足以下约束：

$$\limsup_{t \rightarrow \infty} \frac{1}{t} \sum_{\tau=0}^{t-1} E\{Q_k(\tau)\} < \infty \quad (4-8)$$

其中 $Q_k(t)$ 是时刻 t 时队列的长度。满足以上约束条件的队列 $Q_k(t), k \in \mathbb{K} = \{1, \dots, K\}$ 都是稳定的，边缘云系统也因此是稳定的。

(2) 能耗模型

系统能耗 $W(t)$ 包含两部分：边缘层多节点计算能耗 $W^e(t)$ ，云节点计算能耗 $W^c(t)$ ：

那么系统能耗表示如下：

$$W(t) = W^e(t) + W^c(t) \quad (4-9)$$

① 边缘层多节点计算能耗 $W^e(t)$

假设每个边缘服务器只有一个 CPU 内核，由系统模型已知边缘服务器具有不同的计算能力，对于计算简单的任务，系统需要较少的 CPU 周期来处理。而对于计算要求较高的任务，CPU 时钟周期需要的较多。在我们的假设情况下边缘层服务器有最大的时钟频率为每秒 f^e 个周期，并且在边缘服务器协作的情况下，也就意味着有 a_k 个 CPU

内核工作，则意味着第 k 类应用程序服务每秒最多被分配计算资源（时钟周期）为 $a_k f^e$ 。因此，根据公式（4-1）和（4-4），在时刻 t 边缘层第 k 种应用程序服务被分配的资源为 $a_k f^e \cdot \sum_i^M \alpha_i^k(t)$ 。而第 i 个边缘服务器被分配的内核 CPU 时钟周期由下式给出：

$$f_{i,k}^e = F(f^e, a_k, \alpha_1^k(t), \dots, \alpha_M^k(t)), i \in [1, \dots, M] \quad (4-10)$$

其中 $F(\cdot)$ 为边缘节点 CPU 单核处理数据的函数，取决不同设备 CPU 硬件设计。

功耗由两个主要部分组成：动态功耗和静态功耗。CPU 的静态功耗是 CPU 时钟周期的线性函数，并且值很小，因此我们忽略静态功耗的开销。所以我们重点关动态功耗，边缘节点动态功耗表示如下：

$$P_i^e = \kappa f^3 \quad (4-11)$$

其中 κ 是功耗常数，取决于 CPU 完成工作负载的具体实现， f 是 CPU 内核的实际时钟速率。因此边缘层的总体功耗为：

$$W_k^e(t) = \sum_{j=1}^{a_k} W_j^e(t) \quad (4-12)$$

其中 $W_j^e(t)$ 表示协作的第 j 个边缘节点 CPU 计算功耗，由 (4-11) 给出，内核工作时钟频率 f 由 (4-10) 代替，因此 $W_j^e(t) = k(f_{j,k}^e)^3$ 。请注意，对于给定的 f^e 和 a_k ，所以针对第 k 个应用程序 t 时刻的功耗 $W_k^e(t)$ 的是控制动作变量 $\alpha_i^k(t)$ 的函数，明确表示为

$$\begin{aligned} W_k^e(t) &= W_k^e(\alpha_1^k(t), \alpha_2^k(t), \dots, \alpha_{a_k}^k(t)) \\ &= \sum_{j=1}^{a_k} k(f_{j,k}^e)^3 \end{aligned} \quad (4-13)$$

因此边缘层整体能耗为：

$$W^e(t) = \sum_{k=1}^K W_k^e(t) \quad (4-14)$$

② 云节点计算能耗 $W^c(t)$

基于公式（4-11）、（4-12）和（4-13），再根据云节点卸载的第 k 类应用程序服务数据量 $D^k(t)$ 来定义能耗函数 $W_k^c(t)$ ，其中 $D^k(t)$ 由等式（4-7）给出。

由于 $W_k^c(t)$ 大小取决于应用程序服务卸载的两个动作 $\alpha^k(t)$ 和 $\beta^k(t)$ ，因此包含有控制变量的云节点计算能耗函数 $W_k^c(t)$ 如下所示。

$$\begin{aligned} W_k^c(t) &= W_k^c(\alpha^k(t), \beta^k(t)) \\ &= \kappa(w_k D^k(t))^3 \end{aligned} \quad (4-15)$$

因此云节点的整体计算能耗为：

$$W^c(t) = \sum_{k=1}^K W_k^c(t) \quad (4-16)$$

本文系统能耗 $W(t)$ 以瓦特为单位。在后面章节中，我们将使用 $W^e(t)$ 和 $W^c(t)$ 作为 Lyapunov 惩罚项。

4.3.2 基于 Lyapunov 的问题制定

我们制定了系统队列稳定性下应用程序服务的最优卸载策略问题。由于任务到达过程是随机的，因此能耗函数的优化也是随机的。为了长期最小化平均系统能耗，同时保证系统的长期稳定运行，本文优化问题表述如下：

$$P_1 : \min \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} E\{W(t)\} \quad (4-17)$$

s.t.

$$\forall k \in \mathbb{K}, \limsup_{t \rightarrow \infty} \frac{1}{t} \sum_{\tau=0}^{t-1} E[Q_k(\tau)] < \infty \quad (4-18)$$

$$\forall k \in \mathbb{K}, t \in T, \sum_{k=1}^K \alpha^k(t) \leq 1, \sum_{k=1}^K \beta^k(t) \leq 1 \quad (4-19)$$

其中 $W(t)$ 为边缘层多节点和云节点协作卸载的能耗函数。 $Q_k(t)$ 为其第 k 个应用程序服务的队列长度，（4-18）作为约束条件，使其队列达到强稳定的状态。（4-19）为针对第 k 类应用程序服务资源分配比例约束。

接下来，进行基于 Lyapunov 的问题制定。关于 Lyapunov 函数定义与 Lyapunov 漂移函数如 2.4.1 节公式（2-20）和（2-21）所示。注：本章节的 i 为应用程序的序号。

ω_k 为第 k 类应用程序服务平均数据包大小，根据排队论原理，只有在服务速率大于到达率时，对于 P_1 的优化才可行，即对于 $\varepsilon > 0$ 和常数 $\overline{d_e^k}$ （多边缘节点处理的第 k 类服务数据量平均值）， $\overline{d_c^k}$ （多边缘节点处理的第 k 类服务数据量平均值）， $k \in \mathbb{K}$ 。因此 $\lambda_k u_k - \overline{d_e^k} - \overline{d_c^k} < -\varepsilon$ ，系统需要给定服务速率的值来保证稳定运行以及性能优化。而本文的优化问题（包含 $\alpha^k(t)$ 与 $\beta^k(t)$ 的速率），使用传统的 Lyapunov 优化算法是无法解决的。传统的 Lyapunov 漂移加惩罚函数如下：

$$\Delta L(t) + VE \{W(t) | Q(t)\} \quad (4-20)$$

其中控制参数 V 实现了系统队列积压与优化目标系统能耗函数的权衡。时隙之间的队列变化 $\Delta L(t)$ 即为漂移项。因此，在我们的策略中，基于 Lyapunov 漂移加惩罚函数上限表示如下：

$$\begin{aligned} P_2: \Delta L(t) + VE \{W^e(t) + W^c(t) | Q(t)\} \\ \leq \frac{1}{2} \sum_{k=1}^K (A_k(t) - \mu_k(t))^2 \\ + \sum_{k=1}^K Q_k(t) (A_k(t) - \mu_k(t)) \\ + VE \{W^e(\alpha^k(t)) + W^c(\alpha^k(t), \beta^k(t)) | Q(t)\} \end{aligned} \quad (4-21)$$

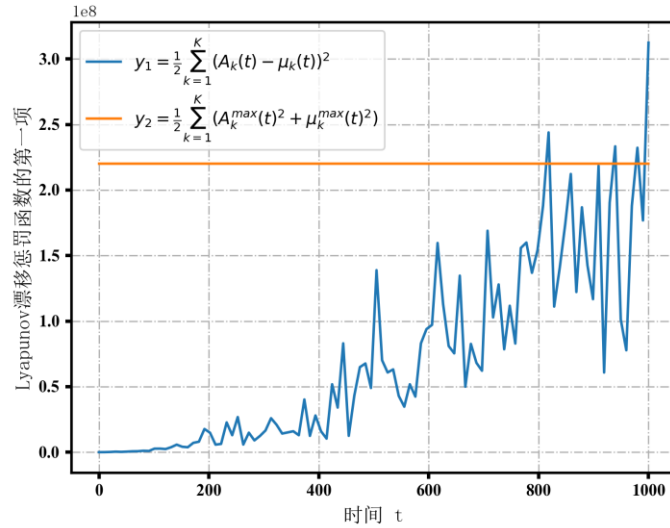


图 4-2 Lyapunov 漂移惩罚函数的第一项

以上传统的 Lyapunov 每一时隙都要求出最优解，并且寻找一个满足系统稳定性的局部贪心近似最优解，使漂移加惩罚的优化目标值落在阈值范围之内。从不等式右侧可以看出第一项为到达的数据量与可以处理的数据量之差的平方，第二项为队列长度与到达量与计算量的差的乘积。以往 Lyapunov 在线优化相关研究工作中经常将第一项 y_1 以最大上限值 y_2 代替（如图 4-2 可示），然后只考虑第二项整体或者第二项内部的队列与到达数据量的乘积。由于最大上限值是一个固定值，不能反映动态系统实际的处理量，且并未根据现实网络中出现的动态数据进行自适应处理。因此本文提出了本文基于 DRL 的解决方案。

4.4 基于 DRL 的解决框架

传统的 Lyapunov 优化方法在求解漂移加惩罚项上界时通常采用近似方法。然而，在大多数情况下，算法设计只考虑到上界的部分项，通过伸缩求和得到瞬时最优解，这种方法在面对连续时隙预测场景下愈发明显，以及对于更复杂的惩罚函数项计算时会变得更加困难。因此，本研究采用深度强化学习（DRL）方法来解决这个问题。通过训练系统控制策略并记录系统动态演化轨迹，无需在系统运行过程中一直进行局部最优解的计算。这种方法能够更好地应对复杂的优化问题。

首先 RL 由五部分组成。1) 动作空间 $\mathcal{A}$ 。2) 状态空间 S 。3) 状态转移概率 $P: S \times \mathcal{A} \rightarrow S$ 。4) 策略函数 $\pi: S \rightarrow \mathcal{A}$ 。5) 奖励函数 $R: S \times \mathcal{A} \rightarrow \mathbb{R}$ 。智能体在时刻 t ，采用已有的策略 π 观察环境状态 $s_t \in S$ ，然后采取动作 a_t ，即 $a_t \in \mathcal{A} \sim \pi(a_t | s_t)$ ，然后根据当前状态 s_t 和智能体动作 a_t 得到奖励 $R_t = R(s_t, a_t)$ ，继而根据状态转移概率，出现一个新的环境状态 s_{t+1} ，即 $s_{t+1} \sim P(s_{t+1} | s_t, a_t)$ 。最终的目标是通过学习和训练获取最优的控制策略，从而实现最大化期望回报，即奖励。

根据系统模型和能耗函数，我们将问题描述为一个包含队列变化和 Lyapunov 漂移加惩罚项上界的强化学习（RL）问题。在该问题中，SAC 算法被视为智能体，尝试学习一种策略来稳定队列的稳定性。本研究并不涉及到状态转移概率 P 的相关知识，因此提出的方法是基于无模型的强化学习（RL）。设计状态空间、动作空间和奖励函数的过程具有一定难度，因此为了解决我们所提出的优化问题，我们将队列约束纳入 RL 框架中。这样可以确保在学习过程中智能体能够考虑到队列的稳定性，从而实现对优化问题 P_2 的解决。

4.4.1 状态与动作空间设计

根据利特尔定律，平均延迟与平均队列长度成正比。因此，强稳定的数据队列转换为每个任务数据位的有限处理延迟。基于状态与动作的因果关系与排队论，假设每个任务数据位的有限处理延迟。基于状态与动作的因果关系与排队论，假设每个完整动作都存在一个时间步长的延迟，即时隙间队列长度的变化。本文采用离散时隙 t 作为连续时刻 t 的标志，按照时序关系，如图 4-3 所示， $Q_k(t)$ 为连续时刻 t 的队列长度，而 $A_k(t)$ 是在时间间隔内 $[t, t+1)$ 到达的数据量总和，计算的数据量 $\mu_k(t)$ 作为在连续时间

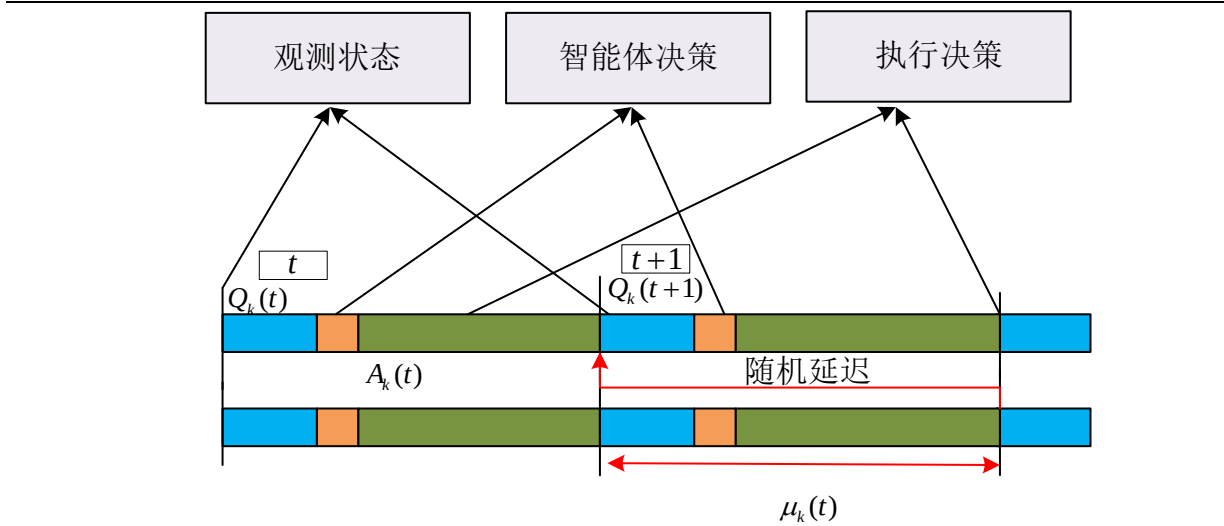


图 4-3 为系统操作时序图

间隔 $[t+1, t+2)$ 对 $Q_k(t) + A_k(t)$ 的观察。 t 时刻的系统计算量 $\mu_k(t)$ 将在时刻 $t+1$ 改变队列的长度，因此整合到 $Q_k(t+1)$ 。同时针对第 k 种应用程序服务，那么基于 RL 的离散时间 t 索引处的边缘云系统被确定的状态变量如下所示：

关键状态变量：队列长度： $Q_k(t) + A_k(t), k \in \mathbb{K}$ 。

我们将在 t 时刻之前的 $Q_k(t)$ 以及 t 时刻要到达的 $A_k(t)$ 都包含在系统状态集合中。

第 k 类应用程序服务单位数据所需的计算资源： $w_k, k \in \mathbb{K}$ 。

第 k 个队列在时刻 $t-1$ 的实际卸载比例为 $\alpha^k(t), k \in \mathbb{K}$ 。

注：时刻 t 针对第 k 类服务 CPU 使用量为 $a_k f^e \alpha^k(t)$ 。然而，当第 k 个队列中的数据量小于值 $(a_k \cdot f^e \alpha^k(t)) / w_k$ 时，此时动作对第 k 个队列的实际 CPU 使用情况表示为 $a_k \cdot f^e \alpha^k(t) < a_k \cdot f^e \alpha^k(t)$ 。此时刻我们将队列长度视为 0，在队列动力学^[49]中，定义由公式 $[x]^+ = \max\{x, 0\}$ 处理。

时刻 t 云节点处理任务所需要的 CPU 周期 $\sum_{k=1}^K w_k D^k(t)$ 。

卸载策略控制动作变量为其中 $\alpha^k(t)$ 和 $\beta^k(t)$ ， $k \in \mathbb{K}$ ，动作约束为 $\sum_{k=1}^K \alpha^k(t) \leq 1$ 和 $\sum_{k=1}^K \beta^k(t) \leq 1$ 。

本文将连续时间 $(t, t+1]$ 的间隔作为参考时间，基于 $Q_k(t) + A_k(t)$ 状态变量，应用程序服务卸载操作处理的数据量 $\mu_k(t)$ 可以被观察，因此更新系统状态，我们获得：

当前状态: $s_t = \{Q_k(t) + A_k(t), \dots\}$

当前动作: $a_t = (\alpha^k(t), \beta^k(t))$ 取决于 $\mu_k(t)$

当前奖励: 根据状态 s_t 和动作 a_t 的奖励函数 $R_t(s_t, a_t)$

更新队列: $Q_k(t+1) = [Q_k(t) - \mu_k(t)]^+ + A_k(t)$

更新状态: $s_{t+1} = \{Q_k(t+1) + A_k(t+1), \dots\}$

在强化学习中, 针对系统运行的时间和状态的定义至关重要, 因此我们将针对以上设计在下一小节 4.4.2 进行更详细的叙述。

4.4.2 奖励设计

根据 4.2 问题陈述, 我们首先将 t 时刻 Lyapunov 漂移加惩罚上界作为目标奖励函数 $R_t(t)$, 因此 RL 奖励函数 $R_t(t)$ 分为两部分, 即系统稳定 R_t^Q 和惩罚目标函数 R_t^C , 表示为:

$$\begin{aligned} R_t(t) &= \underbrace{[-\Delta L(t)]}_{R_t^Q} + V \cdot \underbrace{\left[-E\{W^e(t) + W^c(t) | \mathbb{Q}(t)\}\right]}_{R_t^C} \\ &= R_t^Q + VR_t^C \end{aligned} \quad (4-22)$$

其中 Lyapunov 漂移项 $\Delta L(t)$ 如下所示:

$$\Delta L(t) = \frac{1}{2} \sum_{k=1}^K (Q_k^2(t+1) - Q_k^2(t)) \quad (4-23)$$

该项能够保证系统的稳定性。

其中 $\Delta L(t)$ 记录了时隙之间的系统队列长度变化, 而为惩罚项 $W^e(t) + W^c(t)$ 作为扰动系统的因子。通过 DRL 框架最大化累积预期系统奖励 $\sum_{t \in \mathbb{T}} R_t$ (而非瞬时贪婪), 进而推导适合本文问题的奖励函数, 解决以往 Lyapunov 优化过程中的每一时隙的局部最优问题。所以我们将基于 DRL 对 R_t 进行系统稳定性分析和惩罚最小化的奖励目标函数的进一步具体设计, 以便最大化系统奖励和保证系统稳定性。

4.4.3 系统的稳定性

根据先前设计的系统目标奖励函数, 我们获得目标函数中的系统强稳定性, 并因此获得定理 1。

定理 1: 队列长度的上限: :

$$\frac{1}{t} \sum_{\tau=0}^t \sum_{k=1}^K E\{Q_k(\tau)\} \leq \frac{1}{\varepsilon} (U - R_l) \quad (4-24)$$

其中 U 为有限常数 (累积的奖励) 和有限正实数 $\varepsilon \rightarrow 0$, R_l 为奖励有限常数下界。

证明:

首先, 假设系统在时刻 $t = 0$, 队列积压 $Q_k(t) = 0$ 。同时, 根据研究工作[103], RL 在时刻 t 的奖励 R_t 满足以下不等式:

$$R_t \leq U - \varepsilon \sum_{k=1}^K Q_k(t+1) \quad (4-25)$$

由于累积的奖励存在下限, 因此

$$R_t \leq R_l, \forall t \in \mathbb{T} \quad (4-26)$$

因此以上的 R_t 进行 RL 训练的队列是强稳定的。对不等式 (4-25) 两边取期望:

$$E\{R_t\} \leq U - \varepsilon \sum_{k=1}^K E\{Q_k(t+1)\} \quad (4-27)$$

对 $\tau = 0, 1, \dots, t-1$ 求和, 我们有

$$\sum_{\tau=0}^{t-1} E\{R_\tau\} \leq Ut - \varepsilon t \sum_{\tau=0}^{t-1} \sum_{k=1}^K E\{Q_k(\tau+1)\}. \quad (4-28)$$

重新排列不等式 (4-28), 除以 εt , 我们有

$$\frac{1}{t+1} \cdot \frac{t+1}{t} \sum_{\tau=1}^t \sum_{k=1}^K E\{Q_k(\tau)\} \leq \frac{U}{\varepsilon} - \frac{1}{\varepsilon} \cdot \frac{1}{t} \sum_{\tau=0}^{t-1} E\{R_\tau\} \quad (4-29)$$

其中当时间 t 逐渐增大, $\frac{t+1}{t} \rightarrow 1$, 我们可以看出 $Q_k(0) = 0$ 。

从 (4-29) 式可以看出, 式子左侧为队列的长期平均值。RL 是最大化累计奖励 $\sum_{\tau=0}^{t-1} E\{R_\tau\}$, 可以通过式子左侧减小平均队列积压量来获得 RL 的目标, 因此在时刻 t , 累积奖励 $\sum_{\tau=0}^{t-1} E\{R_\tau\}$ 越大系统队列长度越小, 同时代表系统越稳定。根据不等式

(4-26), 那么不等式 (4-29) 右边的第二项为 $-\frac{1}{t} \sum_{\tau=0}^{t-1} E\{R_\tau\} \leq -R_l$, 所以不等式 (4-29) 进一步推导如下:

$$\frac{1}{t+1} \cdot \frac{t+1}{t} \sum_{\tau=1}^t \sum_{k=1}^K E\{Q_k(\tau)\} \leq \frac{U}{\varepsilon} - \frac{R_l}{\varepsilon} \quad (4-30)$$

从不等式 (4-30) 中可以看出, 所有系统队列平均长度是有界的, 因此队列长度

也有界, 定理 1 被证明。

4.4.4 惩罚项最小化的奖励

根据定理 1 以及公式 (2-20), 我们在进行系统惩罚项奖励函数构造过程中获得奖励上界以及队列的具体折现形式。

(1) 系统奖励函数上界

定理 2: 系统奖励函数 R_t 的上限 R_t^u 如下所示:

$$R_t^u \stackrel{(a)}{\leq} -\omega \sum_{k=1}^K [Q_k(t+1)]^v \quad (4-31)$$

$$R_t^u \stackrel{(a)}{\leq} -\omega \sum_{k=1}^K Q_k(t+1) + \omega K \quad (4-32)$$

其中 ω 为有限正常数, v 为队列长度的次方数。

证明:

根据公式 (4-23), 首先构造 R_t^Q 队列稳定部分奖励函数如下所示:

$$R_t^Q = -\omega \sum_{k=1}^K [Q_k(t+1)]^v, v \geq 1 \quad (4-33)$$

因此在时刻 t 总目标奖励函数如下所示

$$\begin{aligned} R_t &= R_t^Q + VR_t^C \\ &= -\omega \sum_{k=1}^K [Q_k(t+1)]^v - V \cdot E\{W(t) | Q(t)\} \end{aligned} \quad (4-34)$$

由等式 (4-34) 可知, 奖励函数的队列稳定性部分也具有强稳定性。

从上文可知, 第 j 个边缘节点 CPU 内核时钟周期频率 $f_{j,k}^e$ 以及从第 i 个边缘节点卸载到云节点的数据量 $\sum_{k=1}^K D^k(t)$ 由公式 (4-7) 可知。同时 $W^e(t) = \sum_{k=1}^K W_k^e(t)$, $W_{j,k}^e(t) = \kappa (f_{j,k}^e)^3$ 和 $W^c(t) = \kappa \sum_{k=1}^K (w_k D^k(t))^3$ 。本文设置 $W^e(t) \geq 0$ 和 $W^c(t) \geq 0$ 。我们还有

$$f_{i,k}^e \leq f^e, D^k(t) \leq B, \forall k \in \mathbb{K}, i \in \quad (4-35)$$

由于计算与通信的资源完整性, 所以我们有

$$W^e(t) \leq \sum_{k=1}^K \kappa (a_k \cdot f^e)^3, W^c(t) \leq \kappa \cdot B^3 \sum_{k=1}^K (w_k)^3 \quad (4-36)$$

根据不等式 (4-36), 我们有

$$\begin{aligned} & -\sum_{k=1}^K \kappa (a_k \cdot f^e)^3 - \kappa \cdot B^3 \sum_{k=1}^K (w_k)^3 \\ & \leq -[W(t)] \leq 0 \end{aligned} \quad (4-37)$$

因此将不等式 (4-37) 带入 (4-34)，我们有两种上限情况，即定理 2(a) 和定理 2(b) 两种情况。由不等式 (4-37)： $-[W(t)] \leq 0$ 可知，为定理 2(a) 情况。当 $v \geq 1$ 时， $x^v \geq x - 1$ ，为定理 2(b) 情况。因此证明了定理 2。同时设置 $U = \omega K$ 和 $\varepsilon = \omega$ 同时也验证了定理 1。

接下来我们进行上限分析：奖励上限与状态之间的关系。

在定理 2 的证明过程中，我们可以观察到以下情况：当 V 的值较小且接近 0 时， $-V \cdot W(t) \approx 0$ ，奖励函数中与系统稳定性相关的部分占据主导地位，那么降低平均队列长度的动作会获得更大的奖励。然而，当 V 的值较大时，只有当平均队列长度减小到一定程度时，惩罚部分 $W(t)$ 变得越小，奖励才会越大，则 $R_t^c(t)$ 占据系统奖励比重更大。

由于系统趋于稳定意味着数据积压量较小，因此奖励函数倾向于促使平均队列长度减小的动作。在 V 的较大值情况下，系统获得惩罚 $W(t)$ 项值，继而通过减小 $Q_k(t+1)$ 长度来获取更大的奖励。然而，这种情况下并不能保证队列的稳定性，因为不满足不等式 (4-26)。因此，通过在奖励函数中引入 V 控制参数，可以实现系统稳定性和惩罚代价减少之间的平衡。

(2) 奖励函数中的队列的具体分解

本小节考虑队列稳定奖励 R_t^Q 中的 $Q_k(t+1)$ 具体分析。奖励函数 R_t^c 部分是动作 $\alpha^k(t)$ 和 $\beta^k(t)$ 的确定性函数，所以不需要进行再次分析。因此，通过推导，我们获得基于 DRL 的队列稳定奖励 R_t^Q 的两种具体分解式。

定理 3： 系统的队列稳定奖励 $R_t^{Q_1}$ 和 $R_t^{Q_2}$ ：

$$R_t^{Q_1} = -\omega \sum_{k=1}^K [\lambda_k - \mu_k(t)] \quad (4-38)$$

$$R_t^{Q_2} = -\omega \sum_{k=1}^K \left\{ 2Q_k(t) [\lambda_k - \mu_k(t)] + [\lambda_k - \mu_k(t)]^2 \right\} \quad (4-39)$$

其中 $\lambda_k = E\{A_k(t)\}$ 。

证明：

首先我们将前 $t-1$ 个时刻展开, 当 $Q_k(t) + A_k(t) \geq \mu_k(t)$, 因此有以下公式。

$$\begin{aligned}
 Q_k(t+1) &= \max \left\{ Q_k(t) - \underbrace{(d_e^k(t) + d_c^k(t))}_{=\mu_k(t)}, 0 \right\} + A_k(t) \\
 &= \underbrace{Q_k(0) + \sum_{\tau=0}^{t-1} [A_k(\tau) - \mu_k(\tau)]}_{Q_k(t)} + A_k(t) - \mu_k(t) \\
 &= \underbrace{Q_k(t) + A_k(t)}_{\text{state at time } t} - \underbrace{\mu_k(t)}_{\text{action at time } t}
 \end{aligned} \tag{4-40}$$

其中, 对于 $\tau = 0, 1, \dots, t$, $\mu_k(\tau)$ 是 $\alpha^k(\tau)$ 和 $\beta^k(\tau)$ 的确定性函数, 由于数据到达率 $A_k(\tau)$ 是随机的, 因此无法通过策略进行控制。根据强化学习的概念, 具有概率奖励的环境比确定性奖励环境的更难学习。也就是说, 在已有确定的状态下, 智能体执行动作接受奖励, 当接收到的奖励 (具有概率性) 与真实奖励之间存在较大方差, 那么智能体无法准确判断当前动作的好坏。因此, 我们引入时序框架, 将 $Q_k(t) + A_k(t)$ 定义为状态变量, 决定的奖励函数状态变量 $Q_k(t+1)$ 成为状态和动作的确定性函数, 如公式 (4-40) 所示, $A_k(\tau), \tau = 0, 1, \dots, t$ 表示处于随机进入队列的状态。因此, $A_k(t+1)$ 所出现的随机性就体现在状态转换中。

$$\begin{aligned}
 s_t &= \{Q_k(t) + A_k(t), \dots\} \\
 s_{t+1} &= \left\{ \underbrace{Q_k(t) + A_k(t)}_{\in s_t(t)} - \underbrace{\mu_k(t)}_{\text{action at time } t} + \underbrace{A_k(t+1)}_{\text{random at time } t}, \dots \right\}.
 \end{aligned} \tag{4-41}$$

即系统下一个状态遵循 $s_{t+1} \sim P(s_{t+1} | s_t, a_t)$, 而数据量的随机到达 $A_k(t+1)$ 影响状态转移概率 P 。因此以上转换情况属于 MDP, 并且 $A_k(t+1)$ 在时隙之间是独立到达的。因此, 系统演化过程落入到一个带有确定性奖励函数的 MDP 模块。然而如果我们将状态定义为 $Q_k(t)$, 那么是不存在一个时隙延迟步长的因果关系的。因为只考虑状态变量 $Q_k(t)$ 的话, 将产生随机性概率奖励, 这会加大系统训练策略的难度。

接下来, 为了使我们的系统达到强稳定的状态, 我们进行更加贴合基于 DRL 的奖励函数设计 (添加折扣因子)。

首先通过不等式 (4-29) 来维持最大预期奖励和所需的队列长度控制。由于系统训练是基于回合的, 因此我们将一个回合的时间长度设为 T 。由上文可知 $Q_k(0) = 0$, 一个回合的累积奖励表达式为:

$$\frac{1}{T} \sum_{t=0}^{T-1} R_t^Q = -\omega \frac{1}{T} \sum_{t=0}^{T-1} \sum_{k=1}^K [Q_k(t+1)]^\nu \quad (4-42)$$

关于 (4-37) 式的推导, 是因为当 $t = \zeta - 1$, 对于 ζ 来说 $[Q_k(\zeta)]^\nu$ 的系数如下所示:

$$\frac{T - (\zeta - 1)}{T} - \frac{T - \zeta}{T} = \frac{1}{T} \quad (4-43)$$

将等式 (4-43) 带入等式 (4-24), 我们有

$$\frac{1}{T} \sum_{t=0}^{T-1} R_t^Q \stackrel{(a)}{=} -\omega \sum_{t=0}^{T-1} \sum_{k=1}^K \frac{T-t}{T} [Q_k(t+1)^\nu - Q_k(t)^\nu]. \quad (4-44)$$

因此, 我们接着定义队列稳定部分的奖励平均值 $\bar{R}_t^Q = \frac{1}{T} \sum_{t=0}^{T-1} R_t^Q$, 如下所示:

$$\bar{R}_t^Q = -\omega \sum_{k=1}^K \frac{T-t}{T} [Q_k(t+1)^\nu - Q_k(t)^\nu], \nu \geq 1 \quad (4-45)$$

根据等式 (4-42) 和 (4-44), 除因子 $1/T$ 外, 等式 (4-33) 中的 R_t^Q 与重新构造后的队列奖励 $\bar{R}_t^Q$ 等价。根据等式 (4-34), R_t^Q 满足定理 2 中的

$R_t^Q \leq U - \varepsilon \sum_{k=1}^K Q_k(t+1)$, 因此 $\bar{R}_t^Q$ 满足 $\sum_{\tau=0}^{t-1} \bar{R}_\tau^Q = \frac{1}{t} \sum_{\tau=0}^{t-1} R_\tau^Q$, 对时间 $\tau = 0, 1, \dots, t-1$ 求

和, 然后根据等式 (4-45), 我们有

$$\sum_{\tau=0}^{t-1} \bar{R}_\tau^Q = \frac{1}{t} \sum_{\tau=0}^{t-1} R_\tau^Q \leq U - \varepsilon \frac{1}{t} \sum_{\tau=0}^{t-1} \sum_{k=1}^K Q_k(t+1) \quad (4-46)$$

重新排列 (4-46), 取期望得

$$\frac{t+1}{t} \cdot \frac{1}{t+1} \sum_{\tau=0}^{t-1} \sum_{k=1}^K E\{Q_k(\tau)\} \leq \frac{U}{\varepsilon} - \frac{1}{\varepsilon} \sum_{\tau=0}^{t-1} E\{\bar{R}_\tau^Q\} \quad (4-47)$$

所以重新构造后的奖励函数 $\bar{R}_t^Q$ 也可以对队列长度进行最优控制。与不等式 (4-30) 相比, (4-47) 不等式右侧第二项前的因子 $1/t$ 消失, 这是因为重塑的奖励函数在时刻 t 通过 $(T-t)/T$ 因子进行折扣产生回报, 折扣的值是随着 t 增加而减少, 并且随着时间增加从 1 变为 0。因此更加适合基于 DRL 的实际系统稳定部分奖励。因此:

$$\bar{R}_t^Q = -\omega \sum_{k=1}^K \frac{T-t}{T} \left[\left(\underbrace{Q_k(t) + A_k(t)}_{\text{state at time } t} - \underbrace{\mu_k(t)}_{\text{action at time } t} \right)^\nu - [Q_k(t)]^\nu \right]. \quad (4-48)$$

随着时间的推移, 实际的 DRL 为了保证贝尔曼方程的收敛, 采取折扣因子 $\gamma < 1$ 而

非每一时刻奖励函数的求和，以便实现最小化指数折扣奖励的总和 $\sum_t \gamma^t R_t$ 。在强化学习概念中，估计状态动作值函数的是贝尔曼算子，当奖励按照折扣因子 $\gamma < 1$ 呈指数变化，因此随着时间推移 γ^t 所占更小比例并且靠近 0，会发生收敛映射。然后，状态动作值函数收敛到某个固定值并且完成了适合环境的学习。所以我们引入折扣因子进行系统奖励的再设计。由上文可知构造的奖励 $\bar{R}_t^Q$ 具有内部折扣因子 $(T-t)/T$ ，同时也随着时间的推移从 1 减少到 0。本文创建的折扣因子与以往的折扣因子不同，但对奖励的影响是相同的。因此，我们针对队列稳定性和惩罚项最小化，对奖励 R^t 进行重新设计，即 $\hat{R}_t$ ：

$$R_t = -\omega \sum_{k=1}^K [Q_k(t+1)^\nu - Q_k(t)^\nu] - V[W(t)], \nu \geq 1 \quad (4-49)$$

然后，奖励函数为外部折扣因子为 $\gamma (0 < \gamma < 1)$ ，并且系统队列稳定奖励部分设计为 $-\omega \sum_{k=1}^K \gamma^t [Q_k(t+1)^\nu - Q_k(t)^\nu]$ ，在每个回合开始时候，外部的折扣因子在系统运行一开始给予队列的权重过大，会引起队列在系统性能评估中的占比过大，从而影响系统策略的训练。所以根据公式 (4-42) 和 (4-44)，目标通过时序差分近似实际奖励，并且引入内部折扣因子 $(T-t)/T$ 来提高系统奖励达到收敛的速度。另外间隙间的延迟差（一步时差）以 $Q_k(t+1)^\nu - Q_k(t)^\nu$ 为标准。

注意：我们将不包含内部折扣因子和符合时序差分情况下的队列稳定部分奖励从公式 (4-49) 提取并设置为 R_t^Q ，则系统累积队列部分的最大奖励为 $\sum_{t=0}^{T-1} R_t^Q$ ，因此有

$$\begin{aligned} \frac{1}{\omega} \sum_{t=0}^{T-1} R_t^Q &= -\sum_{t=0}^{T-1} \sum_{k=1}^K [Q_k(t+1)^\nu - Q_k(t)^\nu] \\ &= -\sum_{k=1}^K [Q_k(T)^\nu + Q_k(T-1)^\nu - Q_k(T-1)^\nu + \\ &\quad \dots + Q_k(0)^\nu] \\ &= -\sum_{k=1}^K Q_k(T)^\nu + \sum_{k=1}^K \underbrace{Q_k(0)^\nu}_0 \end{aligned} \quad (4-50)$$

根据 (4-50) 可知，所推导的结果只包含队列最后一刻的长度，体现不出来系统队列的平均长度，因此反应不了系统稳定性。所以加上内部折扣因子对系统策略的训练是很有必要的。

综上所述，我们引入了内部折扣因子对队列稳定性奖励部分的进一步推导对系统

策略训练是有效的，并且也获得以下两个具体分解式。

当 $\nu=1$ 时，根据系统操作时序图，时隙之间的队列稳定部分奖励 $R_t^{Q_1}$ 将简化为 $R_t^{Q_1}$ ，即 $R_t^{Q_1} = -\omega \sum_{k=1}^K [A_k(t) - \mu_k(t)]$ 。当 $\nu=2$ 时，我们的奖励(4-49)的队列稳定部分 $R_t^{Q_2}$ 为二次函数 $R_t^{Q_2}$ ，因此减少到了 Lyapunov 优化框架中的二次漂移项 $-\Delta L(t)$ ，正如(4-23)所示。为了进一步稳定奖励，我们可以通过用其平均值 λ_k 替换随机到达 $A_k(t)$ 因此定理 3 中的两种情况被证明。

4.4.5 实际预期总奖励

预期总奖励包含系统稳定和惩罚项最小化两部分。根据定理 3，确定我们的队列稳定性奖励。因此，加上惩罚项最小化奖励，得到总体预期奖励函数，如下式所示：

$$R_t = \hat{R}_t^Q - V \cdot W(t) \quad (4-51)$$

接下来，我们将对边缘云系统数据卸载流程框架以及采用的 SAC 算法进行阐述。

4.4.6 系统算法实现

本文采用 Soft Actor-Critic(SAC)算法进行本文设计奖励函数的验证。首先，SAC 是一种采用离线策略优化训练的最新算法^{[86],[103]}，通过最大化折扣预期奖励和政策上来改善系统的最优控制。所以，算法目标函数由以下式给出：

$$G(\pi) = \mathbb{E}_{\phi \sim \pi} \left[\sum_{t=0}^{T-1} \gamma^t \left(R_t + \phi \mathcal{H}(\pi(\cdot|s_t)) \right) \right], \quad (4-52)$$

其中 π 是策略， $\phi = \{s_0, a_0, s_1, a_1, \dots\}$ 是状态-动作轨迹， γ 是奖励外部折扣因子， ϕ 是熵权重因子， $\mathcal{H}(\pi)$ 是策略熵， R_t 是奖励，图 3 是我们的算法框架图。

对于 SAC 的实现，我们根据研究工作[103]进行设计，同时为了满足本文所设置的资源分配约束(4-4)，添加了虚拟变量 $\alpha^{K+1}(t) (\geq 0)$ 和 $\beta^{K+1}(t) (\geq 0)$ 来实现维度为 $2K+2$ 的策略深度神经网络，并在输出策略网络时使用 softmax 函数，同时满足：

$$\sum_{k=1}^{K+1} \alpha^k(t) = 1, \sum_{k=1}^{K+1} \beta^k(t) = 1 \quad (4-53)$$

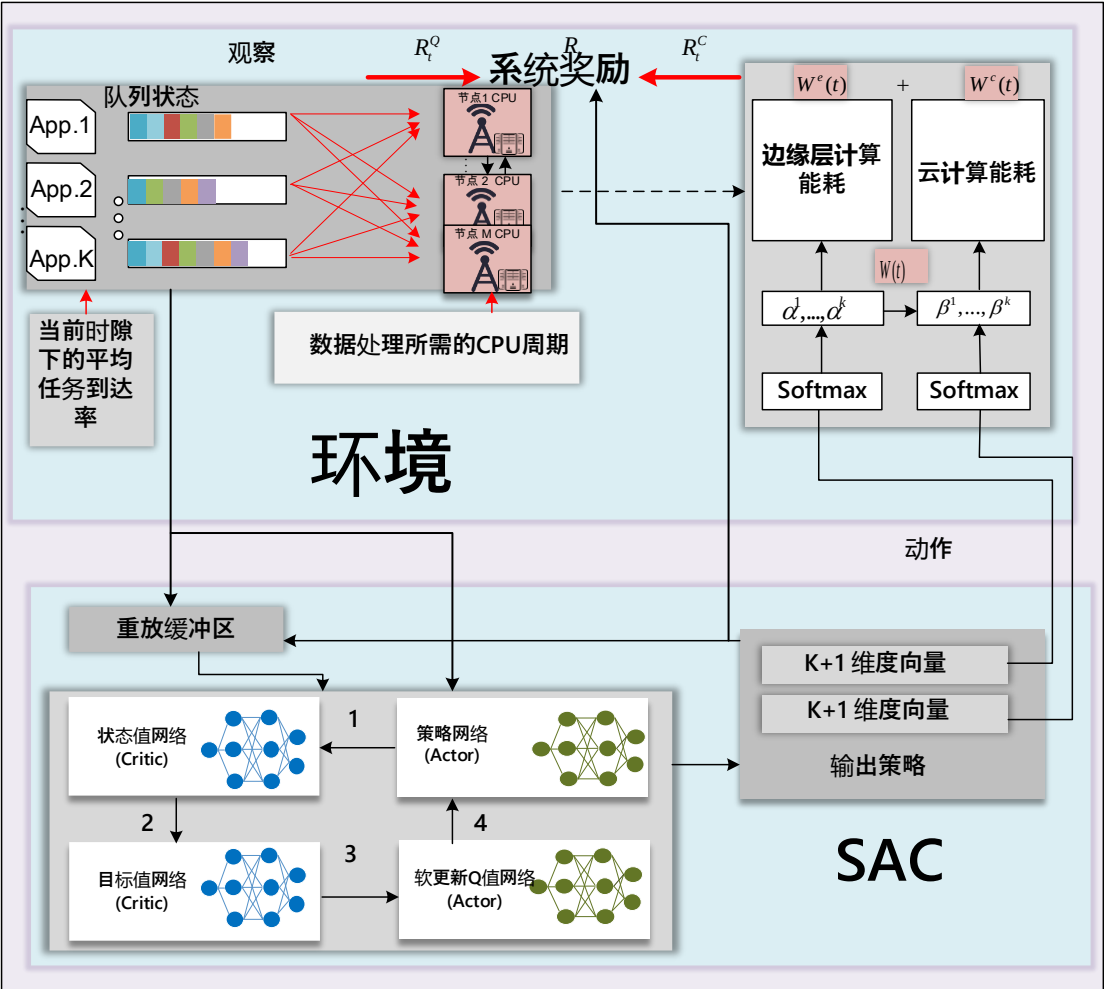


图 4-4 算法框架图

然后，取神经网络输出层 $\alpha^k(t)$ 和 $\beta^k(t)$ ，除了折扣因子之外，使用的其他超参数与研究工作[103]相同。

接下来，我们介绍系统在线计算卸载算法 4-1。

本文自适应神经网络的在线卸载算法中如上表所示。其后系统训练的回合数为 N^{step} ，在每一回合并开始的时候，遍历每一个时隙与每一种 App 应用程序。通过我们前边对系统模型架构状态与动作空间的设计，当 App 生成数据的时候， s_1 为初始状态，APP 数据传输到边缘层节点或云节点进行处理，此时，App 通过接收到边缘节点服务请求 $(\alpha^k(t), \beta^k(t))$ 的神经网络超参数，继而选择 a_t ，然后获取 SAC 智能体中 Actor 策略网络和 Critic 状态值网络的函数值。在下一个时隙 $t+1$ 中，智能体观察 s_{t+1} 。并根据 Critic 值函数的神经网络参数储存经验，最后更新 Actor 与 Critic 神经网络的参数值。

算法 4-1 在线计算卸载算法

```

1: 输入: 状态和动作:  $\mathcal{S}$ ,  $\mathcal{A}$ ;
2: 输出:  $\alpha^k(t)$ ,  $\beta^k(t)$ ;
4: for episode 1 to  $N^{step}$  do
3:   initial:  $s_t = s_1$ 
5:   for  $t \in \mathbb{T}$ 
6:     for  $k = \{1, 2, \dots, K\}$ 
7:       if APP  $k$  生成数据  $A_k(t)$ 
8:         数据传输到边缘层多节点
           接收深度神经网络参数后, 确定 Critic 值函数和 Actor 策略函数的值
           选择动作  $a_t$ 
9:       end if
10:      根据  $s_t$  and  $a_t$  观测  $W(t)$ ;
11:      观测下一状态  $s_{t+1}$ ;
12:      更新神经网络参数, 并储存经验;
13:    end for
14:  end for
15: end for

```

4.5 仿真结果

为了验证本文提出的策略, 系统将应用程序服务卸载到边缘节点与云节点。每种应用程序服务为三元组 $(\lambda_k, d_k, \omega_k)$, 分别代表任务的到达率(泊松分布)、每个任务的数据大小, 一个任务需要的 CPU 时钟周期。文中涉及 8 种应用程序服务, 参数如表一所示。现实世界许多场景都是泊松分布的, 因此任务到达率分布近似连续时间中离散事件频率的概率设置, 具有一定的实用性与广泛性。场景中边缘层有 7 个边缘节点, 一个云节点, 其中云节点处理能力比边缘节点大。每个边缘节点为单核 CPU, CPU 具有每秒 10Gcycles/s 的处理能力。而云节点的计算能力我们设置为每秒 100Gcycles/s。此外参数 $\omega = 10^{-9}$, $\kappa = 1/(100\text{GHz})^3$, $B = 10\text{Mbps}$, 事实上, 这些参数并不影响本文系统性能的优化, 因为本文将队列与惩罚项之间设置了控制参数 V 。同时本文的 如表 4-2, 表 4-3 所示。

我们将从系统稳定性, 参数 V , 更高维度的系统操作以及比较实验进行评估本文的策略。

表 4-2 App 参数

APP 序号	λ_k (任务量 / s)	d_k (byte)	ω_k
1	0.5	20	12
2	1	45	4
3	2	70	7
4	5	35	23
5	10	66	16
6	20	38	17
7	30	90	37
8	40	12	11

表 4-3 SAC 算法参数

参数	值
优化器	Adam
学习速率	10^{-4}
外部折扣因子 γ	0.99
隐藏层数量	2
每个隐藏层的神经元数量	128
每批的样本采样数量	128
重放缓冲区大小	2×10^5
非线性激活函数	Relu
目标更新频率	2
梯度	1

4.5.1 系统稳定性

本小节将从设计的两个奖励函数中的系统稳定部分（平均队列长度）进行分析。首先图 4-5 为第一种奖励函数中的系统队列积压，图 4-6 为第二种奖励函数中的系统队列积压。两种情况均训练了 100 个回合，每个回合为 $T=1000$ 个时间步长，所以为 100000 个采样样本，我们将样本放在采样缓冲区作为经验，进行神经网络 SAC 训练，在训练开始时，队列长度都重置为 0。然后重复以上过程，进行下一个 V 值下的系统策略的训练。根据本文训练的策略，我们随机生成 1 个回合的队列来进行稳定性评估。从两个图中可以观察到本文策略的两个系统稳定部分，在训练过程初期，很快达到了稳定，且随着 V 值的增大，我们系统中的队列长度依然处于其所在阈值范围内，这是

因为通过本文策略的系统训练，即使带有 V 值的惩罚部分在一步步增加奖励函数的占比，但是依然不影响本文系统的稳定，因此验证了定理 1：队列的强稳定性，也表明本文策略的有效性。

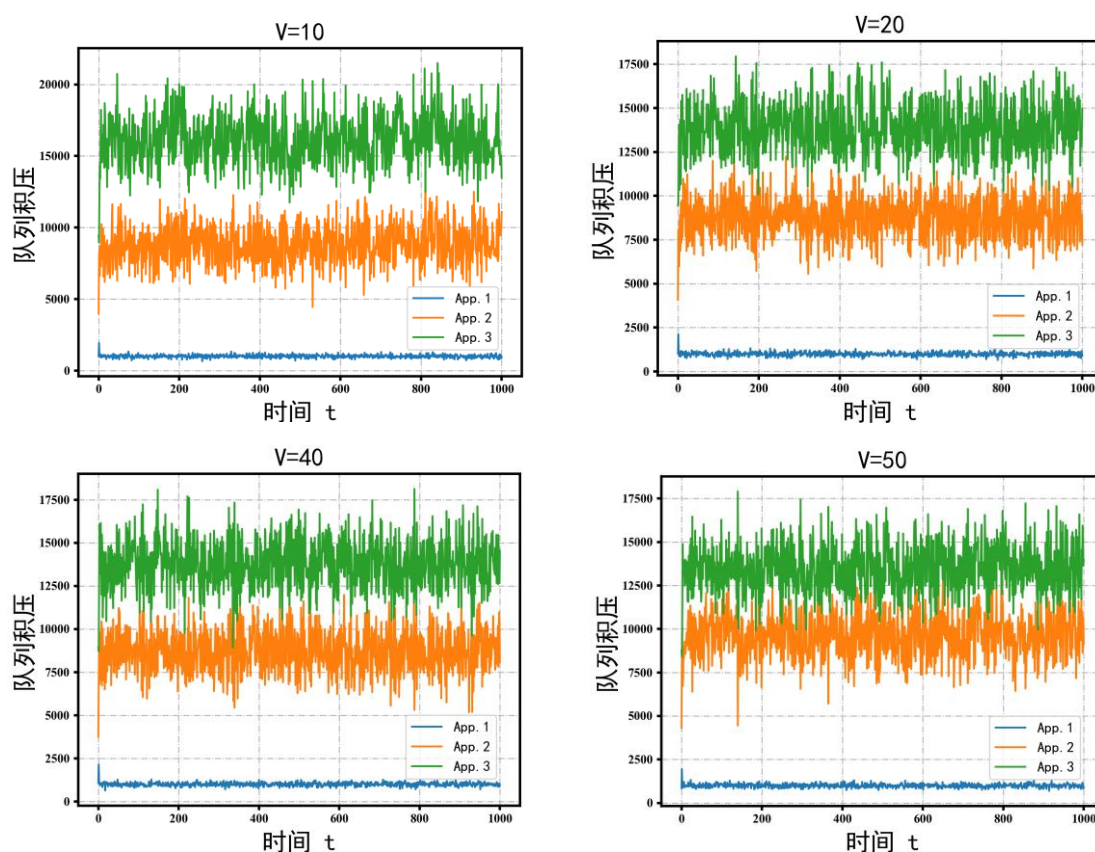
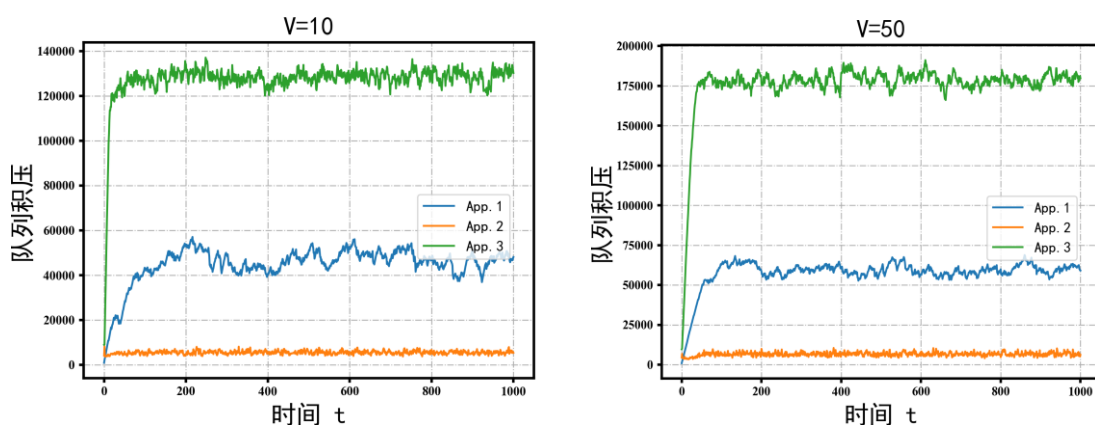


图 4-5 第一种队列积压



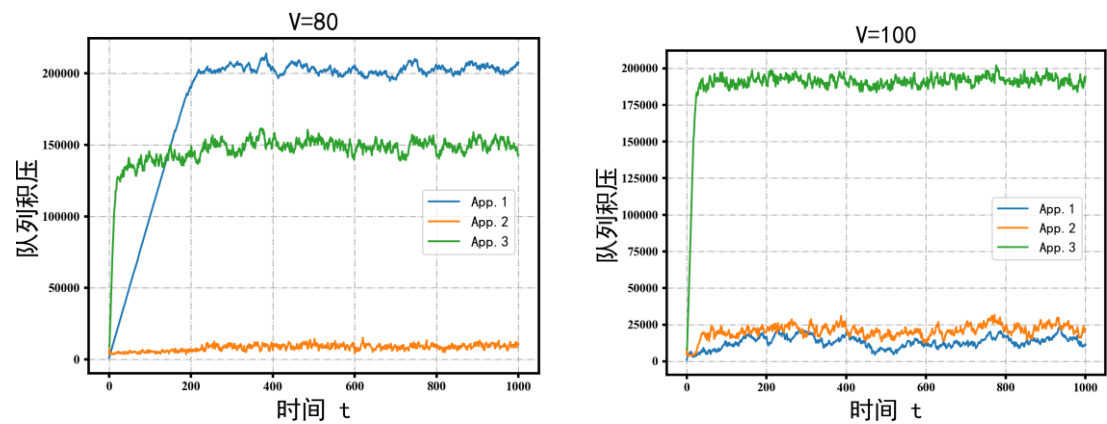


图 4-6 第二种队列积压

4.5.2 系统参数 V 影响

图 4-7，图 4-8 为 100000 个时间步长下不同 V 值的两种奖励函数。两种情况下的 R_t 均为训练了 100 个回合，从图 4-7，图 4-8 中可以看出随着训练轮数的增加，奖励函数数值最终趋于稳定的阈值，由于本文的奖励函数包含两部分，其中奖励函数中的惩罚部分两者是一致的，因此两个奖励函数在同 V 值下的差值即为公式之中的差量。图 4-7 随着时间步长增加，奖励函数先变小后来增大并最终收敛。这是由于系统初期训练，并未从缓冲区获得较好的经验，因此奖励函数值变小，随着训练次数增多，系统达到了自适应神经网络的状态，因此奖励函数值达到阈值。图 4-8 为带有二次项奖励函数的结果，从图中可以看出奖励函数值先增大后降低再增加，这说明该奖励函数的设计在系统开始训练时，达到了很好的效果。然而由于该函数相对于第一种情况下的奖励函数更为复杂，所以初期训练过程中变化幅度较大，且在达到收敛时所需时间更多，第一种奖励函数相对第二种奖励函数达到上限的速度提高了%5。两种情况下的奖励函数最终都达到了阈值范围内，因此验证了定理 2。

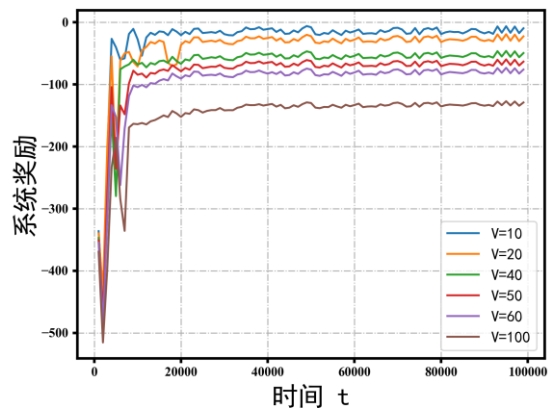


图 4-7 第一种系统奖励

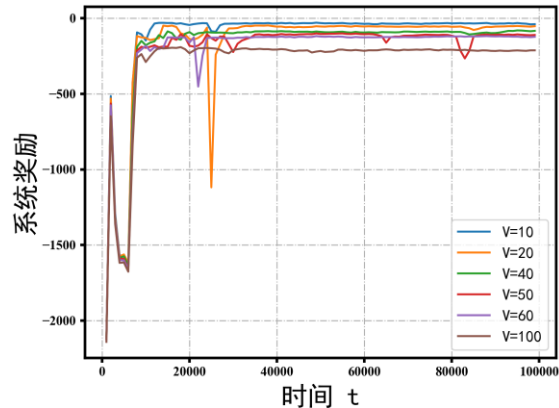


图 4-8 第二种系统奖励

图 4-9 为 100000 个时间步长下不同 V 值的平均奖励函数。由于图 4-6 中的云节点计算过程是将边缘节点上传的数据量在 CPU 每个内核进行均分卸载，因此我们需要考虑不均分的情况下本文策略的有效性。图 4-9 与图 4-7 不同的是我们将奖励函数中的惩罚部分的云节点计算卸载方式改为当云节点单个 CPU 内核计算资源完成分配后，才能进行下一个 CPU 内核对数据的计算卸载，因此为 $W^c(t)$ 为不连续函数，而边缘服务器的计算方式保持不变。如图 4-9 所示，本文提出来的策略在应对不连续函数的情况下依然满足定理 2，并且可以快速达到最大奖励的阈值。因此本文的策略在更复杂的函数情况下也是有效的。同时，图 4-10，图 4-11 为该情况的下的两种奖励函数的系统队列积压，从图 4-10、图 4-11 中可以看出本文策略依然可以使系统快速达到稳定。图 4-12 为训练策略形成后的 1 个回合下的不同系统队列平均积压的变化下的系统能耗。从图中可以看出，即使系统队列积压变大，我们的系统能耗也逐渐减低，这说明了本文所提出的自适应神经网络策略在保证队列稳定的同时也降低了系统能耗。

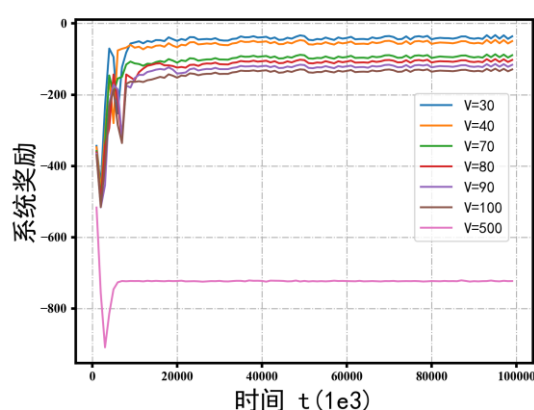


图 4-9 不同 V 值下的系统奖励

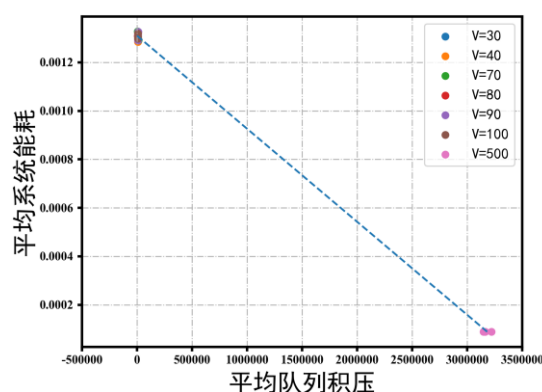
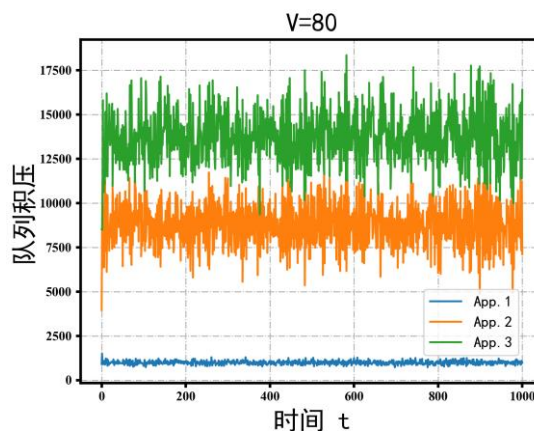
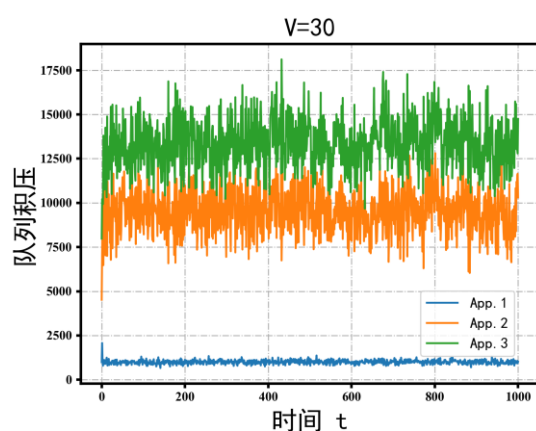


图 4-12 长期下的平均队列积压 VS 平均系统能耗



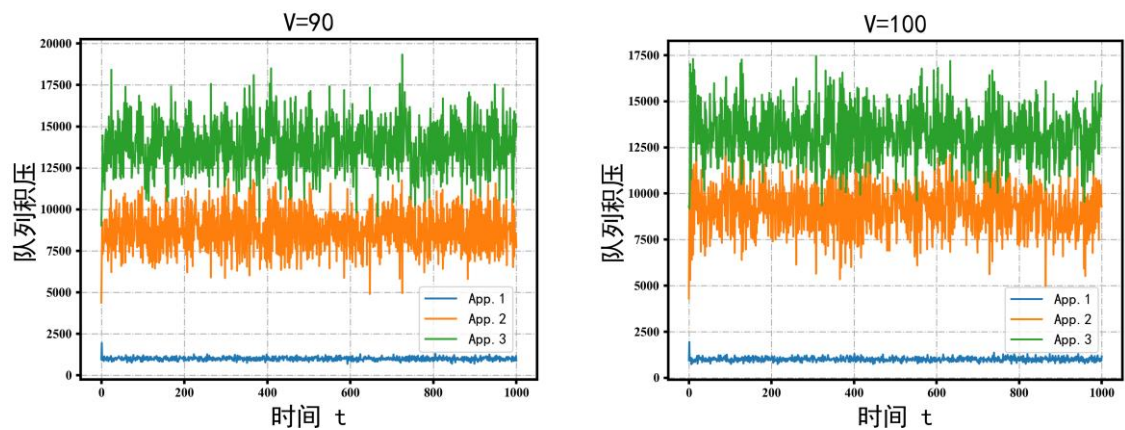


图 4-10 第一种队列积压 $V=30,80,80,100$

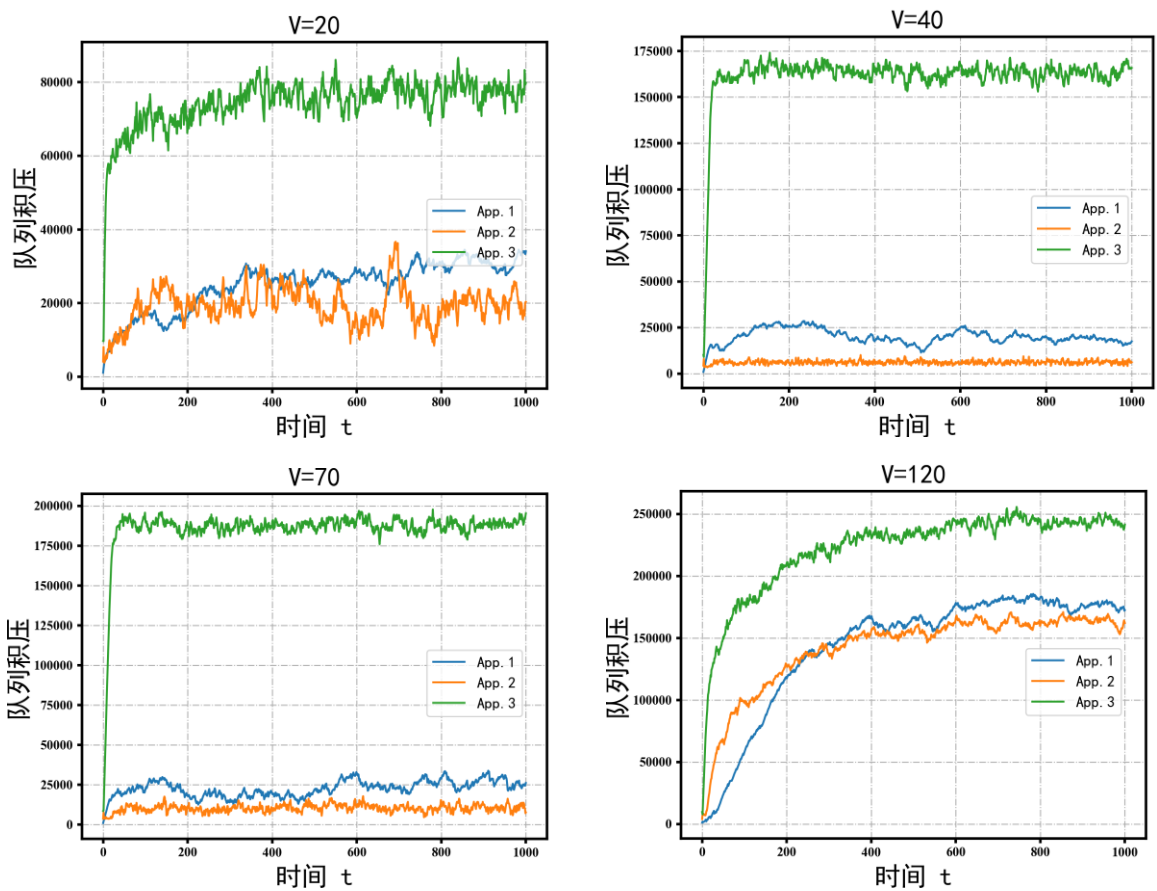


图 4-11 第二种队列积压 $V=30,80,80,100$

4.5.3 系统高动作维度操作

我们增加队列数量，从 3 个到 8 个队列，因此增加了动作的搜索维度，动作维度为 $2N+2=18$ ，相对之前实验增加了 10 个维度。关于 8 个应用程序服务的具体参数如表 3 所示。图 4-13 为系统自适应神经网络训练 100 轮后的 1 个回合，从图中可以看出，系统队列在前 50 个时隙就达到了稳定的状态。图 4-14 为系统训练了 200 个回合的结果，从图中可以看出来系统长期的奖励函数在训练初期很快就达到了其阈值范围之

内。因此证明了本文策略在动态环境下的自适应性较强。

4.5.4 比较实验

图 4-15 为我们所提方案的系统性能与 LYDROOY^[79]、AAC^[88]的比较。本文系统策略训练完成后，通过扫描为每个 V 控制参数训练的 300 个回合，计算出平均一个回合的惩罚部分和队列部分的数值，通过每种卸载方案的算法连线，并将其绘制在图 13。从图中可以看出来，我们的方案相对另外两种方案能耗降低的速度更快，这说明了本文系统训练的策略可以在保证系统稳定的同时较快地进行自适应动态环境变化的节能控制。

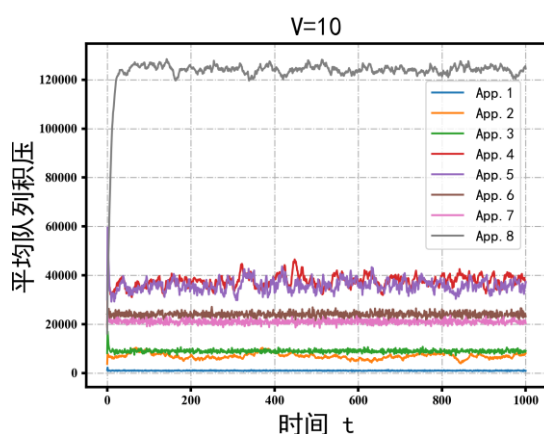


图 4-13 8 个队列的平均队列积压

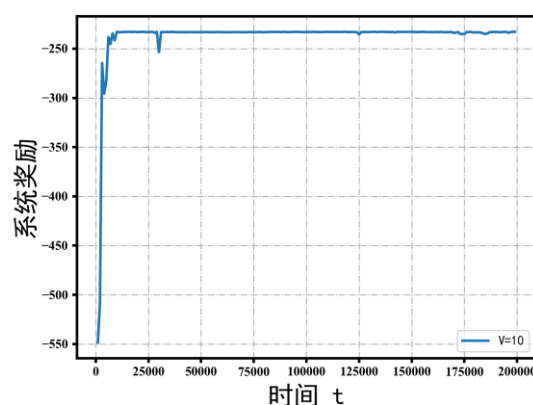


图 4-14 长期的系统奖励

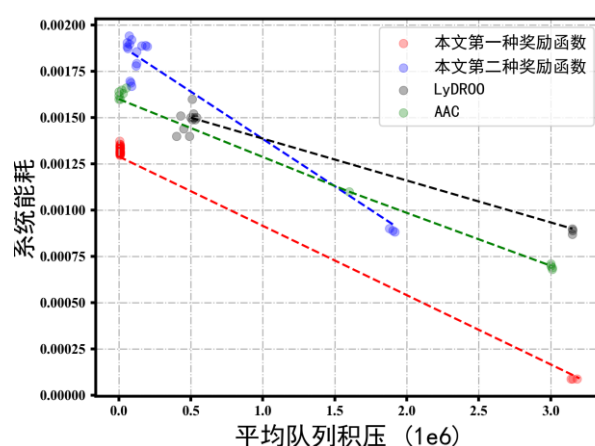


图 4-15 平均队列积压 VS 系统能耗

4.6 本章小结

本文研究了云边协同服务在线卸载节能问题。基于 DRL 的方法，我们提出了一种解决传统 Lyapunov 优化服务卸载过程中实时贪心求局部最优解的策略。本文方案可以在保持队列稳定性的同时最大限度地减少系统能耗。系统构建适合服务队列的状态和

动作空间以及奖励函数，同时经过自适应神经网络 SAC 算法的训练，获得经验，形成了长期适应动态环境变化的策略。

第五章 总结与展望

5.1 总结

本文以边缘物联网为背景，对其数据协作卸载机制进行研究。随着 6G 技术的普及和移动设备的激增，越来越多的计算密集型应用程序涌现出来。然而，这些应用程序需要实时在线计算，而移动设备的计算能力有限。为了解决这一问题，移动边缘计算（MEC）应运而生。MEC 通过与 6G 基础设施的结合，将计算任务从移动设备转移到边缘服务器上进行处理。这种分布式计算模型极大地提高了应用程序的性能和效率，并降低了延迟。MEC 为移动设备用户提供了更快速、实时的应用程序服务，同时减轻了移动设备的负担，为移动通信领域带来了巨大的创新和进步。

目前关于基于 Lyapunov 优化的数据协作卸载研究工作中，需要对单小区、多边缘服务器之间动态流量控制以及云边三种场景下的在线数据协作卸载模型与方案的再设计，本文针对以上三种场景进行数据协作卸载机制研究。首先，我们研究了单小区的设备数据协作卸载方案。数据通过上行链路进行传输，由于服务器资源有限，因此针对关联时延敏感的计算密集型任务的用户，设备数据被处理的顺序显得非常重要，所以我们提出一种联合设备优先级和在线部分卸载算法（PODP）。第二章协作数据处理方式与以往的大多数研究存在一个相同点，即都是多设备-单服务器通信场景。因此第三章我们研究了多设备-多边缘服务器协作通信场景中服务实时卸载系统成本最小化问题。通过 Lyapunov 优化框架求解，提出了一种在线动态服务卸载算法，合理制定卸载决策，确定服务单跳卸载还是多跳协作卸载。然而针对大型应用程序的卸载，边缘层的算力也是有限的，再加上复杂的通信场景，因此，第四章研究了一种基于 Lyapunov 优化框架下的云边协同服务智能卸载决策方式。通过将 Lyapunov 漂移加惩罚上界嵌入在 RL 框架内，以及采用 DRL 模块与未知环境实时强交互，完成动态网络的系统资源分配，并且无需未来的网络信息，就可以形成长期适应系统的策略。本文三个研究内容之间相互独立，且层层递进，越来越贴合现实复杂的通信场景的物理实体模型与系统数据协作处理。

5.2 展望

本文在单边缘服务器，多边缘服务器，云边服务器联合的三种不同通信场景下提出的数据协作卸载方案都显示出不错的性能，为了进一步深入研究，在未来还应该考虑以下问题。

(1) 第二章中我们考虑的是单服务器根据设备优先级，从高到低进行数据处理。然而，并未保护数据隐私和安全。我们可以联合 FL（联邦学习），通过移动设备和基站（BS）之间交换模型参数进行通信。首先进行本地模型训练，上传到服务器设备参数，进而通过下行链路发送全局模型参数进行模型部署与在线推理，而非大量数据，形成一个具有隐私保护且稳定性的单小区在线 FL 系统。

(2) 第三章中为分布式蜂窝下的多边缘服务器协作数据卸载。然而，数据在通过上行链路上传过程中，由于瑞利衰落信道的存在，导致数据传输质量下降，我们可以联合目前的 RIS（可重配智能表面）通信技术，将其作为媒介，重塑无线信道，再将反射信号传输到多边缘服务器进行优化处理。因此，通信系统成本模型与理论需要进一步的再设计。

(3) 第四章提出了一种新颖的云边协同资源管理框架。然而并未应用到目前的主流容器资源框架进行推理。目前针对云边协同资源管理的强大工具是 kubernetes，它的核心是一个服务器集群，能够在集群的每个节点上运行指定的程序，以管理节点中的容器。我们可以通过 Lyapunov 函数创建双队列模型，联合容器与服务，使其终端应用层与边缘集群数据得到稳定处理，因此其资源分配架构及模型与算法需要进一步再设计。

参考文献

- [1] Mao Y, You C, Zhang J, et al. A survey on mobile edge computing: The communication perspective[J]. IEEE communications surveys & tutorials, 2017, 19(4): 2322-2358.
- [2] Duan, H., et al., Treasure collection on foggy islands: Building secure network archives for Internet of Things. IEEE Internet of Things Journal, 2018. 6(2): 2637-2650.
- [3] Mohammad U, Sorour S. Adaptive task allocation for mobile edge learning[C]//2019 IEEE Wireless Communications and Networking Conference Workshop (WCNCW). IEEE, 2019: 1-6.
- [4] Chiang M, Zhang T. Fog and IoT: An overview of research opportunities[J]. IEEE Internet of things journal, 2016, 3(6): 854-864.
- [5] Yaqub U, Sorour S. Multi-objective resource optimization for hierarchical mobile edge computing[C]//2018 IEEE Global Communications Conference (GLOBECOM). IEEE, 2018: 1-6.
- [6] Liu C F, Bennis M, Poor H V. Latency and reliability-aware task offloading and resource allocation for mobile edge computing[C]//2017 IEEE Globecom Workshops (GC Wkshps). IEEE, 2017: 1-7.
- [7] Osseiran A, Braun V, Hidekazu T, et al. The foundation of the mobile and wireless communications system for 2020 and beyond: Challenges, enablers and technology solutions[C]//2013 IEEE 77th Vehicular Technology Conference (VTC Spring). IEEE, 2013: 1-5.
- [8] Mohammad U, Sorour S, Hefaida M. Task allocation for asynchronous mobile edge learning with delay and energy constraints[J]. arXiv preprint arXiv:2012.00143, 2020.
- [9] Cebula III S L, Ahmad A, Graham J M, et al. Empirical channel model for 2.4 GHz ieee 802.11 wlan[C]//Proceedings of the International Conference on Wireless Networks (ICWN).
- [10] Committee of The World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp), 2011: 1.
- [11] Lyu X, Tian H, Sengul C, et al. Multiuser joint task offloading and resource optimization in proximate clouds[J]. IEEE Transactions on Vehicular Technology, 2016, 66(4): 3435-3447.
- [12] Zhang X, Mao Y, Zhang J, et al. Multi-objective resource allocation for mobile edge computing systems[C]//2017 IEEE 28th annual international symposium on personal, indoor, and mobile radio communications (PIMRC). IEEE, 2017: 1-5.
- [13] Qian J, Gochhayat S P, Hansen L K. Distributed active learning strategies on edge computing[C]//2019 6th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)/2019 5th IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom). IEEE, 2019: 221-226.
- [14] Guo, M., et al., Lyapunov-based Partial Computation Offloading for Multiple Mobile Devices Enabled by Harvested Energy in MEC. IEEE Internet of Things Journal, 2021.
- [15] Zhou, B., et al., An online algorithm for task offloading in heterogeneous mobile clouds. ACM Transactions on Internet Technology (TOIT), 2018. 18(2): 1-25.

- [16] Beloglazov, A., et al., A taxonomy and survey of energy-efficient data centers and cloud computing systems. *Advances in computers*, 2011. 82: 47-111.
- [17] Ashrafi, T.H., et al., Iot infrastructure: fog computing surpasses cloud computing, in *Intelligent Communication and Computational Technologies*. 2018, Springer. 43-55.
- [18] Voorsluys, W., J. Broberg, and R. Buyya, Introduction to cloud computing. *Cloud computing: Principles and paradigms*, 2011: 1-44.
- [19] Zhao, H., et al. A mobility-aware cross-edge computation offloading framework for partitionable applications. in *2019 IEEE International Conference on Web Services (ICWS)*. 2019. IEEE.
- [20] Zyane, A., M.N. Bahiri, and A. Ghammaz, IoTScal - H: hybrid monitoring solution based on cloud computing for autonomic middleware - level scalability management within IoT systems and different SLA traffic requirements. *International Journal of Communication Systems*, 2020. 33(14): 4495.
- [21] Soyata, T., et al. Cloud-vision: Real-time face recognition using a mobile-cloudlet-cloud acceleration architecture. in *2012 IEEE symposium on computers and communications (ISCC)*. 2012. IEEE.
- [22] Rahmani, A.M., et al., Exploiting smart e-Health gateways at the edge of healthcare Internet-of-Things: A fog computing approach. *Future Generation Computer Systems*, 2018. 78: p. 641-658.
- [23] Mekki K, Bajic E, Chaxel F, et al. A comparative study of LPWAN technologies for large-scale IoT deployment[J]. *ICT express*, 2019, 5(1): 1-7.
- [24] Shi W, Cao J, Zhang Q, et al. Edge computing: Vision and challenges[J]. *IEEE internet of things journal*, 2016, 3(5): 637-646.
- [25] Taleb T, Samdanis K, Mada B, et al. On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration[J]. *IEEE Communications Surveys & Tutorials*, 2017, 19(3): 1657-1681.
- [26] Bi S, Ho C K, Zhang R. Wireless powered communication: Opportunities and challenges[J]. *IEEE Communications Magazine*, 2015, 53(4): 117-125.
- [27] Bi S, Zhang Y J. Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading[J]. *IEEE Transactions on Wireless Communications*, 2018, 17(6): 4177-4190.
- [28] Ulukus S, Yener A, Erkip E, et al. Energy harvesting wireless communications: A review of recent advances[J]. *IEEE Journal on Selected Areas in Communications*, 2015, 33(3): 360-381.
- [29] Xu X, Shen B, Yin X, et al. Edge server quantification and placement for offloading social media services in industrial cognitive IoV[J]. *IEEE Transactions on Industrial Informatics*, 2020, 17(4): 2910-2918.
- [30] Sudevalayam S, Kulkarni P. Energy harvesting sensor nodes: Survey and implications[J]. *IEEE communications surveys & tutorials*, 2010, 13(3): 443-461.
- [31] Wang Y, Sheng M, Wang X, et al. Mobile-edge computing: Partial computation offloading using dynamic voltage scaling[J]. *IEEE Transactions on Communications*, 2016, 64(10): 4268-4282.

-
- [32] Dinh T Q, Tang J, La Q D, et al. Offloading in mobile edge computing: Task allocation and computational frequency scaling[J]. IEEE Transactions on Communications, 2017, 65(8): 3571-3584.
 - [33] Yu Z, Gong Y, Gong S, et al. Joint task offloading and resource allocation in UAV-enabled mobile edge computing[J]. IEEE Internet of Things Journal, 2020, 7(4): 3147-3159.
 - [34] Zhan C, Hu H, Sui X, et al. Completion time and energy optimization in the UAV-enabled mobile-edge computing system[J]. IEEE Internet of Things Journal, 2020, 7(8): 7808-7822.
 - [35] R. S. Sutton and A. G. Barto, Reinforcement Learning: An introduction. MIT Press, 1998.
 - [36] Jiang W, Han B, Habibi M A, et al. The road towards 6G: A comprehensive survey[J]. IEEE Open Journal of the Communications Society, 2021, 2: 334-366.
 - [37] Park J, Samarakoon S, Bennis M, et al. Wireless network intelligence at the edge[J]. Proceedings of the IEEE, 2019, 107(11): 2204-2239.
 - [38] Giordani M, Polese M, Mezzavilla M, et al. Toward 6G networks: Use cases and technologies[J]. IEEE Communications Magazine, 2020, 58(3): 55-61.
 - [39] Merluzzi M, Di Lorenzo P, Barbarossa S. Dynamic resource allocation for wireless edge machine learning with latency and accuracy guarantees[C]//ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2020: 9036-9040.
 - [40] Merluzzi M, Battiloro C, Di Lorenzo P, et al. Energy-efficient classification at the wireless edge with reliability guarantees[C]//2022 IEEE International Conference on Communications Workshops (ICC Workshops). IEEE, 2022: 109-114.
 - [41] Y. Sun, S. Zhou, and J. Xu, "EMM: Energy-aware mobility management for mobile edge computing in ultra dense networks," IEEE Journal on Selected Areas in Communications, 2017, 35(11), 2637-2646.
 - [42] Pezone F, Barbarossa S, Di Lorenzo P. Goal-oriented communication for edge learning based on the information bottleneck[C]//ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2022: 8832-8836.
 - [43] Shao J, Mao Y, Zhang J. Learning task-oriented communication for edge inference: An information bottleneck approach[J]. IEEE Journal on Selected Areas in Communications, 2021, 40(1): 197-211.
 - [44] Binucci F, Banelli P, Di Lorenzo P, et al. Adaptive resource optimization for edge inference with goal-oriented communications[J]. EURASIP Journal on Advances in Signal Processing, 2022, 2022(1): 123.
 - [45] Binucci F, Banelli P, Di Lorenzo P, et al. Dynamic resource allocation for multi-user goal-oriented communications at the wireless edge[C]//2022 30th European Signal Processing Conference (EUSIPCO). IEEE, 2022: 697-701.
 - [46] Liu C, Guo C, Yang Y, et al. Adaptable semantic compression and resource allocation for task-oriented communications[J]. IEEE Transactions on Cognitive Communications and Networking, 2023.
 - [47] Kang X, Song B, Guo J, et al. Task-oriented image transmission for scene classification in unmanned aerial systems[J]. IEEE Transactions on Communications, 2022, 70(8): 5181-5192.

-
- [48] Vu, T.T., et al., Optimal energy efficiency with delay constraints for multi-layer cooperative fog computing networks. *IEEE Transactions on Communications*, 2021. 69(6): 3911-3929.
 - [49] Neely, M.J., Stochastic network optimization with application to communication and queueing systems. *Synthesis Lectures on Communication Networks*, 2010. 3(1): 1-211.
 - [50] Xing, H., et al., Joint task assignment and resource allocation for D2D-enabled mobile-edge computing. *IEEE Transactions on Communications*, 2019. 67(6): 4193-4207.
 - [51] Liu, C.-F., et al., Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing. *IEEE Transactions on Communications*, 2019. 67(6): 4132-4150.
 - [52] Tran, T.X. and D. Pompili, Joint task offloading and resource allocation for multi-server mobile-edge computing networks. *IEEE Transactions on Vehicular Technology*, 2018. 68(1): 856-868.
 - [53] Mao, Y., J. Zhang, and K.B. Letaief, Dynamic computation offloading for mobile-edge computing with energy harvesting devices. *IEEE Journal on Selected Areas in Communications*, 2016. 34(12): 3590-3605.
 - [54] Josilo, S. and G. Dán, Computation offloading scheduling for periodic tasks in mobile edge computing. *IEEE/ACM Transactions on Networking*, 2020. 28(2): 667-680.
 - [55] Chen, Y., et al., Energy efficient dynamic offloading in mobile edge computing for internet of things. *IEEE Transactions on Cloud Computing*, 2019. 9(3): 1050-1060.
 - [56] Shu, P., et al. eTime: Energy-efficient transmission between cloud and mobile devices. in 2013 *Proceedings IEEE INFOCOM*. 2013. IEEE.
 - [57] Cui, Y., et al., A survey on delay-aware resource control for wireless systems—Large deviation theory, stochastic Lyapunov drift, and distributed stochastic learning. *IEEE Transactions on Information Theory*, 2012. 58(3): 1677-1701.
 - [58] Chen M, Hao Y. Task offloading for mobile edge computing in software defined ultra-dense network[J]. *IEEE Journal on Selected Areas in Communications*, 2018, 36(3): 587-597.
 - [59] Chen, L., et al., Collaborative Content Placement Among Wireless Edge Caching Stations With Time-to-Live Cache. *IEEE Transactions on Multimedia*, 2019. PP(99): 1-1.
 - [60] Bahn, H. Efficiency of Buffer Caching in Computing-Intensive Workloads. in 2020 7th International Conference on Information Science and Control Engineering (ICISCE).
 - [61] Wu, W., et al., Accuracy-Guaranteed Collaborative DNN Inference in Industrial IoT via Deep Reinforcement Learning. *IEEE Transactions on Industrial Informatics*, 2020. PP(99): 1-1.
 - [62] Ding Y, Liu C, Li K, et al. Task offloading and service migration strategies for user equipments with mobility consideration in mobile edge computing[C]//2019 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCLOUD/SocialCom/SustainCom). IEEE, 2019: 176-183.

-
- [63] Nadembega A, Hafid A S, Brisebois R. Mobility prediction model-based service migration procedure for follow me cloud to support QoS and QoE[C]//2016 IEEE International Conference on Communications (ICC). IEEE, 2016: 1-6.
 - [64] Wang S, Urgaonkar R, He T, et al. Dynamic service placement for mobile micro-clouds with predicted future costs[J]. IEEE [14]Transactions on Parallel and Distributed Systems, 2016, 28(4): 1002-1016.
 - [65] Wu, H., et al., EEDTO: an energy-efficient dynamic task offloading algorithm for blockchain-enabled IoT-edge-cloud orchestrated computing. IEEE Internet of Things Journal, 2020. 8(4): 2163-2176.
 - [66] Ouyang, T., Z. Zhou, and X. Chen, Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing. IEEE Journal on Selected Areas in Communications, 2018.36(10): 2333-2345.
 - [67] Xia, X., et al., Online collaborative data caching in edge computing. IEEE Transactions on Parallel and Distributed Systems, 2020. 32(2): 281-294.
 - [68] Chen, X., et al., Dynamic Service Migration and Request Routing for Microservice in Multi-cell Mobile Edge Computing. IEEE Internet of Things Journal, 2022.
 - [69] Duan, X., F. Xu, and Y. Sun. Research on Offloading Strategy in Edge Computing of Internet of Things. in 2020 International Conference on Computer Network, Electronic and Automation (ICCNEA). 2020. IEEE.
 - [70] Ning, Z., et al., Distributed and Dynamic Service Placement in Pervasive Edge Computing Networks. IEEE Transactions on Parallel and Distributed Systems, 2020. (99).
 - [71] Feng, J., et al., Service characteristics-oriented joint optimization of radio and computing resource allocation in mobile-edge computing. IEEE Internet of Things Journal, 2021. 8(11): 9407-9421.
 - [72] Pan, M. and Z. Li. Multi-user Computation Offloading Algorithm for Mobile Edge Computing. in 2021 2nd International Conference on Electronics, Communications and Information Technology (CECIT). 2021. IEEE.
 - [73] Hu, H., et al. Mobility-aware offloading and resource allocation in MEC-enabled IoT networks. in 2020 16th International Conference on Mobility, Sensing and Networking (MSN). 2020, 1: 23-26.
 - [74] Tang, C., et al. Caching Assisted Correlated Task Offloading for IoT Devices in Mobile Edge Computing. in 2021 IEEE Global Communications Conference (GLOBECOM). 2021. IEEE.
 - [75] Li, Y., et al., Learning-aided computation offloading for trusted collaborative mobile edge computing. IEEE Transactions on Mobile Computing, 2019. 19(12): 2833-2849..
 - [76] Gao, J., et al., A task offloading algorithm for cloud-edge collaborative system based on Lyapunov optimization. Cluster Computing, 2022: 1-12.
 - [77] Cai, Y., et al. Mobile edge computing network control: Tradeoff between delay and cost. in GLOBECOM 2020-2020 IEEE Global Communications Conference. 2020. IEEE.
 - [78] Tong, Z., et al., Dynamic Energy-Saving Offloading Strategy Guided by Lyapunov Optimization for IoT Devices. IEEE Internet of Things Journal, 2022.
 - [79] Bi, S., et al., Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks. IEEE Transactions on Wireless.

- [80] Mao, Y., et al. Power-delay tradeoff in multi-user mobile-edge computing systems. in 2016 IEEE global communications conference (GLOBECOM). 2016. IEEE.
- [81] Zhang, Y., et al., A mobility-aware vehicular caching scheme in content centric networks: Model and optimization. IEEE Transactions on Vehicular Technology, 2019. 68(4): 3100-3112.
- [82] Hu, H., et al., Energy efficiency and delay tradeoff in an mec-enabled mobile iot network. IEEE Internet of Things Journal, 2022. 9(17): 15942-15956.
- [83] You C, Huang K, Chae H, et al. Energy-efficient resource allocation for mobile-edge computation offloading[J]. IEEE Transactions on Wireless Communications, 2016, 16(3): 1397-1411.
- [84] Fan, Q., et al. Joint Service Caching and Computation Offloading to Maximize System Profits in Mobile Edge-Cloud Computing. in 2020 16th International Conference on Mobility, Sensing and Networking (MSN). 2020.
- [85] Zhang, N., et al., Joint task offloading and data caching in mobile edge computing networks. Computer Networks, 2020. 182: 107446
- [86] Han, Seungyul, and Youngchul Sung. "Diversity actor-critic: Sample-aware entropy regularization for sample-efficient exploration." International Conference on Machine Learning. PMLR, 2021.
- [87] Zhuang Z, Wang J, Qi Q, et al. Adaptive and robust routing with Lyapunov-based deep RL in MEC networks enabled by blockchains[J]. IEEE Internet of Things Journal, 2020, 8(4): 2208-2225.
- [88] Y. Dai, K. Zhang, S. Maharjan, and Y. Zhang, "Deep reinforcement learning for stochastic computation offloading in digital twin networks," IEEE Trans. Ind. Inform., 2021, 17(7), 4968 – 4977.
- [89] Ansere J A, Gyamfi E, Li Y, et al. Optimal Computation Resource Allocation in Energy-Efficient Edge IoT Systems with Deep Reinforcement Learning[J]. IEEE Transactions on Green Communications and Networking, 2023.
- [90] Tong, Z., et al., DDQN-TS: A novel bi-objective intelligent scheduling algorithm in the cloud environment. Neurocomputing, 2021. 455: 419-430.
- [91] Tang H, Wu H, Zhao Y, et al. Joint computation offloading and resource allocation under task-overflowed situations in mobile-edge computing[J]. IEEE Transactions on Network and Service Management, 2021, 19(2): 1539-1553.
- [92] Zhang G, Zhang W, Cao Y, et al. Energy-delay tradeoff for dynamic offloading in mobile-edge computing system with energy harvesting devices[J]. IEEE Transactions on Industrial Informatics, 2018, 14(10): 4642-4655.
- [93] Salodkar N, Bhorkar A, Karandikar A, et al. An online learning algorithm for energy efficient delay constrained scheduling over a fading channel[J]. IEEE Journal on Selected Areas in Communications, 2008, 26(4): 732-742.
- [94] Lin R, Xie T, Luo S, et al. Energy-efficient computation offloading in collaborative edge computing[J]. IEEE Internet of Things Journal, 2022, 9(21): 21305-21322.

-
- [95] J. L. D. Neto, S. Yu, D. F. Macedo, J. M. S. Nogueira, R. Langar, and S. Secci, "ULOOF: A user level online offloading framework for mobile edge computing," *IEEE Transactions on Mobile Computing*, 2018, 17(11): 2660-2674.
- [96] M. Neely, E. Modiano, and C.p. Li, "Fairness and optimal stochastic control for heterogeneous networks," *IEEE/ACM Transactions on Networking*, 2008, 16, 396-409.
- [97] M. J. Neely, "Energy optimal control for time-varying wireless networks," *IEEE Transactions on Information Theory*, 2006, 52 (7): 2915-2934.
- [98] L. Georgiadis, M. J. Neely, and L. Tassiulas, "Resource allocation and cross-layer control in wireless networks," *FNT Netw.*, 2005, 1(1): 1-144.
- [99] R. Li, Z. Zhao, Q. Sun, I. Chih-Lin, C. Yang, X. Chen, M. Zhao, and H. Zhang, "Deep reinforcement learning for resource management in network slicing," *IEEE Access*, 2018, 6: 74 429-74 441.
- [100] Y. He, N. Zhao, and H. Yin, "Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach," *IEEE Transactions on Vehicular Technology*, 2017, 67(1): 44-55 .
- [101] X. Chen, Z. Zhao, C. Wu, M. Bennis, H. Liu, Y. Ji, and H. Zhang, "Multi-tenant cross-slice resource orchestration: A deep reinforcement learning approach," *IEEE Journal on Selected Areas in Communications*, 2019, 37(10): 2377-2392.
- [102] U. Challita, L. Dong, and W. Saad, "Proactive resource management for LTE in unlicensed spectrum: A deep learning perspective," *IEEE Transactions on Wireless Communications*, 2018, 17(7): 4674-4689.
- [103] Haarnoja, Tuomas, et al. "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor." *International conference on machine learning*. PMLR, 2018.
- [104] Neely M J. A simple convergence time analysis of drift-plus-penalty for stochastic optimization and convex programs[J]. *arXiv preprint arXiv:1412.0791*, 2014.
- [105] Bertsekas D P, Nedic A, Ozdaglar A E. Convex analysis and optimization athena scientific[J]. *Journal of Mathematical Analysis & Applications*, 2003, 129(2): 420-432.